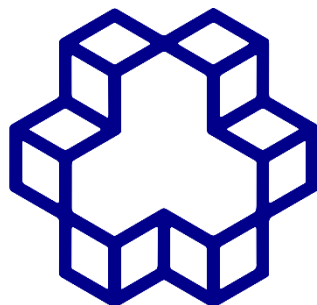




دانشگاه صنعتی خواجه نصیرالدین طوسی
دانشکده مهندسی برق

گزارش پروژه پایانی درس یادگیری تقویتی

طراحی و تحلیل عامل هوشمند در هزارتوی پویا

مبینا اسکندری نیا

۴۰۳۰۱۶۹۴

مخزن پروژه (GitHub):

https://github.com/Mobino80/RL_FinalProject_40301694

استاد درس

دکتر سید حسین خواسته

تابستان ۱۴۰۵

فهرست

فصل اول: مقدمه	۱۲
۱-۱. یادگیری تقویتی	۱۲
۱-۲. اهمیت مسئله	۱۳
۱-۳. هدف پروژه	۱۵
فصل دوم: تعریف مسئله	۱۷
۲-۱. معرفی محیط هزارتو	۱۷
۲-۲. مأموریت عامل	۱۸
۲-۳. قوانین محیط	۱۹
۲-۳-۱. انتخاب و اجرای عمل	۱۹
۲-۳-۲. برخورد با دیوار و مرز محیط	۱۹
۲-۳-۳. خانه‌های جریمه	۱۹
۲-۳-۴. دریافت کلید	۲۰
۲-۳-۵. در قفل شده	۲۰
۲-۳-۶. رسیدن به هدف	۲۰
۲-۳-۷. ثبت رخدادهای محیط	۲۰
۲-۴. قابلیت تکمیلی: انرژی محدود	۲۱
۲-۵. تعریف فضای حالت	۲۲
۲-۶. فضای عمل	۲۳
۲-۷. حفظ خاصیت مارکوف	۲۳
۲-۸. شرایط پایان اپیزود	۲۴
۲-۸-۱. رسیدن موفق به هدف	۲۵
۲-۸-۲. پایان انرژی	۲۵
۲-۸-۳. رسیدن به سقف تعداد گام‌ها	۲۵
فصل سوم: مدل‌سازی فرایند تصمیم‌گیری مارکوف	۲۶
۳-۱. فضای حالت	۲۶
۳-۱-۱. محدودیت‌های فضای حالت	۲۷

۲۸	۳-۲. فضای عمل.....
۲۸	۳-۳. تابع انتقال.....
۲۹	۳-۳-۱. انتقال مکانی.....
۲۹	۳-۳-۲. انتقال وضعیت کلید.....
۳۰	۳-۳-۳. انتقال انرژی.....
۳۰	۳-۳-۴. تجميع انتقال‌های یکسان.....
۳۱	۳-۴. تابع پاداش.....
۳۱	۳-۴-۱. پاداش تنک.....
۳۲	۳-۴-۲. پاداش شکل‌دهی‌شده.....
۳۳	۳-۵. ضریب تنزیل.....
۳۴	۳-۶. حالت‌های پایانی.....
۳۴	۳-۶-۱. رسیدن به هدف.....
۳۵	۳-۶-۲. پایان انرژی.....
۳۵	۳-۶-۳. محدودیت تعداد گام‌ها.....
۳۵	۳-۷. سیاست.....
۳۶	۳-۷-۱. سیاست در Value Iteration
۳۶	۳-۷-۲. سیاست رفتاری در الگوریتم‌های مدل آزاد.....
۳۷	۳-۸. تابع ارزش حالت و عمل.....
۳۸	۳-۹. حفظ خاصیت مارکوف.....
۳۸	۳-۱۰. تفکیک مدل محور و مدل آزاد.....
۳۹	۳-۱۱. هدف بهینه‌سازی.....
۴۰	فصل چهارم: طراحی و پیاده‌سازی محیط.....
۴۰	۴-۱. رویکرد طراحی محیط.....
۴۰	۴-۲. ساختار کلی فایل‌های مرتبط با محیط.....
۴۱	۴-۳. نمایش عددی خانه‌های نقشه.....
۴۱	۴-۴. تولید نقشه مبدأ.....
۴۱	۴-۴-۱. تعیین seed و اندازه نقشه.....
۴۲	۴-۴-۲. ایجاد ماتریس اولیه.....

۴۲	۴-۵. طراحی محفظه هدف و در
۴۳	۴-۶. تعیین نقطه شروع
۴۳	۴-۷. انتخاب محل کلید
۴۴	۴-۸. تولید دیوارها
۴۵	۴-۹. تولید خانه‌های جریمه
۴۵	۴-۱۰. تولید قطعی و بازتولیدپذیر نقشه
۴۶	۴-۱۱. اعتبارسنجی نقشه با BFS
۴۶	۴-۱۱-۱. نقش BFS در پروژه
۴۶	۴-۱۱-۲. شرط دسترسی از شروع به کلید
۴۶	۴-۱۱-۳. شرط دسترسی از کلید به هدف
۴۷	۴-۱۱-۴. جلوگیری از دسترسی بدون کلید
۴۷	۴-۱۱-۵. بررسی تناسب مسیر با انرژی
۴۸	۴-۱۲. ذخیره نقشه در قالب JSON
۴۹	۴-۱۳. طراحی کلاس MazeEnv
۴۹	۴-۱۴. شناسایی خانه‌های قابل عبور
۵۰	۴-۱۵. تابع reset
۵۰	۴-۱۶. تابع step
۵۱	۴-۱۷. تابع انتقال قطعی داخلی
۵۱	۴-۱۸. دسترسی مدل محور با تابع transitions
۵۲	۴-۱۹. تولید فضای حالت کامل
۵۳	۴-۲۰. محاسبه فاصله‌های BFS برای شکل‌دهی پاداش
۵۳	۴-۲۱. ثبت رخدادها با MazeLogger
۵۴	۴-۲۲. بازتولیدپذیری انتقال‌های تصادفی
۵۴	۴-۲۳. آزمون‌های واحد محیط
۵۵	۴-۲۴. نحوه استفاده محیط در پروژه
۵۵	۴-۲۵. ارزیابی طراحی محیط
۵۷	فصل پنجم: طراحی و ارزیابی تابع پاداش
۵۷	۵-۱. چارچوب طراحی تابع پاداش

۵۷	۵-۲. اهداف طراحی تابع پاداش.....
۵۸	۵-۳. تابع پاداش تنک.....
۵۹	۵-۴. منطق انتخاب مقادیر پاداش.....
۵۹	۵-۴-۱. هزینه هر گام.....
۵۹	۵-۴-۲. جریمه برخورد با دیوار و در قفل شده.....
۵۹	۵-۴-۳. جریمه خانه‌های خطرناک.....
۶۰	۵-۴-۴. پاداش کلید.....
۶۰	۵-۴-۵. پاداش هدف.....
۶۰	۵-۴-۶. جریمه پایان انرژی.....
۶۱	۵-۵. محدودیت پاداش تنک.....
۶۱	۵-۶. تابع پاداش شکل‌دهی شده.....
۶۲	۵-۷. اثر حرکت بر پاداش شکل‌دهی.....
۶۳	۵-۸. حفظ سیاست بهینه در شکل‌دهی مبتنی بر پتانسیل.....
۶۴	۵-۹. مقایسه تجربی دو تابع پاداش.....
۶۴	۵-۹-۱. تنظیمات مقایسه.....
۶۵	۵-۹-۲. سرعت آغاز یادگیری.....
۶۵	۵-۹-۳. ارزیابی سیاست نهایی.....
۶۶	۵-۹-۴. محدودیت مقایسه بازده عددی.....
۶۶	۵-۱۰. تحلیل رفتارهای ناخواسته.....
۶۶	۵-۱۰-۱. حرکت‌های حلقه‌ای.....
۶۷	۵-۱۰-۲. دریافت مکرر پاداش کلید.....
۶۷	۵-۱۰-۳. تکرار پاداش هدف.....
۶۷	۵-۱۰-۴. ثابت ماندن در یک خانه.....
۶۷	۵-۱۰-۵. ترجیح کوتاه‌ترین مسیر بدون توجه به خطر.....
۶۸	۵-۱۰-۶. تمرکز بیش از حد بر کلید.....
۶۸	۵-۱۰-۷. تفسیر پاداش انتقال پایانی.....
۶۸	۵-۱۱. انتخاب تابع پاداش مرجع.....
۷۰	فصل ششم: الگوریتم Value Iteration و تحلیل نتایج.....

۶-۱. چارچوب اجرای Value Iteration	۷۰
۶-۲. تنظیمات آزمایش	۷۰
۶-۳. تحلیل همگرایی	۷۱
۶-۴. اثر ضریب تنزیل بر مقدار حالت اولیه	۷۳
۶-۵. اثر ضریب تنزیل بر سیاست نهایی	۷۴
۶-۶. تحلیل Heatmap پیش از دریافت کلید	۷۴
۶-۷. تغییر تابع ارزش پس از دریافت کلید	۷۶
۶-۸. ارزیابی سیاست مرجع	۷۸
۶-۹. زمان محاسباتی	۷۹
۶-۱۰. انتخاب سیاست مرجع	۷۹
۶-۱۱. جمع‌بندی نتایج	۸۰
فصل هفتم: الگوریتم Q-Learning و تحلیل نتایج	۸۱
۷-۱. چارچوب اجرای Q-Learning	۸۱
۷-۲. تنظیمات آموزش	۸۱
۷-۳. تفاوت عملی برنامه‌های خطی و نمایی	۸۲
۷-۴. روند یادگیری با پاداش تنک	۸۳
۷-۵. نتایج ارزیابی نهایی	۸۴
۷-۶. تفاوت عملکرد آموزش و ارزیابی	۸۵
۷-۷. پوشش فضای حالت	۸۵
۷-۸. تحلیل مقادیر Q و سیاست مدل منتخب	۸۶
۷-۸-۱. وضعیت پیش از دریافت کلید	۸۸
۷-۸-۲. وضعیت پس از دریافت کلید	۸۹
۷-۹. بازسازی دستی یک به‌روزرسانی واقعی Q	۸۹
۷-۱۰. میزان توافق با سیاست Value Iteration	۹۱
۷-۱۱. انتخاب مدل نهایی Q-Learning	۹۱
۷-۱۲. جمع‌بندی نتایج	۹۲
فصل هشتم: الگوریتم $SARSA(\lambda)$ و تحلیل نتایج	۹۳
۸-۱. چارچوب اجرای $SARSA(\lambda)$	۹۳

۹۳	۸-۲. Eligibility Trace و نقش پارامتر λ
۹۴	۸-۳. نتایج اصلی برای مقادیر مختلف λ
۹۵	۸-۴. سرعت یادگیری برای مقادیر مختلف λ
۹۶	۸-۵. تحلیل ایمنی و کیفیت مسیر.....
۹۷	۸-۶. پویایی Eligibility Trace.....
۹۸	۸-۷. بررسی عددی چند گام متوالی.....
۹۹	۸-۸. علت افت شدید عملکرد در $\lambda=0.9$
۱۰۰	۸-۹. Accumulating Trace و Replacing.....
۱۰۱	۸-۱۰. سیاست حاصل از مدل منتخب.....
۱۰۴	۸-۱۱. جمع‌بندی نتایج.....
۱۰۵	فصل نهم: مقایسه جامع الگوریتم‌ها و تحلیل سیاست‌ها.....
۱۰۵	۹-۱. چارچوب مقایسه.....
۱۰۶	۹-۲. مقایسه زمان اجرا و هزینه محاسباتی.....
۱۰۷	۹-۳. کارایی نمونه‌ای و سرعت یادگیری.....
۱۰۸	۹-۴. پایداری بین اجراهای مستقل.....
۱۰۹	پایداری Value Iteration.....
۱۱۰	۹-۵. مقایسه حافظه مصرفی.....
۱۱۰	۹-۶. کیفیت مسیر حاصل.....
۱۱۲	۹-۷. میزان شباهت سیاست‌های مدل آزاد با Value Iteration.....
۱۱۴	۹-۸. تحلیل سه حالت نمونه دارای اختلاف سیاست.....
۱۱۴	۹-۸-۱. حالت اول: اختلاف Q-Learning در نقطه شروع.....
۱۱۵	۹-۸-۲. حالت دوم: اختلافی با اعمال تقریباً هم‌ارزش.....
۱۱۶	۹-۸-۳. حالت سوم: اختلاف مهم در نزدیکی هدف.....
۱۱۷	۹-۹. حساسیت الگوریتم‌ها به پارامترها.....
۱۱۸	۹-۱۰. جمع‌بندی مقایسه.....
۱۲۰	فصل دهم: انتقال یادگیری در Q-Learning و تحلیل نتایج.....
۱۲۰	۱۰-۱. چارچوب آزمایش انتقال یادگیری.....
۱۲۰	۱۰-۲. محیط‌های مقصد.....

۱۰-۳. سناریوهای انتقال دانش	۱۲۲
آموزش از صفر	۱۲۲
انتقال کامل	۱۲۲
انتقال مقیاس شده	۱۲۳
انتقال انتخابی	۱۲۳
۱۰-۴. معیارهای ارزیابی انتقال	۱۲۳
۱۰-۵. نتایج انتقال در محیط مشابه	۱۲۴
۱۰-۵-۱. برتری آشکار انتقال نسبت به آموزش از صفر	۱۲۵
۱۰-۵-۲. اثر ضریب β	۱۲۵
۱۰-۵-۳. انتقال انتخابی	۱۲۶
۱۰-۶. منحنی‌های یادگیری محیط مشابه	۱۲۶
۱۰-۷. نتایج انتقال در محیط متفاوت	۱۲۷
۱۰-۷-۱. انتقال منفی در عملکرد آغازین	۱۲۷
۱۰-۷-۲. سرعت یادگیری: مزیت انتقال با وجود Jumpstart منفی	۱۲۸
۱۰-۷-۳. عملکرد نهایی و باقی ماندن اثر منفی	۱۲۹
۱۰-۸. موفقیت انتقال انتخابی در محیط متفاوت	۱۲۹
۱۰-۹. منحنی‌های یادگیری محیط متفاوت	۱۳۰
۱۰-۱۰. نمونه واقعی انتقال منفی و اصلاح آن	۱۳۰
۱۰-۱۱. میزان تغییر جدول Q پس از سازگاری	۱۳۲
۱۰-۱۲. مقایسه انتقال مثبت و منفی در دو محیط	۱۳۳
۱۰-۱۳. اثر شباهت محیط بر راهبرد مناسب انتقال	۱۳۳
۱۰-۱۴. جمع‌بندی نتایج	۱۳۴
فصل یازدهم: رابط گرافیکی و نمایش تعاملی عامل	۱۳۶
۱۱-۱. طراحی رابط گرافیکی	۱۳۶
۱۱-۲. ساختار کلی رابط	۱۳۶
۱۱-۳. نمایش عناصر محیط	۱۳۷
۱۱-۴. نمایش پویای وضعیت کلید و در	۱۳۸
۱۱-۵. نمایش محدودیت انرژی	۱۳۸

۱۳۹	۱۱-۶. نمایش رخدادهای محیط
۱۴۰	۱۱-۷. انتخاب الگوریتم و محیط
۱۴۰	۱۱-۸. Evaluation, Training و حالت‌های
۱۴۰	Evaluation
۱۴۰	Training
۱۴۱	۱۱-۹. آموزش سریع یا Fast-forward
۱۴۱	۱۱-۱۰. کنترل اجرای اپیزود
۱۴۲	۱۱-۱۱. نمایش مسیر طی شده عامل
۱۴۲	۱۱-۱۲. نمایش سیاست حریمانه
۱۴۳	۱۱-۱۳. نحوه نمایش سیاست Value Iteration و روش‌های مدل آزاد
۱۴۴	۱۱-۱۴. اطلاعات لحظه‌ای
۱۴۴	۱۱-۱۵. اجرای Value Iteration در رابط
۱۴۵	۱۱-۱۶. هماهنگی GUI با منطق اصلی پروژه
۱۴۵	۱۱-۱۷. انطباق با الزامات رابط گرافیکی پروژه
۱۴۷	فصل دوازدهم: پاسخ به پرسش‌های تحلیلی
۱۴۷	۱۲-۱. تعریف مسئله به صورت MDP و حفظ خاصیت مارکوف
۱۴۸	۱۲-۲. تفاوت on-policy و off-policy در رفتار واقعی SARSA و Q-Learning
۱۵۰	۱۲-۳. علت نیاز Value Iteration به مدل انتقال و یادگیری بدون مدل در SARSA و Q-Learning
۱۵۱	۱۲-۴. بهترین مقدار λ از نظر سرعت یادگیری و پایداری
۱۵۲	۱۲-۵. سه حالت دارای اختلاف میان سیاست مدل آزاد و Value Iteration
۱۵۲	حالت اول: اختلاف Q-Learning در حالت شروع
۱۵۳	حالت دوم: اختلاف SARSA میان دو تصمیم تقریباً هم‌ارزش
۱۵۴	حالت سوم: اختلاف مهم SARSA در نزدیکی هدف
۱۵۵	۱۲-۶. تفاوت انتقال یادگیری در محیط مشابه و محیط متفاوت
۱۵۵	عملکرد اولیه
۱۵۶	سرعت یادگیری
۱۵۶	عملکرد نهایی
۱۵۷	نمونه انتقال منفی
۱۵۹	فصل سیزدهم: جمع‌بندی و نتیجه‌گیری

۱۵۹	۱۳-۱. جمع‌بندی کلی پروژه
۱۵۹	۱۳-۲. نتایج اصلی الگوریتم‌ها
۱۶۰	۱۳-۳. مقایسه نهایی روش‌های اصلی
۱۶۰	۱۳-۴. نتایج انتقال یادگیری
۱۶۱	۱۳-۵. نکات حاصل از تحلیل تجربی
۱۶۲	۱۳-۶. محدودیت‌های پروژه
۱۶۲	۱۳-۷. پیشنهادهایی برای توسعه آینده
۱۶۳	۱۳-۸. نتیجه‌گیری نهایی
۱۶۴	پیوست‌ها
۱۶۴	پیوست الف: تنظیمات نهایی محیط و آزمایش‌ها
۱۶۴	الف-۱. پارامترهای محیط
۱۶۵	الف-۲. پارامترهای Value Iteration
۱۶۶	الف-۳. پارامترهای Q-Learning
۱۶۶	الف-۴. پارامترهای SARSA(λ)
۱۶۷	الف-۵. تنظیمات آزمایش پایداری
۱۶۷	الف-۶. تنظیمات انتقال یادگیری
۱۶۹	پیوست ب: ساختار مخزن پروژه
۱۷۰	پیوست ج: نصب و بازتولید نتایج
۱۷۰	ج-۱. نیازمندی‌های نرم‌افزاری
۱۷۰	ج-۲. ترتیب بازتولید نتایج
۱۷۱	پیوست د: فایل‌های اصلی خروجی
۱۷۱	د-۱. خروجی‌های Value Iteration
۱۷۲	د-۲. خروجی‌های Q-Learning
۱۷۳	د-۳. خروجی‌های SARSA(λ)
۱۷۳	د-۴. خروجی‌های مقایسه و پایداری
۱۷۳	د-۵. خروجی‌های انتقال یادگیری
۱۷۴	پیوست ه: آزمون‌های صحت پیاده‌سازی
۱۷۵	پیوست و: استفاده از ابزارهای هوش مصنوعی

پیوست ز: منابع اصلی مورد استفاده ۱۷۷

فصل اول: مقدمه

۱-۱. یادگیری تقویتی

یادگیری تقویتی یکی از شاخه‌های یادگیری ماشین است که به مطالعه تصمیم‌گیری متوالی در محیط‌های ناشناخته یا دارای عدم قطعیت می‌پردازد. در این چارچوب، یک عامل با محیط تعامل می‌کند، در هر مرحله وضعیت فعلی را مشاهده می‌کند و عملی را از مجموعه اعمال مجاز برمی‌گزیند. محیط پس از اجرای عمل، به حالت جدیدی منتقل می‌شود و یک پاداش عددی به عامل باز می‌گرداند. عامل با استفاده از تجربه‌های حاصل از این تعامل تلاش می‌کند سیاستی بیاموزد که بازده مورد انتظار را در طول زمان بیشینه کند.

تعامل عامل و محیط را می‌توان به صورت دنباله زیر نمایش داد:

$$St \rightarrow At \rightarrow Rt + 1, St + 1$$

روش‌های یادگیری تقویتی را می‌توان از نظر دسترسی به مدل محیط به دو گروه کلی تقسیم کرد. در روش‌های مدل‌محور، احتمال انتقال میان حالت‌ها و پاداش هر انتقال در اختیار الگوریتم قرار دارد. الگوریتم می‌تواند بدون نیاز به تجربه مستقیم گسترده، پیامد اعمال را با استفاده از مدل محاسبه کند. در مقابل، روش‌های مدل‌آزاد به مدل صریح محیط دسترسی ندارند و تابع ارزش یا مقدار اعمال را از نمونه‌های حاصل از تعامل می‌آموزند.

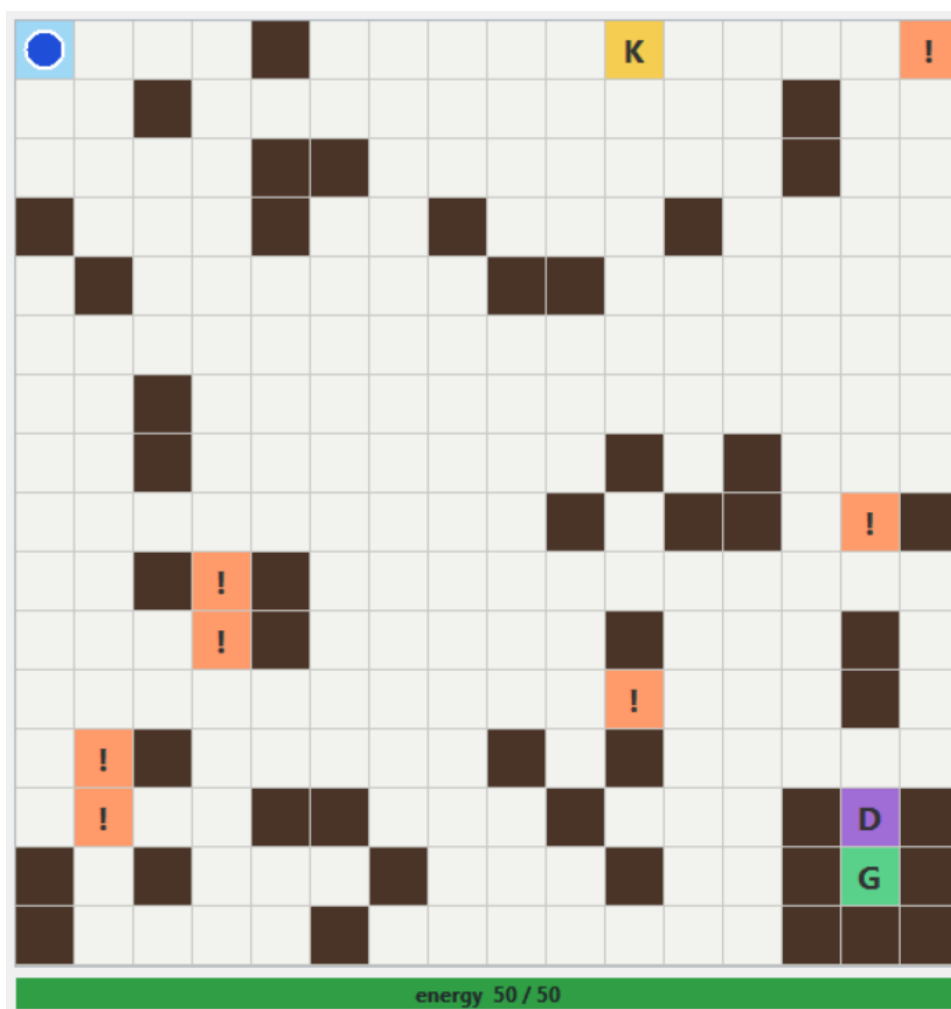
در این پروژه، الگوریتم Value Iteration به عنوان روش مدل‌محور استفاده شده است. این الگوریتم با اجرای مکرر به‌روزرسانی بلمن، تابع ارزش بهینه را محاسبه و سپس سیاست حریصانه متناظر را استخراج می‌کند. الگوریتم‌های Q-Learning و SARSA(λ) دو روش مدل‌آزاد مورد بررسی هستند. Q-Learning الگوریتم off-policy است؛ زیرا مقدار عمل جاری را بر اساس بهترین عمل قابل انتخاب در حالت بعدی به‌روزرسانی می‌کند، حتی اگر سیاست رفتاری عامل دیگری را انتخاب کند. در مقابل، SARSA(λ) یک روش on-policy است و به‌روزرسانی آن به عمل واقعی انتخاب‌شده در حالت بعدی وابسته است.

در SARSA(λ) از ردپای شایستگی (Eligibility Traces) برای انتقال خطای اختلاف زمانی به حالت‌ها و اعمال قبلی استفاده می‌شود. پارامتر λ تعیین می‌کند که اثر تجربه جدید تا چه اندازه در مسیر پیموده‌شده به عقب منتشر شود. هنگامی که $\lambda=0$ باشد، الگوریتم به SARSA یک‌مرحله‌ای نزدیک می‌شود. افزایش λ می‌تواند انتشار اطلاعات را سریع‌تر کند، اما مقادیر بسیار بزرگ ممکن است نوسان به‌روزرسانی‌ها را افزایش دهند یا در محیط‌هایی با پاداش‌ها و انتقال‌های تصادفی، پایداری یادگیری را کاهش دهند.

۲-۱. اهمیت مسئله

محیط‌های هزارتو نمونه‌ای قابل کنترل برای مطالعه مسائل تصمیم‌گیری متوالی هستند. اگر حرکت عامل قطعی باشد، هدف در یک موقعیت ثابت قرار داشته باشد و هیچ محدودیتی برای تعداد حرکات وجود نداشته باشد، مسئله را می‌توان با یک الگوریتم جست‌وجوی مسیر حل کرد. مسئله این پروژه شرایط متفاوتی دارد و عامل باید تحت عدم قطعیت، محدودیت انرژی و وابستگی میان زیرهدف‌ها تصمیم بگیرد.

مأموریت عامل شامل سه مرحله وابسته به یکدیگر است. عامل ابتدا باید کلید را پیدا کند، سپس از در قفل شده عبور کند و در پایان به خانه هدف برسد. بنابراین، موقعیت مکانی به‌تنهایی برای توصیف وضعیت کافی نیست. دو عامل ممکن است در یک خانه یکسان قرار داشته باشند، اما اگر یکی کلید را دریافت کرده باشد و دیگری فاقد کلید باشد، اعمال مجاز و پیامدهای آینده آن‌ها یکسان نخواهد بود.



شکل ۱-۱: نمای کلی نقشه مبدأ پروژه

ویژگی تکمیلی انتخاب شده در این پروژه، انرژی محدود است. عامل در ابتدای هر اپیزود ۵۰ واحد انرژی دارد و هر عمل، حتی برخورد با دیوار یا تلاش ناموفق برای عبور از در بسته، یک واحد انرژی مصرف می کند. در نتیجه، انتخاب مسیرهای طولانی، تکرار حرکت های ناموفق و اکتشاف کنترل نشده می تواند پیش از رسیدن به هدف موجب پایان اپیزود شود. مقدار انرژی باقی مانده نیز بر امکان تکمیل مأموریت اثر دارد و باید بخشی از حالت محیط باشد.

حرکت عامل قطعی نیست. عمل انتخاب شده با احتمال 0.8 مطابق جهت موردنظر اجرا می شود و با احتمال 0.1 به هر یک از دو جهت عمود منحرف می گردد. از این رو، سیاستی که صرفاً کوتاه ترین مسیر هندسی را دنبال می کند، ممکن است عامل را در مجاورت دیوارها یا خانه های جریمه در معرض خطر بیشتری قرار دهد. عامل باید کیفیت مسیر را با احتمال انحراف و هزینه رخدادهای نامطلوب بسنجد.

نقشه اصلی پروژه با استفاده از seed برابر ۹، برگرفته از رقم یکی مانده به آخر شماره دانشجویی، تولید شده است. بر اساس رابطه تعیین شده برای اندازه محیط، هزارتوی حاصل دارای ابعاد 16×16 است. استفاده از seed ثابت و ذخیره نقشه در قالب فایل مستقل سبب می شود تمام الگوریتم ها روی محیط یکسان اجرا شوند و تفاوت نتایج به تغییر تصادفی ساختار نقشه وابسته نباشد.

این مسئله امکان مقایسه روش های مدل محور و مدل آزاد را در شرایط کنترل شده فراهم می کند. Value Iteration از مدل کامل انتقال استفاده می کند و یک سیاست مرجع برای ارزیابی روش های یادگیری ارائه می دهد. Q-Learning و $SARSA(\lambda)$ باید همان ساختار تصمیم گیری را تنها از طریق تعامل با محیط بیاموزند. میزان موفقیت، تعداد نمونه های آموزشی، زمان اجرا، مصرف حافظه، پایداری میان اجراهای مستقل و شباهت سیاست های مدل آزاد با سیاست مرجع، جنبه های متفاوت عملکرد این الگوریتم ها را مشخص می کنند.

طراحی تابع پاداش نیز بخشی از مسئله است. در پاداش تنک، بازخوردهای مثبت اصلی هنگام دریافت کلید و رسیدن به هدف ارائه می شوند و عامل در مراحل میانی اطلاعات محدودی درباره مناسب بودن جهت حرکت دریافت می کند. این وضعیت می تواند یادگیری را کند کند، زیرا موفقیت کامل در مراحل ابتدایی آموزش به ندرت رخ می دهد. در نسخه شکل دهی شده، یک مؤلفه مبتنی بر تغییر فاصله تا زیرهدف جاری به پاداش افزوده می شود. مقایسه این دو حالت باید مشخص کند که بازخورد میانی چه اثری بر سرعت یادگیری، مسیر نهایی و احتمال بروز رفتارهای ناخواسته دارد.

انتقال یادگیری بعد دیگری از مسئله را تشکیل می دهد. دانشی که عامل در نقشه مبدأ آموخته است، ممکن است در محیطی با تغییرات محدود مفید باشد؛ اما انتقال مستقیم همان دانش به نقشه ای با تغییرات گسترده می تواند عامل را به اعمال نامناسب هدایت کند. بررسی انتقال کامل، انتقال تعدیل شده و انتقال انتخابی امکان مطالعه رابطه

میان شباهت محیط‌ها و سودمندی دانش قبلی را فراهم می‌کند. نمونه‌های انتقال منفی نیز نشان می‌دهند که مقدار بالای یک عمل در محیط مبدأ، در صورت تغییر ساختار محلی محیط مقصد، لزوماً معتبر باقی نمی‌ماند.

۳-۱. هدف پروژه

هدف اصلی پروژه، طراحی و تحلیل یک عامل هوشمند برای انجام مأموریت چندمرحله‌ای در یک هزارتوی تصادفی با محدودیت انرژی است. محیط به صورت یک فرایند تصمیم‌گیری مارکوف پیاده‌سازی شده و تمام متغیرهایی که بر انتقال‌ها و پاداش‌های آینده اثر دارند، در نمایش حالت قرار گرفته‌اند. عامل باید با در نظر گرفتن وضعیت کلید، انرژی باقی‌مانده، موانع، خانه‌های جریمه و عدم قطعیت حرکت، سیاستی برای رسیدن به هدف بیاموزد.

نخستین هدف فنی، طراحی محیطی مستقل و قابل بازتولید است. نقشه باید سهم تعیین‌شده‌ای از دیوارها و خانه‌های جریمه را داشته باشد و مسیر معتبر شروع تا کلید و کلید تا هدف در آن وجود داشته باشد. برای این منظور، نقشه به صورت قطعی تولید و با BFS اعتبارسنجی می‌شود. استفاده از BFS تنها به کنترل اعتبار نقشه محدود است و در تصمیم‌گیری عامل نقشی ندارد.

هدف دوم، پیاده‌سازی مستقل سه الگوریتم Value Iteration، Q-Learning و $SARSA(\lambda)$ است. در Value Iteration، اثر مقادیر مختلف ضریب تنزیل بر همگرایی و سیاست حاصل بررسی می‌شود. در Q-Learning، برنامه‌های خطی و نمایی کاهش ϵ و همچنین دو تعریف پاداش تنک و شکل‌دهی‌شده مقایسه می‌شوند. برای $SARSA(\lambda)$ ، مقادیر مختلف λ ارزیابی می‌شوند تا رابطه میان طول ردپای شایستگی، سرعت انتشار اطلاعات و پایداری آموزش مشخص شود.

هدف سوم، انجام مقایسه‌ای فراتر از پاداش نهایی است. معیارهای ارزیابی شامل نرخ موفقیت، بازده میانگین، تعداد گام، زمان آموزش، تعداد حالت‌های بازدیدشده، مصرف حافظه، تعداد برخورد با دیوار، ورود به خانه‌های جریمه و پایداری میان seedهای مختلف هستند. سیاست‌های Q-Learning و $SARSA(\lambda)$ نیز با سیاست مرجع Value Iteration مقایسه می‌شوند و حالت‌های دارای تصمیم متفاوت به صورت محلی تحلیل خواهند شد.

هدف دیگر پروژه، بررسی انتقال یادگیری Q-Learning از محیط مبدأ به دو محیط مقصد است. محیط مقصد مشابه دارای تغییرات محدود در ساختار موانع است، در حالی که محیط متفاوت شامل جابه‌جایی گسترده‌تر موانع، تغییر محل کلید و افزودن خانه‌های جریمه جدید است. عملکرد آموزش از صفر با انتقال کامل، انتقال تعدیل‌شده و انتقال انتخابی مقایسه می‌شود. این مقایسه عملکرد اولیه، سرعت یادگیری، نتیجه نهایی و نمونه‌های انتقال منفی را در بر می‌گیرد.

برای مشاهده رفتار عامل، یک رابط گرافیکی مستقل نیز در پروژه پیاده‌سازی شده است. این رابط امکان انتخاب الگوریتم و محیط، اجرای حالت آموزش یا ارزیابی، کنترل سرعت، توقف و ادامه اجرا و نمایش سیاست را فراهم می‌کند. وضعیت عامل، کلید، در، انرژی، پاداش و رخدادهای محیط به‌صورت مرحله‌به‌مرحله نمایش داده می‌شوند.

تمام آزمایش‌ها با تنظیمات ثبت‌شده اجرا شده‌اند. داده‌های هر اپیزود، نمونه‌هایی از انتقال‌ها، جزئیات به‌روزرسانی‌های Q ، تغییرات ردپای شایستگی، مدل‌های آموزش‌دیده و نمودارهای نهایی در پوشه‌های جداگانه ذخیره شده‌اند. آزمایش پایداری نیز با چند seed مستقل انجام شده است. افزون بر این، آزمون‌های واحد برای تولید نقشه، احتمال‌های انتقال، مصرف انرژی، سازوکار کلید و در، شرایط پایان و بازتولیدپذیری محیط در نظر گرفته شده‌اند.

فصل دوم: تعریف مسئله

۲-۱. معرفی محیط هزارتو

مسئله این پروژه، هدایت یک عامل هوشمند در یک هزارتوی دوبعدی، تصادفی و دارای محدودیت انرژی است. عامل حرکت خود را از خانه شروع آغاز می‌کند و برای تکمیل مأموریت باید ابتدا کلید را به دست آورد، سپس از در قفل‌شده عبور کند و در نهایت به خانه هدف برسد. ترتیب این مراحل ثابت است و رسیدن به در یا هدف بدون دریافت کلید، مأموریت را تکمیل نمی‌کند.

نقشه اصلی پروژه با استفاده از شماره دانشجویی 40301694 تولید شده است. رقم یکی‌مانده به آخر این شماره برابر ۹ است و به عنوان seed پایه در نظر گرفته می‌شود. اندازه هزارتو نیز مطابق رابطه تعیین شده در صورت پروژه محاسبه شده است:

$$N = 15 + (base_{seed} \bmod 4) = 15 + (9 \bmod 4) = 16$$

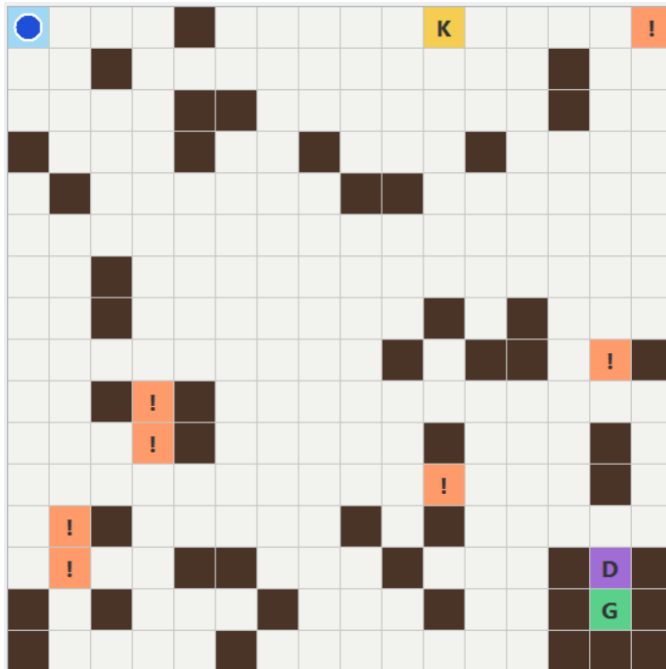
بنابراین، محیط اصلی یک شبکه 16×16 شامل ۲۵۶ خانه است. نقشه نهایی دارای ۴۶ خانه دیوار است که ۱۷,۹۷ درصد کل شبکه را تشکیل می‌دهد و از حداقل ۱۵ درصد تعیین شده در صورت پروژه بیشتر است. همچنین، هفت خانه جریمه در محیط قرار گرفته‌اند. با حذف دیوارها، ۲۱۰ خانه قابل عبور در نقشه وجود دارد.

عناصر محیط شامل خانه عادی، دیوار، خانه جریمه، نقطه شروع، کلید، در قفل‌شده و هدف هستند. موقعیت عناصر اصلی در نقشه مبدأ، با استفاده از اندیس‌گذاری صفرمبنا، به صورت زیر تعیین شده است:

$$\text{Start} = (0,0), \quad \text{Key} = (0,10), \quad \text{Door} = (13,14), \quad \text{Goal} = (14,14)$$

هدف در یک محفظه کوچک نزدیک گوشه پایین سمت راست نقشه قرار گرفته است. تمام مسیرهای ورود به این محفظه، به جز خانه در، با دیوار مسدود شده‌اند. بنابراین، عامل نمی‌تواند از مسیر دیگری به هدف دسترسی پیدا کند و سازوکار کلید و در بر روند تصمیم‌گیری اثر واقعی دارد.

با توجه به اینکه نقشه مبدأ در تمام آزمایش‌های اصلی ثابت نگه داشته می‌شود، واژه «پویا» در این پروژه به تغییر چیدمان نقشه در طول یک اپیزود اشاره ندارد. پویایی مسئله از تغییر وضعیت داخلی عامل، شامل دریافت کلید و کاهش انرژی، و همچنین تصادفی بودن نتیجه اعمال ناشی می‌شود. نقشه‌های متفاوت تنها در بخش انتقال یادگیری و به عنوان محیط‌های مقصد استفاده می‌شوند.



شکل ۲-۲: نمای محیط هزارتوی مبدأ در ابتدای اپیزود، شامل نقطه شروع، کلید، در قفل‌شده، هدف، دیوارها، خانه‌های جریمه و نوار انرژی عامل

۲-۲. مأموریت عامل

عامل در آغاز هر اپیزود در خانه $(0,0)$ قرار می‌گیرد، کلید را در اختیار ندارد و انرژی آن برابر ۵۰ واحد است. مأموریت عامل از دو زیرهدف متوالی تشکیل شده است:

۱. حرکت از نقطه شروع به سمت خانه کلید؛
۲. حرکت از خانه کلید، عبور از در و رسیدن به هدف.

پس از ورود عامل به خانه کلید، وضعیت کلید از صفر به یک تغییر می‌کند و این تغییر تا پایان اپیزود باقی می‌ماند. دریافت کلید به‌خودی‌خود موجب پایان اپیزود نمی‌شود، بلکه شرایط عبور از در را فراهم می‌کند. در صورتی که عامل پیش از دریافت کلید برای ورود به خانه در تلاش کند، حرکت انجام نمی‌شود و عامل در موقعیت قبلی باقی می‌ماند.

کوتاه‌ترین مسیر قطعی از شروع تا کلید، بدون عبور از در، ۱۲ گام است. کوتاه‌ترین مسیر از کلید تا هدف، در حالتی که در قابل عبور باشد، ۱۸ گام طول دارد. در نتیجه، کوتاه‌ترین مسیر ترکیبی در مدل قطعی ۳۰ گام است. باین‌حال، به دلیل تصادفی‌بودن حرکت، عامل ممکن است برای تکمیل مأموریت به بیش از ۳۰ گام نیاز داشته

باشد. همچنین، کوتاه‌ترین مسیر هندسی لزوماً بیشترین بازده مورد انتظار را ندارد؛ زیرا نزدیکی مسیر به دیوارها و خانه‌های جریمه می‌تواند احتمال رخدادهای نامطلوب را افزایش دهد.

۲-۳. قوانین محیط

۲-۳-۱. انتخاب و اجرای عمل

در هر گام زمانی، عامل یکی از چهار عمل بالا، پایین، چپ یا راست را انتخاب می‌کند. عمل انتخاب‌شده همواره به همان شکل اجرا نمی‌شود. جهت مؤثر حرکت براساس توزیع احتمال زیر تعیین می‌شود:

- با احتمال 0.8، عمل انتخاب‌شده اجرا می‌شود؛
- با احتمال 0.1، عامل در یکی از دو جهت عمود منحرف می‌شود؛
- با احتمال 0.1، عامل در جهت عمود دیگر منحرف می‌شود.

برای نمونه، اگر عامل عمل «بالا» را انتخاب کند، با احتمال 0.8 به سمت بالا، با احتمال 0.1 به سمت چپ و با احتمال 0.1 به سمت راست حرکت می‌کند. برای اعمال افقی نیز دو جهت عمود، بالا و پایین هستند.

این تصادفی بودن در هر سه الگوریتم لحاظ شده است. در Value Iteration، سه نتیجه ممکن مستقیماً در مدل انتقال وارد می‌شوند. Q-Learning و SARSA(λ) به مدل احتمالات دسترسی ندارند و نتیجه واقعی هر عمل را از طریق تعامل با محیط مشاهده می‌کنند.

۲-۳-۲. برخورد با دیوار و مرز محیط

اگر جهت مؤثر حرکت عامل به خارج از محدوده نقشه یا به داخل یک خانه دیوار منتهی شود، موقعیت عامل تغییر نمی‌کند. این رخداد به عنوان برخورد با دیوار ثبت می‌شود. با وجود ثابت ماندن موقعیت، گام انجام‌شده محسوب می‌شود و یک واحد انرژی مصرف می‌گردد. عامل همچنین جریمه متناظر با برخورد را دریافت می‌کند.

این قانون سبب می‌شود انتخاب مکرر اعمال نامعتبر فقط به ثابت ماندن عامل محدود نشود، بلکه از نظر پاداش و انرژی نیز هزینه داشته باشد. در نتیجه، عامل باید ساختار محلی نقشه را بیاموزد و احتمال برخورد با موانع را کاهش دهد.

۲-۳-۳. خانه‌های جریمه

خانه‌های جریمه قابل عبور هستند و برخلاف دیوار، مانع حرکت عامل نمی‌شوند. در صورت ورود عامل به یکی از این خانه‌ها، موقعیت آن به خانه جدید تغییر می‌کند، اما یک جریمه اضافی دریافت می‌شود. خانه جریمه حالت پایانی نیست و عامل می‌تواند در گام بعدی از آن خارج شود.

وجود این خانه‌ها مسئله را از یک مسیریابی ساده متمایز می‌کند. ممکن است مسیری کوتاه‌تر از چند خانه جریمه عبور کند، درحالی‌که مسیر بلندتر بازده مورد انتظار بیشتری داشته باشد. اثر این تفاوت بر رفتار الگوریتم‌های on-policy و off-policy در فصل‌های مربوط به نتایج بررسی می‌شود.

۲-۳-۴. دریافت کلید

اگر عامل برای نخستین بار وارد خانه کلید شود، متغیر وضعیت کلید از 0 به 1 تغییر می‌کند. این تغییر برگشت‌پذیر نیست و عامل تا پایان اپیزود کلید را در اختیار خواهد داشت. ورود مجدد به خانه کلید، پاداش دریافت کلید را دوباره ایجاد نمی‌کند؛ زیرا شرط دریافت کلید تنها زمانی برقرار است که مقدار قبلی متغیر کلید صفر باشد.

کلید یک زیرهدف میانی است. دریافت آن برای تکمیل مأموریت ضروری است، اما اپیزود پس از دریافت کلید ادامه پیدا می‌کند.

۲-۳-۵. در قفل‌شده

رفتار خانه در به وضعیت کلید وابسته است. اگر عامل بدون کلید تلاش کند وارد خانه در شود، حرکت مسدود می‌شود و موقعیت عامل تغییر نمی‌کند. این رخداد به‌عنوان تلاش برای عبور از در قفل‌شده ثبت می‌شود، یک واحد انرژی مصرف می‌کند و جریمه متناظر را به همراه دارد.

پس از دریافت کلید، خانه در مانند یک خانه قابل عبور عمل می‌کند. ورود موفق عامل به در به‌عنوان رخداد مستقل ثبت می‌شود، اما پایان اپیزود محسوب نمی‌شود. عامل پس از عبور از در باید وارد خانه هدف شود.

۲-۳-۶. رسیدن به هدف

خانه هدف مقصد نهایی مأموریت است. به دلیل ساختار محفظه هدف، ورود به این خانه تنها از مسیر در امکان‌پذیر است. هنگامی که عامل به هدف می‌رسد، پاداش نهایی را دریافت می‌کند و اپیزود با موفقیت پایان می‌یابد.

۲-۳-۷. ثبت رخدادهای محیط

محیط در هر گام نوع رخداد حاصل از انتقال را ثبت می‌کند. رخدادهای تعریف‌شده شامل حرکت عادی، برخورد با دیوار، ورود به خانه جریمه، دریافت کلید، تلاش برای عبور از در قفل‌شده، عبور موفق از در، رسیدن به هدف، پایان انرژی و رسیدن به سقف تعداد گام‌ها هستند.

علاوه بر نوع رخداد، عمل انتخاب‌شده و عمل مؤثر نیز به‌صورت جداگانه ذخیره می‌شوند. این تمایز برای تحلیل اثر لغزش تصادفی لازم است؛ زیرا ممکن است عامل یک جهت را انتخاب کند، اما محیط جهت دیگری را اجرا کند. موقعیت، وضعیت کلید، انرژی، پاداش و حالت بعدی نیز در فایل‌های ثبت رویداد نگهداری می‌شوند.

۴-۲. قابلیت تکمیلی: انرژی محدود

از میان قابلیت‌های تکمیلی پیشنهادی در صورت پروژه، ویژگی انرژی محدود انتخاب شده است. عامل در ابتدای هر اپیزود با بودجه انرژی

$$E_0 = 50$$

شروع می‌کند. هر عمل، مستقل از نتیجه آن، یک واحد انرژی مصرف می‌کند. بنابراین، تحول انرژی به‌صورت زیر است:

$$e_{t+1} = e_t - 1$$

مصرف انرژی در تمام حالت‌های زیر رخ می‌دهد:

- حرکت عادی به یک خانه قابل عبور؛
- ورود به خانه جریمه؛
- دریافت کلید؛
- عبور از در؛
- برخورد با دیوار یا مرز نقشه؛
- تلاش ناموفق برای عبور از در قفل‌شده؛
- ورود به خانه هدف.

در نتیجه، انرژی تنها به مسافت جابه‌جایی وابسته نیست، بلکه تعداد تصمیم‌های عامل را محدود می‌کند. حتی عملی که موجب تغییر موقعیت نمی‌شود، بخشی از بودجه انرژی را مصرف می‌کند. این طراحی مانع از آن می‌شود که عامل بدون هزینه در مقابل یک دیوار یا در قفل‌شده باقی بماند.

کوتاه‌ترین مسیر قطعی کامل ۳۰ گام است و ۶۰ درصد انرژی اولیه را مصرف می‌کند. بنابراین، در حالت بدون لغزش، ۲۰ واحد انرژی برای خطاها یا انحراف‌های احتمالی باقی می‌ماند. با این حال، سیاست‌های ناکارآمد، حرکات‌های حلقه‌ای یا برخوردهای متعدد می‌توانند پیش از رسیدن به هدف انرژی عامل را به پایان برسانند.

ویژگی انرژی محدود بخشی از منطق تصمیم‌گیری است و صرفاً برای نمایش در رابط گرافیکی استفاده نمی‌شود. دو عامل با موقعیت و وضعیت کلید یکسان، اما انرژی متفاوت، لزوماً شرایط آینده یکسانی ندارند. عاملی که انرژی کمی دارد ممکن است دیگر قادر به تکمیل مسیر باقی‌مانده نباشد، در حالی که همان مسیر برای عاملی با انرژی بیشتر قابل اجرا است. به همین دلیل، انرژی در تعریف حالت وارد شده است.

۵-۲. تعریف فضای حالت

حالت عامل در زمان t به صورت یک چهارتایی تعریف می‌شود:

$$s_t = (r_t, c_t, k_t, e_t)$$

که در آن:

- r_t شماره سطر موقعیت عامل است؛
- c_t شماره ستون موقعیت عامل است؛
- $k_t \in \{0, 1\}$ وضعیت دریافت کلید را مشخص می‌کند؛
- e_t انرژی باقی‌مانده عامل است.

مختصات r_t و c_t در بازه صفر تا ۱۵ قرار دارند، اما حالت‌هایی که مختصات آن‌ها متناظر با دیوار باشد، در فضای حالت معتبر قرار نمی‌گیرند. همچنین، قرار گرفتن عامل روی خانه در با $k_t = 0$ یک حالت دسترس‌ناپذیر است؛ زیرا عامل بدون کلید اجازه ورود به در را ندارد.

در ابتدای هر اپیزود، حالت عامل برابر است با

$$s_0 = (0, 0, 0, 50)$$

پس از دریافت کلید، مؤلفه k_t برابر یک باقی می‌ماند. مقدار انرژی در هر گام یک واحد کاهش پیدا می‌کند و رسیدن آن به صفر موجب پایان اپیزود می‌شود.

وضعیت باز یا بسته بودن در به عنوان یک متغیر مستقل ذخیره نشده است؛ زیرا این وضعیت مستقیماً از k_t تعیین می شود. وقتی $k_t = 0$ باشد، در بسته است و وقتی $k_t = 1$ باشد، عامل می تواند از آن عبور کند. موقعیت کلید، در، هدف، دیوارها و خانه های جریمه نیز بخشی از حالت متغیر عامل نیستند، زیرا برای یک محیط مشخص در فایل نقشه ذخیره شده اند و در طول اپیزود تغییر نمی کنند.

۲-۶. فضای عمل

فضای عمل عامل گسسته و شامل چهار عضو است:

$$A = \{UP, DOWN, LEFT, RIGHT\}$$

در پیاده سازی، این اعمال به ترتیب با مقادیر عددی صفر تا سه نمایش داده شده اند:

$$0 = UP, \quad 1 = DOWN, \quad 2 = LEFT, \quad 3 = RIGHT$$

تغییر مختصات متناظر با هر عمل به صورت زیر است:

$$\Delta(UP) = (-1, 0)$$

$$\Delta(DOWN) = (1, 0)$$

$$\Delta(LEFT) = (0, -1)$$

$$\Delta(RIGHT) = (0, 1)$$

این مقادیر عمل انتخابی عامل را مشخص می کنند، نه لزوماً جهت نهایی حرکت را. جهت مؤثر پس از اعمال احتمال های 0.8، 0.1 و 0.1 تعیین می شود.

فضای عمل در تمام حالت های غیر پایانی یکسان است. محیط اعمالی را که به دیوار، مرز یا در قفل شده منتهی می شوند از فضای عمل حذف نمی کند؛ بلکه پیامد آن ها را به صورت ثابت ماندن عامل و دریافت جریمه مدل می کند. این روش به عامل اجازه می دهد نامناسب بودن چنین اعمالی را از تجربه یا مدل انتقال بیاموزد.

۲-۷. حفظ خاصیت مارکوف

یک نمایش حالت زمانی خاصیت مارکوف را حفظ می‌کند که توزیع حالت و پاداش بعدی، با داشتن حالت فعلی و عمل انتخاب‌شده، به تاریخچه کامل اپیزود وابسته نباشد. در محیط این پروژه، استفاده از مختصات مکانی به‌تنهایی برای تحقق این شرط کافی نیست.

برای نمونه، دو عامل ممکن است در مختصات یکسانی قرار داشته باشند، اما یکی کلید را دریافت کرده باشد و دیگری هنوز کلید را در اختیار نداشته باشد. پیامد تلاش آن‌ها برای ورود به در یکسان نیست. عامل اول می‌تواند عبور کند، درحالی‌که عامل دوم در جای خود باقی می‌ماند. بنابراین، وضعیت کلید باید در حالت گنجانده شود.

همین مسئله درباره انرژی نیز برقرار است. اگر موقعیت و وضعیت کلید دو عامل برابر باشد، اما انرژی باقی‌مانده آن‌ها متفاوت باشد، احتمال تکمیل مأموریت و زمان پایان اپیزود برای آن‌ها یکسان نیست. عامل دارای یک واحد انرژی، پس از اجرای عمل بعدی وارد حالت پایانی می‌شود، درحالی‌که عامل دارای انرژی بیشتر می‌تواند اپیزود را ادامه دهد. حذف انرژی از حالت سبب می‌شود رفتار آینده به تعداد گام‌های قبلی و در نتیجه به تاریخچه وابسته شود.

با تعریف $s_t = (r_t, c_t, k_t, e_t)$ تمام متغیرهای متغیری که بر انتقال بعدی اثر دارند در حالت قرار می‌گیرند. موقعیت بعدی از موقعیت فعلی، عمل مؤثر و ساختار ثابت نقشه تعیین می‌شود. امکان عبور از در به kt وابسته است و انرژی بعدی نیز مستقیماً از e_t محاسبه می‌شود. در نتیجه، برای محیط مبدأ و تنظیمات استفاده‌شده در آزمایش‌ها، اطلاعات مربوط به مراحل پیشین برای تعیین رفتار گام بعد لازم نیست.

شمارنده گام در حالت وارد نشده است. سقف پیش‌فرض تعداد گام‌ها برابر سه برابر تعداد خانه‌های قابل عبور تعیین شده است:

$$\text{max_steps} = 3 \times 210 = 630$$

از آنجا که انرژی اولیه فقط ۵۰ واحد است و هر گام یک واحد انرژی مصرف می‌کند، اپیزود در تنظیمات فعلی حداکثر پس از ۵۰ عمل به دلیل پایان انرژی خاتمه می‌یابد. بنابراین، سقف ۶۳۰ گام در اجرای فعلی فعال نمی‌شود و تنها یک سازوکار ایمنی برای تنظیمات احتمالی دیگر است. به همین دلیل، حذف شمارنده گام از حالت، خاصیت مارکوف مسئله مورد ارزیابی را نقض نمی‌کند.

تصادفی بودن لغزش نیز با خاصیت مارکوف ناسازگار نیست. نتیجه عمل از یک توزیع ثابت وابسته به عمل فعلی نمونه‌برداری می‌شود و احتمال آن به اعمال یا حالت‌های قبلی وابسته نیست.

۸-۲. شرایط پایان اپیزود

هر اپیزود در یکی از سه وضعیت زیر پایان می‌یابد:

۲-۸-۱. رسیدن موفق به هدف

اگر موقعیت بعدی عامل برابر خانه هدف باشد، اپیزود با موفقیت پایان می‌یابد. این وضعیت تنها حالت موفقیت در محیط است. دریافت کلید و عبور از در، زیرهدف‌های میانی هستند و به‌تنهایی موجب پایان اپیزود نمی‌شوند.

۲-۸-۲. پایان انرژی

اگر پس از اجرای یک عمل مقدار انرژی به صفر یا کمتر برسد، اپیزود با شکست پایان می‌یابد. این شرط حتی زمانی اعمال می‌شود که عامل در همان گام وارد خانه عادی، به دیوار برخورد یا برای عبور از در قفل‌شده تلاش کرده باشد. اگر عامل در آخرین واحد انرژی به هدف برسد، رسیدن به هدف پیش از بررسی پایان انرژی ارزیابی می‌شود و اپیزود به‌عنوان موفق ثبت خواهد شد.

۲-۸-۳. رسیدن به سقف تعداد گام‌ها

اگر عامل پیش از رسیدن به هدف یا پایان انرژی به مقدار `max_steps` برسد، اپیزود به‌صورت `truncation` متوقف می‌شود. مقدار پیش‌فرض این سقف در نقشه مبدأ ۶۳۰ گام است. همان‌طور که بیان شد، در تنظیمات فعلی انرژی پس از ۵۰ گام پایان می‌یابد؛ بنابراین، این شرط در آزمایش‌های اصلی نقش محدودکننده ندارد و برای جلوگیری از اپیزودهای نامتناهی در صورت تغییر تنظیمات محیط نگه داشته شده است.

برخورد با دیوار، ورود به خانه جریمه، دریافت کلید، تلاش برای عبور از در بسته و عبور موفق از در هیچ‌کدام حالت پایانی نیستند. پس از وقوع این رخدادها، عامل در صورت داشتن انرژی کافی می‌تواند تصمیم‌گیری را ادامه دهد.

فصل سوم: مدل سازی فرایند تصمیم گیری مارکوف

برای حل مسئله هزارتو با استفاده از الگوریتم های یادگیری تقویتی، لازم است تعامل میان عامل و محیط به صورت یک مدل ریاضی دقیق تعریف شود. چارچوب استاندارد برای مدل سازی مسائل تصمیم گیری متوالی، فرایند تصمیم گیری مارکوف یا Markov Decision Process است.

در این چارچوب، عامل در هر گام زمانی وضعیت فعلی محیط را مشاهده می کند، عملی را انتخاب می کند و سپس محیط بر اساس تابع انتقال احتمالاتی به حالت بعدی منتقل می شود. پس از این انتقال، یک پاداش عددی نیز به عامل اختصاص می یابد. عامل با تکرار این فرایند تلاش می کند سیاستی به دست آورد که مجموع بلندمدت پاداش ها را بیشینه کند.

مدل محیط این پروژه به صورت یک MDP متناهی تعریف می شود:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma, \mathcal{S}_{\mathcal{T}})$$

۱-۳. فضای حالت

حالت محیط باید تمام اطلاعاتی را در بر گیرد که برای پیش بینی توزیع حالت بعدی و پاداش حاصل از یک عمل ضروری است. در محیط این پروژه، حالت عامل شامل موقعیت مکانی، وضعیت دریافت کلید و انرژی باقی مانده است:

$$s_t = (r_t, c_t, k_t, e_t)$$

در این رابطه، r_t و c_t به ترتیب شماره سطر و ستون عامل در زمان t هستند. متغیر k_t نشان می دهد که آیا عامل کلید را دریافت کرده است یا خیر و e_t مقدار انرژی باقی مانده را مشخص می کند.

محدوده مختصات در نقشه 16×16 به صورت زیر است:

$$r_t, c_t \in \{0, 1, \dots, 15\}$$

و متغیر وضعیت کلید دو مقدار ممکن دارد:

$$k_t \in \{0, 1\}$$

انرژی اولیه عامل برابر ۵۰ است و در هر گام یک واحد کاهش پیدا می‌کند. بنابراین، انرژی در حالت‌های غیرپایانی در مجموعه زیر قرار می‌گیرد:

$$e_t \in \{1, 2, \dots, 50\}$$

فضای حالت را می‌توان به صورت رسمی چنین نوشت:

$$\mathcal{S} = \{(r, c, k, e) \mid (r, c) \in \mathcal{W}, k \in \{0, 1\}, e \in \{1, \dots, 50\}\}$$

در این رابطه، $\mathcal{W}$ مجموعه خانه‌های قابل عبور نقشه است. خانه‌های دیوار در فضای حالت قرار نمی‌گیرند، زیرا عامل نمی‌تواند داخل آن‌ها قرار گیرد.

حالت اولیه تمام اپیزودها برابر است با:

$$s_0 = (0, 0, 0, 50)$$

یعنی عامل در مختصات شروع (0,0)، بدون کلید و با ۵۰ واحد انرژی وارد محیط می‌شود.

۳-۱-۱. محدودیت‌های فضای حالت

همه ترکیب‌های ممکن از مقادیر r, c, k و e الزاماً حالت معتبر یا قابل دسترس نیستند. نخست، خانه‌های دیوار از فضای حالت حذف می‌شوند. دوم، قرارگرفتن عامل روی خانه در در حالی که کلید ندارد، نامعتبر است؛ زیرا محیط در چنین حالتی ورود عامل را مسدود می‌کند.

شرط حذف این حالت‌ها را می‌توان به صورت زیر بیان کرد:

$$(r, c) = \text{Door} \Rightarrow k = 1$$

همچنین، خانه هدف به عنوان حالت غیرپایانی در فهرست حالت‌های مورد پیمایش Value Iteration قرار نمی‌گیرد. پس از ورود عامل به هدف، اپیزود بلافاصله پایان می‌یابد و ارزش ادامه مسیر برابر صفر در نظر گرفته می‌شود.

نقشه دارای ۲۱۰ خانه قابل عبور است. با حذف خانه هدف، در نظر گرفتن دو وضعیت کلید و ۵۰ سطح انرژی و سپس حذف حالت‌های قرارگرفتن روی در بدون کلید، تعداد حالت‌های غیرپایانی فهرست شده در Value Iteration برابر است با:

$$N_{\text{states}} = (210 - 1) \times 2 \times 50 - 50 = 20850$$

این مقدار نشان دهنده تعداد حالت‌هایی است که در جدول کامل Value Iteration نگهداری می‌شوند. با این حال، برخی ترکیب‌ها ممکن است در عمل از حالت اولیه قابل دسترسی نباشند. برای نمونه، ورود عامل به خانه کلید بلافاصله مقدار k را از صفر به یک تغییر می‌دهد؛ بنابراین، باقی‌ماندن روی خانه کلید با $k=0$ یک حالت عملی پایدار نیست.

۳-۲. فضای عمل

عامل در هر حالت غیرپایانی می‌تواند یکی از چهار جهت اصلی را انتخاب کند. فضای عمل به صورت زیر تعریف می‌شود:

$$A = \{\text{UP}, \text{DOWN}, \text{LEFT}, \text{RIGHT}\}$$

در پیاده‌سازی، اعمال با مقادیر صحیح صفر تا سه نمایش داده شده‌اند:

$$0 = \text{UP}, \quad 1 = \text{DOWN}, \quad 2 = \text{LEFT}, \quad 3 = \text{RIGHT}$$

فضای اعمال در تمام حالت‌های غیرپایانی یکسان است. حتی اگر یک عمل به دیوار، مرز نقشه یا در قفل شده منتهی شود، آن عمل از فضای اعمال حذف نمی‌شود. در عوض، محیط پیامد ناموفق آن را به صورت ثابت‌ماندن عامل، کاهش انرژی و دریافت جریمه مدل می‌کند.

این طراحی به عامل اجازه می‌دهد نامناسب بودن اعمال را از طریق مقدار پاداش یا تابع ارزش بیاموزد، نه اینکه اعمال نامعتبر از ابتدا توسط محیط از دسترس خارج شوند.

۳-۳. تابع انتقال

تابع انتقال، احتمال رسیدن به حالت s' را پس از اجرای عمل a در حالت s مشخص می‌کند:

$$P(s'|s, a) = \Pr(S_{t+1} = s' | S_t = s, A_t = a)$$

محیط این پروژه تصادفی است و عمل انتخاب‌شده عامل همیشه به همان جهت اجرا نمی‌شود. اگر عمل انتخابی عامل را با a و عمل مؤثر را با $\tilde{a}$ نمایش دهیم، احتمال اجرای جهت موردنظر برابر است با:

$$P(\tilde{a} = a|a) = 0.8$$

دو جهت عمود بر عمل انتخاب شده نیز هر کدام با احتمال 0.1 اجرا می شوند:

$$P(\tilde{a} = a_{\perp,1}|a) = 0.1$$

$$P(\tilde{a} = a_{\perp,2}|a) = 0.1$$

بنابراین، مجموع احتمال شاخه های انتقال برابر یک است:

$$0.8 + 0.1 + 0.1 = 1$$

۳-۳-۱. انتقال مکانی

فرض شود عامل در موقعیت (r, c) قرار دارد و عمل مؤثر $\tilde{a}$ دارای تغییر مختصات $(\Delta r, \Delta c)$ است. موقعیت پیشنهادی بعدی برابر است با:

$$(r_{\text{cand}}, c_{\text{cand}}) = (r + \Delta r, c + \Delta c)$$

اگر خانه پیشنهادی داخل محدوده نقشه و قابل عبور باشد، موقعیت جدید عامل برابر همان خانه خواهد بود:

$$(r', c') = (r_{\text{cand}}, c_{\text{cand}})$$

اما اگر حرکت به دیوار یا خارج از مرز نقشه منتهی شود، موقعیت عامل ثابت باقی می ماند:

$$(r', c') = (r, c)$$

تلاش برای ورود به در بدون کلید نیز نتیجه مشابهی دارد:

$$(r_{\text{cand}}, c_{\text{cand}}) = \text{Door}, \quad k = 0$$

در این وضعیت:

$$(r', c') = (r, c)$$

بنابراین، ثابت ماندن موقعیت لزوماً به معنای اجرا نشدن گام نیست. انرژی کاهش می یابد و پاداش متناظر با برخورد یا در قفل شده اعمال می شود.

۳-۳-۲. انتقال وضعیت کلید

مقدار بعدی متغیر کلید به موقعیت جدید عامل وابسته است. اگر عامل بدون داشتن کلید وارد خانه کلید شود، وضعیت کلید به یک تغییر می‌کند:

$$(r', c') = \text{Key}, \quad k = 0 \Rightarrow k' = 1$$

در سایر حالت‌ها، وضعیت کلید بدون تغییر باقی می‌ماند:

$$k' = k$$

پس از دریافت کلید، این متغیر تا پایان اپیزود برابر یک باقی می‌ماند و سازوکاری برای رهاکردن یا ازدست‌دادن کلید وجود ندارد.

۳-۳-۳. انتقال انرژی

در تمام انتقال‌ها، بدون توجه به موفق یا ناموفق بودن حرکت، یک واحد انرژی مصرف می‌شود:

$$e' = e - 1$$

بنابراین، حالت بعدی به صورت کلی چنین نوشته می‌شود:

$$s' = (r', c', k', e - 1)$$

این رابطه برای برخورد با دیوار، ورود به خانه جریمه، دریافت کلید، عبور از در و رسیدن به هدف نیز برقرار است.

۳-۳-۴. تجمیع انتقال‌های یکسان

در برخی وضعیت‌ها ممکن است بیش از یک شاخه احتمالاتی به یک حالت بعدی یکسان منتهی شود. برای مثال، اگر عامل در کنار دو دیوار قرار داشته باشد، هم حرکت اصلی و هم یکی از لغزش‌ها ممکن است باعث ثابت ماندن موقعیت شوند.

در چنین شرایطی، پیاده‌سازی احتمالات شاخه‌های یکسان را با یکدیگر جمع می‌کند. تابع انتقال کلی را می‌توان به صورت زیر نوشت:

$$P(s'|s, a) = \sum_{\tilde{a} \in \mathcal{B}(a)} P(\tilde{a}|a) \mathbf{1}\{T(s, \tilde{a}) = s'\}$$

در این رابطه:

- $B(a)$ مجموعه سه جهت ممکن برای عمل a است؛
- $T(s, a \sim)$ انتقال قطعی حاصل از اجرای جهت مؤثر است؛
- تابع شاخص 1 در صورت برابری انتقال با s' مقدار یک و در غیر این صورت مقدار صفر می‌گیرد.

در Value Iteration، تابع transitions تمام خروجی‌های ممکن را همراه با احتمال، حالت بعدی، پاداش و وضعیت پایان در اختیار الگوریتم قرار می‌دهد. در مقابل، Q-Learning و SARSA(λ) به این مدل دسترسی ندارند و در هر گام فقط یک انتقال تصادفی را از طریق تابع step مشاهده می‌کنند.

۳-۴. تابع پاداش

تابع پاداش به صورت کلی به حالت فعلی، عمل و حالت بعدی وابسته است:

$$R(s, a, s') = E[R_{t+1} | S_t = s, A_t = a, S_{t+1} = s']$$

در محیط پروژه دو نوع پاداش پیاده‌سازی شده است:

۱. پاداش تنک یا Sparse Reward؛

۲. پاداش شکل‌دهی شده یا Shaped Reward.

جزئیات طراحی و مقایسه این دو نوع پاداش در فصل پنجم بررسی خواهد شد. در این فصل، تعریف رسمی آن‌ها ارائه می‌شود.

۳-۴-۱. پاداش تنک

در حالت تنک، هر گام دارای هزینه پایه منفی است:

$$r_{\text{step}} = -1$$

جریمه یا پاداش رخدادهای به این مقدار پایه افزوده می‌شوند. تابع پاداش تنک را می‌توان به شکل زیر نوشت:

$$r_{\text{sparse}} = -1 - 4I_{\text{wall}} - 4I_{\text{locked}} - 9I_{\text{penalty}} + 50I_{\text{key}} + 100I_{\text{goal}} - 20I_{\text{energy}}$$

در این رابطه، هر I یک تابع شاخص است که در صورت وقوع رخداد متناظر مقدار یک و در غیر این صورت مقدار صفر دارد.

بنابراین، پاداش رایج هر رخداد به صورت زیر خواهد بود:

- حرکت عادی: -1 ؛
- برخورد با دیوار یا مرز: -5 ؛
- تلاش برای عبور از در قفل شده: -5 ؛
- ورود به خانه جریمه: -10 ؛
- دریافت کلید: 49 ؛
- عبور عادی از در پس از دریافت کلید: -1 ؛
- رسیدن به هدف: 99 ؛
- پایان انرژی در یک حرکت عادی: -21 .

علت آنکه پاداش دریافت کلید برابر 49 و پاداش هدف برابر 99 می شود این است که هزینه پایه یک گام، یعنی -1 ، در همان انتقال نیز اعمال می شود.

اگر چند رخداد قابل جمع در یک گام اتفاق بیفتند، مؤلفه های متناظر با یکدیگر جمع می شوند. برای مثال، اگر عامل با آخرین واحد انرژی وارد خانه جریمه شود، هزینه گام، جریمه خانه و جریمه پایان انرژی همگی در پاداش نهایی اثر خواهند داشت.

۲-۴-۳. پاداش شکل دهی شده

در نسخه شکل دهی شده، پاداش تنک حفظ می شود و یک جمله اضافی مبتنی بر پتانسیل به آن افزوده می شود:

$$r_{\text{shaped}} = r_{\text{sparse}} + \eta F(s, s')$$

ضریب شکل دهی در پیاده سازی برابر یک است:

$$\eta = 1$$

جمله شکل دهی به صورت زیر تعریف می شود:

$$F(s, s') = \gamma_{\text{sh}} \Phi(s') - \Phi(s)$$

مقدار ضریب تنزیل مورد استفاده در این بخش برابر است با:

$$\gamma_{sh} = 0.99$$

تابع پتانسیل براساس منفی فاصله BFS عامل تا زیرهدف جاری تعریف شده است:

$$\Phi(s) = -d(s, g_{sub})$$

زیرهدف جاری به وضعیت کلید وابسته است:

$$g_{sub} = \text{Key} \quad \text{if} \quad k = 0$$

و پس از دریافت کلید:

$$g_{sub} = \text{Goal} \quad \text{if} \quad k = 1$$

هنگامی که عامل هنوز کلید را دریافت نکرده است، فاصله‌ها با فرض بسته‌بودن در محاسبه می‌شوند. پس از دریافت کلید، فاصله تا هدف با فرض بازبودن در نظر گرفته می‌شود.

برای حالت‌های پایانی، پتانسیل حالت بعدی برابر صفر قرار داده شده است:

$$\Phi(s') = 0 \quad \text{if} \quad s' \in \mathcal{S}_T$$

این نوع شکل‌دهی به عامل بازخورد بیشتری درباره نزدیک‌شدن یا دورشدن از زیرهدف می‌دهد، درحالی‌که ساختار اصلی پاداش دریافت کلید و هدف را حفظ می‌کند.

۵-۳. ضریب تنزیل

عامل باید نه تنها پاداش لحظه‌ای، بلکه پیامدهای آینده تصمیم‌های خود را نیز در نظر بگیرد. بازده تنزیل‌شده از زمان t به صورت زیر تعریف می‌شود:

$$G_t = \sum_{j=0}^{T-t-1} \gamma^j R_{t+j+1}$$

در این رابطه، γ ضریب تنزیل است:

$$0 \leq \gamma \leq 1$$

مقدار کوچک γ باعث می‌شود عامل اهمیت بیشتری به پاداش‌های نزدیک بدهد. در مقابل، مقادیر نزدیک به یک سبب می‌شوند پاداش‌های آینده، مانند دریافت کلید و رسیدن به هدف، تأثیر بیشتری بر ارزش حالت فعلی داشته باشند.

در آزمایش Value Iteration سه مقدار زیر بررسی شده‌اند:

$$\gamma \in \{0.90, 0.95, 0.99\}$$

هدف از این آزمایش، بررسی اثر ضریب تنزیل بر سرعت همگرایی، تابع ارزش و سیاست نهایی است.

در آزمایش‌های اصلی Q-Learning، SARSA(λ)، پایداری و انتقال یادگیری از مقدار زیر استفاده شده است:

$$\gamma = 0.99$$

انتخاب مقدار ۰٫۹۹ با ماهیت چندمرحله‌ای مأموریت سازگار است. پاداش اصلی هدف پس از طی تعداد قابل توجهی گام دریافت می‌شود و استفاده از ضریب تنزیل بالا کمک می‌کند اثر این پاداش در حالت‌های دورتر نیز باقی بماند.

برای مثال، اثر یک پاداش که ۳۰ گام بعد دریافت می‌شود، با ضریب تنزیل ۰٫۹۹ به صورت زیر وزن‌دهی می‌شود:

$$0.99^{30} \approx 0.740$$

در حالی که با ضریب ۰٫۹۰ وزن همان پاداش برابر است با:

$$0.90^{30} \approx 0.042$$

بنابراین، در مقدار ۰٫۹۰ پاداش دوردست هدف با شدت بسیار بیشتری تنزیل می‌شود. اثر واقعی این تفاوت بر همگرایی و سیاست Value Iteration در فصل ششم تحلیل خواهد شد.

۳-۶. حالت‌های پایانی

مجموعه حالت‌های پایانی را می‌توان به صورت زیر تعریف کرد:

$$\mathcal{S}_{\mathcal{T}} = \mathcal{S}_{ga} \cup \mathcal{S}_{eeq}$$

۳-۶-۱. رسیدن به هدف

اگر موقعیت عامل برابر مختصات هدف شود، اپیزود با موفقیت پایان می‌یابد:

$$(r', c') = \text{Goal} \Rightarrow \text{done} = \text{True}$$

رسیدن به هدف تنها معیار موفقیت کامل در محیط است. دریافت کلید و عبور از در، حالت پایانی نیستند.

۳-۶-۲. پایان انرژی

اگر انرژی بعد از اجرای عمل به صفر یا کمتر برسد، اپیزود با شکست پایان می‌یابد:

$$e' \leq 0 \Rightarrow \text{done} = \text{True}$$

اگر عامل در همان گامی که انرژی آن به صفر می‌رسد وارد هدف شود، موفقیت هدف در اولویت قرار دارد. در پیاده‌سازی، شرط رسیدن به هدف پیش از شرط پایان انرژی بررسی می‌شود.

۳-۶-۳. محدودیت تعداد گام‌ها

حداکثر تعداد گام‌های یک اپیزود برابر سه برابر تعداد خانه‌های قابل عبور تعیین شده است:

$$\text{max_steps} = 3 \times 210 = 630$$

رسیدن به این حد یک truncation است، نه یک حالت پایانی طبیعی MDP. در این حالت، اپیزود برای جلوگیری از اجرای نامحدود متوقف می‌شود:

$$t \geq 630 \Rightarrow \text{truncated} = \text{True}$$

در تنظیمات اصلی پروژه، عامل فقط ۵۰ واحد انرژی دارد و در هر گام یک واحد مصرف می‌کند. بنابراین، اپیزود پیش از رسیدن به ۶۳۰ گام به دلیل موفقیت یا پایان انرژی خاتمه می‌یابد. محدودیت گام صرفاً به‌عنوان یک سازوکار ایمنی در محیط نگه داشته شده است.

برای همه حالت‌های پایانی، ارزش ادامه مسیر برابر صفر در نظر گرفته می‌شود:

$$V(s) = 0 \quad \text{for} \quad s \in \mathcal{S}_T$$

به همین دلیل، در به‌روزرسانی‌های الگوریتمی از حالت پایانی bootstrap انجام نمی‌شود.

۳-۷. سیاست

سیاست، نحوه انتخاب عمل توسط عامل در هر حالت را مشخص می‌کند. یک سیاست تصادفی به صورت احتمال انتخاب هر عمل در یک حالت تعریف می‌شود:

$$\pi(a|s) = \Pr(A_t = a | S_t = s)$$

برای هر حالت، مجموع احتمال اعمال باید برابر یک باشد:

$$\sum_{a \in \mathcal{A}} \pi(a|s) = 1$$

اگر سیاست در هر حالت یک عمل مشخص را انتخاب کند، سیاست قطعی است و می‌توان آن را به صورت زیر نوشت:

$$\pi(s) = a$$

هدف مسئله یافتن سیاستی است که بازده مورد انتظار را بیشینه کند:

$$J(\pi) = E_{\pi}[G_0]$$

سیاست بهینه به صورت زیر تعریف می‌شود:

$$\pi^* = \arg \max_{\pi} J(\pi)$$

۳-۷-۱. سیاست Value Iteration

پس از همگرایی تابع ارزش، سیاست Value Iteration با انتخاب عملی استخراج می‌شود که بیشترین ارزش مورد انتظار را داشته باشد:

$$\pi_{\{VI\}}(s) = \arg \max_{a \in \mathcal{A}} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')]$$

این سیاست قطعی است و به عنوان سیاست مرجع برای مقایسه با Q-Learning و SARSA(λ) استفاده می‌شود.

۳-۷-۲. سیاست رفتاری در الگوریتم‌های مدل آزاد

در Q-Learning و SARSA(λ)، عامل هنگام آموزش از سیاست ϵ -greedy استفاده می‌کند:

$$\pi_{\epsilon}(a|s) =$$

برای عمل حریصانه، احتمال انتخاب بالا است و با احتمال ϵ ، یک عمل تصادفی انتخاب می‌شود. شکل فشرده این تصمیم‌گیری عبارت است از:

$$a_t = \begin{cases} \text{random action ,} & \text{with probability } \epsilon, \\ \arg \max_a Q(s_t, a), & \text{with probability } 1 - \epsilon. \end{cases}$$

در صورت برابر بودن مقدار چند عمل، پیاده‌سازی Q-Learning و SARSA یکی از اعمال دارای بیشترین مقدار را به صورت تصادفی انتخاب می‌کند. این روش مانع ایجاد ترجیح مصنوعی و ثابت نسبت به یک جهت خاص در حالت‌های دارای مقادیر برابر می‌شود.

در مرحله ارزیابی، اکتشاف حذف می‌شود و عامل فقط سیاست حریصانه را اجرا می‌کند:

$$\pi_{\text{eval}}(s) = \arg \max_{a \in \mathcal{A}} Q(s, a)$$

جزئیات کاهش ϵ و تفاوت سیاست‌های Q-Learning و SARSA در فصل‌های هفتم و هشتم بررسی می‌شوند.

۸-۳. تابع ارزش حالت و عمل

تابع ارزش حالت، بازده مورد انتظار از یک حالت را در صورت پیروی از سیاست π مشخص می‌کند:

$$V^{\pi}(s) = E_{\pi}[G_t | S_t = s]$$

تابع ارزش عمل نیز بازده مورد انتظار از انتخاب عمل a در حالت s و ادامه‌دادن براساس سیاست π است:

$$Q^{\pi}(s, a) = E_{\pi}[G_t | S_t = s, A_t = a]$$

تابع ارزش بهینه به صورت زیر تعریف می‌شود:

$$V^*(s) = \max_{\pi} V^{\pi}(s)$$

و تابع ارزش عمل بهینه برابر است با:

$$Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$$

رابطه میان این دو تابع را می‌توان چنین نوشت:

$$V^*(s) = \max_{a \in \mathcal{A}} Q^*(s, a)$$

Value Iteration مستقیماً تابع V را برای تمام حالت‌های فهرست‌شده محاسبه می‌کند و سپس سیاست را استخراج می‌کند. Q-Learning و SARSA(λ) جدول Q را از نمونه‌های تعامل با محیط می‌آموزند.

۹-۳. حفظ خاصیت مارکوف

خاصیت مارکوف بیان می‌کند که توزیع حالت بعدی و پاداش، با داشتن حالت و عمل فعلی، به تاریخچه پیشین وابسته نیست:

$$P(S_{t+1}, R_{t+1} | S_0, A_0, \dots, S_t, A_t) = P(S_{t+1}, R_{t+1} | S_t, A_t)$$

در این پروژه، موقعیت مکانی به‌تنهایی برای حفظ این خاصیت کافی نیست. وضعیت کلید بر امکان عبور از در اثر دارد و انرژی باقی‌مانده نیز تعیین می‌کند که آیا عامل پس از عمل بعدی قادر به ادامه اپیزود خواهد بود یا خیر.

با تعریف حالت به‌صورت $s_t = (r_t, c_t, k_t, e_t)$ تمام اطلاعات متغیر موردنیاز برای تعیین انتقال بعدی در حالت فعلی وجود دارد. ساختار نقشه، موقعیت دیوارها، کلید، در و هدف برای هر محیط ثابت است و از فایل نقشه خوانده می‌شود؛ بنابراین، لازم نیست این اطلاعات در هر حالت تکرار شوند.

همچنین، احتمال لغزش فقط به عمل انتخاب‌شده فعلی وابسته است و از اعمال قبلی تأثیر نمی‌پذیرد. در نتیجه، حرکت تصادفی نیز خاصیت مارکوف را نقض نمی‌کند.

شمارنده گام در نمایش حالت وجود ندارد. باین‌حال، انرژی اولیه ۵۰ واحد و سقف گام ۶۳۰ است؛ بنابراین، در آزمایش‌های اصلی شرط محدودیت گام پیش از پایان انرژی فعال نمی‌شود. در این تنظیمات، حذف شمارنده گام از حالت بر انتقال‌های قابل وقوع اثر نمی‌گذارد.

۱۰-۳. تفکیک مدل‌محور و مدل‌آزاد

مدل MDP در پیاده‌سازی به دو شیوه متفاوت در اختیار الگوریتم‌ها قرار می‌گیرد.

Value Iteration به تابع انتقال کامل دسترسی دارد و برای هر جفت حالت-عمل، تمام حالت‌های بعدی ممکن را به همراه احتمال و پاداش آن‌ها دریافت می‌کند:

$$\{(p_i, s'_i, r_i, d_i)\}_{i=1}^m$$

بنابراین، این الگوریتم می‌تواند امید ریاضی بازده هر عمل را بدون نمونه‌برداری مستقیم محاسبه کند.

در مقابل، Q-Learning و SARSA(λ) فقط با فراخوانی محیط تعامل می‌کنند:

$$(s_{t+1}, r_{t+1}, \text{done}, \text{info}) = \text{step}(a_t)$$

این دو الگوریتم به احتمال‌های 0.8، 0.1 و 0.1 یا فهرست حالت‌های بعدی دسترسی مستقیم ندارند. آن‌ها باید رفتار تصادفی محیط را از انتقال‌های مشاهده‌شده بیاموزند. این جداسازی، مقایسه منصفانه یک الگوریتم مدل‌محور با دو الگوریتم مدل‌آزاد را امکان‌پذیر می‌کند.

۱۱-۳. هدف بهینه‌سازی

هدف نهایی عامل، بیشینه‌کردن بازده تنزیل‌شده مورد انتظار از حالت اولیه است:

$$\max_{\pi \in \Pi} \left[\sum_{t=0}^{T-1} \gamma^t R_{t+1} \right]$$

با توجه به ساختار پاداش، این هدف عامل را به رفتارهای زیر هدایت می‌کند:

- رسیدن به کلید و سپس هدف؛
- کاهش تعداد گام‌های اضافی؛
- اجتناب از برخورد با دیوار؛
- کاهش تلاش‌های ناموفق برای عبور از در؛
- اجتناب از خانه‌های جریمه؛
- مدیریت انرژی و جلوگیری از پایان ناموفق اپیزود.

با این حال، کوتاه‌ترین مسیر هندسی همیشه سیاست بهینه نیست. به دلیل تصادفی بودن حرکت، یک مسیر کمی طولانی‌تر ممکن است از دیوارها و خانه‌های جریمه فاصله بیشتری داشته باشد و در نتیجه بازده مورد انتظار بالاتری ایجاد کند. سیاست بهینه باید میان طول مسیر، احتمال لغزش، ایمنی مسیر و محدودیت انرژی تعادل برقرار کند.

فصل چهارم: طراحی و پیاده‌سازی محیط

۴-۱. رویکرد طراحی محیط

پس از تعریف مسئله و مدل‌سازی رسمی آن در قالب یک فرایند تصمیم‌گیری مارکوف، در این فصل نحوه طراحی و پیاده‌سازی محیط هزارتو تشریح می‌شود. محیط باید افزون بر اجرای صحیح قوانین مسئله، امکان استفاده هم‌زمان در الگوریتم‌های مدل‌محور و مدل‌آزاد را فراهم کند. همچنین، تولید نقشه، اجرای انتقال‌های تصادفی، مدیریت انرژی، ثبت رخدادها و ذخیره نتایج باید به‌صورت بازتولیدپذیر انجام شوند.

پیاده‌سازی محیط در دو فایل اصلی انجام شده است. فایل `environments/generator.py` مسئول تولید و اعتبارسنجی نقشه است و فایل `environments/maze.py` منطق MDP، اجرای انتقال‌ها، مدیریت اپیزود و ثبت رخدادها را بر عهده دارد. نقشه‌های تولیدشده نیز به‌صورت فایل JSON در پوشه `environments/maps/` ذخیره می‌شوند. این فصل براساس نسخه ZIP بارگذاری‌شده از مخزن پروژه تهیه شده است.

یکی از تصمیم‌های اصلی در طراحی نرم‌افزار، جداسازی «تعریف نقشه» از «منطق محیط» بوده است. در نتیجه، ساختار دیوارها، خانه‌های جریمه، نقطه شروع، کلید، در و هدف در فایل نقشه ذخیره می‌شود، درحالی‌که قوانین حرکت و پاداش مستقل از یک نقشه خاص در کلاس `MazeEnv` پیاده‌سازی شده‌اند. این جداسازی امکان اجرای همان الگوریتم‌ها روی نقشه مبدأ و نقشه‌های مقصد انتقال یادگیری را بدون تغییر کد عامل فراهم می‌کند.

۴-۲. ساختار کلی فایل‌های مرتبط با محیط

فایل `generator.py` نقشه مبدأ را از `seed` وابسته به شماره دانشجویی تولید می‌کند. فایل `maze.py` شامل کلاس `MazeEnv` و کلاس ثبت رخداد `MazeLogger` است. فایل‌های JSON پوشه `maps` ساختار نقشه‌ها و فراداده‌های آن‌ها را نگهداری می‌کنند.

فایل `test_environment.py` شامل آزمون‌های واحد مربوط به تولید نقشه و رفتار محیط است. اسکریپت‌های پوشه `experiments` نیز محیط را بارگذاری کرده و برای اجرای الگوریتم‌های مختلف استفاده می‌کنند. این ساختار موجب می‌شود کد مربوط به عامل، تولید نقشه، اجرای آزمایش و تحلیل نتایج از یکدیگر جدا باشند.

۳-۴. نمایش عددی خانه‌های نقشه

نقشه هزار تو به صورت یک ماتریس دوبعدی از اعداد صحیح ذخیره شده است. هر مقدار عددی نوع یک خانه را مشخص می‌کند. کدهای تعریف شده در `generator.py` عبارت‌اند از:

- مقدار صفر برای خانه عادی و قابل عبور؛
- مقدار یک برای دیوار؛
- مقدار دو برای خانه جریمه؛
- مقدار سه برای نقطه شروع؛
- مقدار چهار برای کلید؛
- مقدار پنج برای در؛
- مقدار شش برای هدف.

این کدها تنها در `generator.py` تعریف شده‌اند و `maze.py` همان مقادیر را از این فایل وارد می‌کند. بنابراین، دو تعریف مستقل و ناسازگار برای انواع خانه‌ها در پروژه وجود ندارد. این تصمیم در کد با عنوان «منبع واحد حقیقت» در نظر گرفته شده است.

برای نمایش متنی نقشه نیز به هر نوع خانه یک نماد اختصاص داده شده است. خانه عادی با نقطه، دیوار با علامت `#`، خانه جریمه با `*`، نقطه شروع با `S`، کلید با `K`، در با `D` و هدف با `G` نمایش داده می‌شود. عامل در نمایش متنی با حرف `A` مشخص می‌شود.

نمایش عددی برای ذخیره‌سازی و محاسبات مناسب است، درحالی‌که نمایش متنی برای کنترل اولیه نقشه و اشکال‌زدایی استفاده می‌شود. نمایش گرافیکی کامل محیط نیز در بخش رابط کاربری انجام می‌شود که در فصل یازدهم بررسی خواهد شد.

۴-۴. تولید نقشه مبدأ

۴-۴-۱. تعیین `seed` و اندازه نقشه

در فایل `generator.py` شماره دانشجویی به صورت یک ثابت ذخیره شده است:

```
STUDENT_ID="40301694"
```

رقم یکی مانده به آخر شماره دانشجویی به عنوان `seed` پایه انتخاب می‌شود. بنابراین:

$$\text{base_seed} = 9$$

اندازه هزارتو نیز براساس رابطه تعیین شده در صورت پروژه محاسبه می شود:

$$N = 15 + (\text{base_seed} \bmod 4) = 16$$

در نتیجه، نقشه مبدأ دارای ۱۶ سطر و ۱۶ ستون است. استفاده از seed وابسته به شماره دانشجویی باعث می شود ساختار محیط برای هر دانشجو متفاوت باشد، اما در اجرای مجدد همان پروژه تغییر نکند.

۴-۴-۲. ایجاد ماتریس اولیه

در آغاز تولید هر نقشه پیشنهادی، یک ماتریس 16×16 ایجاد می شود که تمام خانه های آن از نوع عادی هستند. سپس عناصر ویژه و موانع به تدریج به این ماتریس افزوده می شوند.

ترتیب کلی تولید نقشه به صورت زیر است:

۱. ایجاد شبکه خالی؛
۲. ساخت محفظه هدف و قرار دادن در؛
۳. تعیین نقطه شروع؛
۴. انتخاب محل کلید؛
۵. افزودن دیوارهای تصادفی؛
۶. افزودن خانه های جریمه؛
۷. اعتبارسنجی نقشه با BFS؛
۸. ذخیره نقشه در صورت معتبر بودن.

این ترتیب مهم است، زیرا برخی خانه ها باید پیش از افزودن موانع رزرو شوند تا دیوارهای تصادفی روی آن ها قرار نگیرند.

۴-۵. طراحی محفظه هدف و در

برای آنکه کلید و در نقش واقعی در مسئله داشته باشند، هدف در یک محفظه بسته قرار داده شده است. مختصات هدف براساس اندازه نقشه به صورت زیر تعیین می شود:

$$\text{Goal} = (N - 2, N - 2)$$

برای نقشه 16×16 ، موقعیت هدف برابر است با:

$$\text{Goal} = (14, 14)$$

در مستقیماً در خانه بالای هدف قرار می‌گیرد:

$$\text{Door} = (N - 3, N - 2) = (13, 14)$$

تمام خانه‌های مجاور هدف به دیوار تبدیل می‌شوند، به جز خانه بالایی که به عنوان در در نظر گرفته شده است. همچنین، خانه بیرونی در، یعنی $(12, 14)$ ، به صورت قطعی قابل عبور نگه داشته می‌شود تا ورودی محفظه به وسیله دیوارهای تصادفی مسدود نشود.

این طراحی سبب می‌شود تنها راه ورود به هدف، عبور از خانه در باشد. اگر در فقط به صورت یک خانه عادی در میان نقشه قرار می‌گرفت، ممکن بود عامل از مسیر دیگری به هدف برسد و دریافت کلید تأثیری بر موفقیت مأموریت نداشته باشد.

خانه‌های محفظه هدف، در و فضای بیرونی آن در مجموعه‌ای از خانه‌های رزرو شده قرار می‌گیرند. تولیدکننده اجازه نمی‌دهد در مراحل بعدی، دیوار یا خانه جریمه روی این خانه‌ها قرار گیرد.

۴-۶. تعیین نقطه شروع

موقعیت شروع به صورت ثابت در گوشه بالا و چپ نقشه قرار داده شده است:

$$\text{Start} = (0, 0)$$

علاوه بر خانه شروع، خانه‌های قابل دسترسی مجاور آن نیز رزرو می‌شوند. این کار از محصور شدن کامل عامل در ابتدای اپیزود جلوگیری می‌کند. به عبارت دیگر، تولید دیوارهای تصادفی نمی‌تواند تمام خروجی‌های نقطه شروع را مسدود کند.

ثابت بودن نقطه شروع موجب می‌شود مقایسه میان الگوریتم‌ها ساده‌تر باشد. تمام الگوریتم‌ها اپیزودهای خود را از موقعیت یکسان، بدون کلید و با انرژی یکسان آغاز می‌کنند.

۴-۷. انتخاب محل کلید

برخلاف نقطه شروع و هدف، محل کلید از میان خانه‌های آزاد به صورت تصادفی انتخاب می‌شود. با این حال، همه خانه‌های آزاد نامزد انتخاب نیستند. محل کلید باید شرایط فاصله‌ای مشخصی داشته باشد تا بیش از حد به شروع یا هدف نزدیک نباشد.

فاصله منتهن کلید تا نقطه شروع باید حداقل نصف اندازه نقشه باشد:

$$d_{\text{Manhattan}}(\text{Key}, \text{Start}) \geq N/2$$

برای نقشه فعلی، حداقل فاصله برابر ۸ است. همچنین فاصله منتهن کلید تا هدف باید حداقل چهار خانه باشد:

$$d_{\text{Manhattan}}(\text{Key}, \text{Goal}) \geq 4$$

پس از تشکیل مجموعه خانه‌های واجد شرایط، یکی از آن‌ها با مولد اعداد تصادفی محلی انتخاب می‌شود. در نقشه نهایی پروژه، کلید در مختصات زیر قرار گرفته است:

$$\text{Key} = (0, 10)$$

پس از انتخاب، خانه کلید نیز به مجموعه خانه‌های رزرو شده افزوده می‌شود تا در مراحل بعدی به دیوار یا خانه جریمه تبدیل نشود.

۸-۴. تولید دیوارها

حداقل نسبت دیوار مورد نیاز در صورت پروژه ۱۵ درصد است. در پیاده‌سازی، نسبت هدف برابر ۱۸ درصد در نظر گرفته شده است تا شرط حداقل با حاشیه مطمئن برقرار باشد.

تعداد هدف دیوارها با رابطه زیر محاسبه می‌شود:

$$N_{\text{wall}} = \max([0.15N^2] + 2, \text{round}(0.18N^2))$$

برای $N=16$ ، تعداد هدف دیوارها برابر ۴۶ است:

$$N_{\text{wall}} = 46$$

تعدادی از این دیوارها پیش‌تر هنگام ساخت محفظه هدف ایجاد شده‌اند. سپس فهرست خانه‌های آزاد و رزرو شده تشکیل و به صورت تصادفی مرتب می‌شود. خانه‌های این فهرست تا رسیدن تعداد کل دیوارها به مقدار هدف، به دیوار تبدیل می‌شوند.

در نقشه ذخیره شده، نسبت واقعی دیوارها برابر است با:

$$\text{wall_ratio} = 46/256 = 0.1797$$

بنابراین، حدود ۱۷،۹۷ درصد خانه‌های محیط دیوار هستند و شرط حداقل ۱۵ درصد رعایت شده است.

۴-۹. تولید خانه‌های جریمه

پس از تولید دیوارها، فهرست جدیدی از خانه‌های عادی، قابل عبور و رزوننده تشکیل می‌شود. هفت خانه از این فهرست به صورت تصادفی انتخاب و به خانه جریمه تبدیل می‌شوند:

$$N_{\text{penalty}} = 7$$

این مقدار از حداقل پنج خانه جریمه تعیین شده در صورت پروژه بیشتر است. خانه‌های جریمه قابل عبور باقی می‌مانند و فقط هنگام ورود عامل، جریمه اضافی ایجاد می‌کنند.

قرار دادن خانه‌های جریمه پس از دیوارها تضمین می‌کند که هیچ خانه جریمه‌ای داخل دیوار قرار نگیرد. همچنین، نقطه شروع، کلید، در، هدف و خانه‌های رزوننده اطراف محفظه هدف نمی‌توانند به خانه جریمه تبدیل شوند.

۴-۱۰. تولید قطعی و بازتولیدپذیر نقشه

اگر تنها یک seed ثابت به مولد تصادفی داده شود، تولید نقشه بازتولیدپذیر خواهد بود. با این حال، ممکن است اولین نقشه تولید شده شرایط دسترسی را برآورده نکند. بنابراین، در پیاده‌سازی امکان تولید مجدد نقشه پیش‌بینی شده است.

برای هر تلاش، seed مؤثر به صورت زیر محاسبه می‌شود:

$$\text{seed}_i = \text{base_seed} \times 100000 + i$$

در این رابطه، i شماره تلاش تولید نقشه است. تلاش اول با $i=0$ انجام می‌شود. اگر نقشه نامعتبر باشد، مقدار i یک واحد افزایش پیدا می‌کند و تلاش بعدی با seed جدید اما قابل پیش‌بینی انجام می‌شود.

این روش دو ویژگی دارد. نخست، نقشه نامعتبر می‌تواند بدون دخالت دستی کنار گذاشته شود. دوم، اجرای مجدد برنامه همان دنباله نقشه‌های پیشنهادی را تولید می‌کند. بنابراین، اولین نقشه معتبر در همه اجراها یکسان خواهد بود.

حداکثر ۵۰۰ تلاش برای پیدا کردن نقشه معتبر در نظر گرفته شده است. در نسخه فعلی پروژه، نقشه در همان تلاش صفر معتبر بوده است:

attempt = 0

در نتیجه، نیازی به بازتولید نقشه در تلاش‌های بعدی وجود نداشته است.

۴-۱۱. اعتبارسنجی نقشه با BFS

۴-۱۱-۱. نقش BFS در پروژه

الگوریتم BFS فقط برای بررسی اعتبار ساختاری نقشه و محاسبه فاصله‌های کوتاه‌ترین مسیر استفاده می‌شود. این الگوریتم در تصمیم‌گیری عامل، آموزش Q-Learning، آموزش SARSA یا استخراج سیاست استفاده نمی‌شود.

تابع bfs_distance حرکت چهارج جهت را در شبکه بررسی می‌کند. دیوارها همواره غیرقابل عبور هستند. در نیز بسته به ورودی door_passable یا مسدود و یا قابل عبور در نظر گرفته می‌شود.

خروجی تابع تعداد گام‌های کوتاه‌ترین مسیر است. اگر مسیر وجود نداشته باشد، مقدار None بازگردانده می‌شود.

۴-۱۱-۲. شرط دسترسی از شروع به کلید

اولین شرط اعتبار این است که عامل بتواند در حالت بسته‌بودن در، از نقطه شروع به کلید برسد. این بررسی به صورت زیر انجام می‌شود:

$d_{start,key} = \text{BFS Start,Key,door closed}$

شرط معتبر بودن این مسیر عبارت است از:

$d_{start,key} \neq \text{None}$

در نقشه فعلی، کوتاه‌ترین فاصله از شروع تا کلید برابر ۱۲ گام است:

$d_{start,key} = 12$

۴-۱۱-۳. شرط دسترسی از کلید به هدف

دومین شرط این است که پس از دریافت کلید و باز شدن در، مسیری از کلید به هدف وجود داشته باشد:

$$d_{\text{key,goal}} = \text{BFS Key,Goal,door open}$$

در نقشه فعلی، این فاصله برابر ۱۸ گام است:

$$d_{\text{key,goal}} = 18$$

بنابراین، کوتاه‌ترین مسیر ترکیبی قطعی برابر است با:

$$d_{\text{total}} = 12 + 18 = 30$$

۴-۱۱-۴. جلوگیری از دسترسی بدون کلید

شرط سوم بررسی می‌کند که هدف در حالت بسته‌بودن در از نقطه شروع قابل دسترسی نباشد:

$$\text{BFS(Start,Goal,door closed)} = \text{None}$$

این شرط تضمین می‌کند که عامل نتواند بدون دریافت کلید به هدف برسد. اگر این مسیر وجود داشت، نقشه رد می‌شد و تولیدکننده نقشه جدیدی ایجاد می‌کرد.

در نتیجه، یک نقشه تنها در صورتی پذیرفته می‌شود که هر سه شرط زیر هم‌زمان برقرار باشند:

$$\begin{aligned} d_{\text{start,key}} &\neq \text{None} \\ d_{\text{key,goal}} &\neq \text{None} \\ d_{\text{start,goal}}^{\text{door closed}} &= \text{None} \end{aligned}$$

۴-۱۱-۵. بررسی تناسب مسیر با انرژی

پس از تأیید مسیرها، طول کوتاه‌ترین مسیر کامل با بودجه انرژی مقایسه می‌شود. اگر طول مسیر بیش از ۷۰ درصد انرژی اولیه باشد، تولیدکننده یک هشدار ثبت می‌کند:

$$d_{\text{total}} > 0.7E_0$$

در محیط فعلی:

$$30 < 0.7 \times 50 = 35$$

بنابراین، هشدار امکان‌پذیری انرژی فعال نشده است. این بررسی نقشه را رد نمی‌کند، بلکه فقط درباره احتمال بالای پایان انرژی در حضور لغزش تصادفی هشدار می‌دهد.

۱۲-۴. ذخیره نقشه در قالب JSON

پس از یافتن نقشه معتبر، تابع `save_map` آن را در فایل `source_map.json` ذخیره می‌کند. این فایل فقط ماتریس شبکه را نگهداری نمی‌کند، بلکه فراداده‌های لازم برای بازتولید و تحلیل محیط را نیز شامل می‌شود.

اطلاعات ذخیره‌شده عبارت‌اند از:

- شماره دانشجویی؛
- `seed` پایه؛
- شماره تلاش موفق؛
- اندازه نقشه؛
- ماتریس کامل خانه‌ها؛
- موقعیت شروع؛
- موقعیت کلید؛
- موقعیت در؛
- موقعیت هدف؛
- تعداد و نسبت دیوارها؛
- تعداد خانه‌های جریمه؛
- انرژی اولیه؛
- فاصله شروع تا کلید؛
- فاصله کلید تا هدف؛
- وضعیت هشدار امکان‌پذیری انرژی.

استفاده از JSON دو مزیت اصلی دارد. نخست، نقشه به‌صورت خوانا و مستقل از کد ذخیره می‌شود. دوم، تمام الگوریتم‌ها دقیقاً همان فایل را بارگذاری می‌کنند و نقشه در آغاز هر آزمایش دوباره تولید نمی‌شود.

تابع `load_map` فایل JSON را خوانده و موقعیت‌های شروع، کلید، در و هدف را از لیست به `tuple` تبدیل می‌کند تا استفاده از آن‌ها در منطق محیط ساده‌تر باشد.

۴-۱۳. طراحی کلاس MazeEnv

کلاس MazeEnv هسته اجرایی محیط است. این کلاس از یک کتابخانه آماده یادگیری تقویتی ارث‌بری نمی‌کند و API محیط به صورت مستقیم در پروژه پیاده‌سازی شده است.

پارامترهای اصلی سازنده کلاس عبارت‌اند از:

- داده یا نام فایل نقشه؛
- نوع تابع پاداش؛
- مقادیر پاداش‌ها؛
- ضریب شکل‌دهی پاداش؛
- ضریب تنزیل شکل‌دهی؛
- حداکثر تعداد گام‌ها؛
- seed مولد تصادفی.

اگر داده نقشه مستقیماً به سازنده داده نشود، محیط فایل `source_map.json` را بارگذاری می‌کند. سپس ماتریس نقشه، اندازه، موقعیت عناصر اصلی و انرژی اولیه در متغیرهای داخلی کلاس ذخیره می‌شوند.

کلاس از دو حالت پاداش `sparse` و `shaped` پشتیبانی می‌کند. جزئیات این دو تابع در فصل پنجم ارائه می‌شود.

۴-۱۴. شناسایی خانه‌های قابل عبور

در سازنده کلاس، تمام خانه‌هایی که دیوار نیستند در فهرست `walkable` قرار می‌گیرند. این فهرست برای محاسبه سقف تعداد گام‌ها و تولید فضای حالت کامل در Value Iteration استفاده می‌شود.

در نقشه فعلی:

$$N_{\text{walkable}} = 256 - 46 = 210$$

حداکثر تعداد گام‌های اپیزود نیز سه برابر تعداد خانه‌های قابل عبور تعیین می‌شود:

$$\text{max_steps} = 3 \times 210 = 630$$

از آنجا که انرژی اولیه ۵۰ است، در تنظیمات فعلی معمولاً پایان انرژی بسیار زودتر از سقف ۶۳۰ گام رخ می‌دهد. سقف گام بیشتر یک سازوکار ایمنی برای جلوگیری از اجرای نامحدود اپیزودها در صورت تغییر انرژی یا تنظیمات محیط است.

۱۵-۴. تابع reset

تابع reset محیط را به وضعیت اولیه بازمی‌گرداند. حالت اولیه به صورت زیر ساخته می‌شود:

$$s_0 = (r_{\text{start}}, c_{\text{start}}, 0, E_0)$$

در نقشه فعلی:

$$s_0 = (0, 0, 0, 50)$$

در هنگام reset، شمارنده گام صفر می‌شود و وضعیت پایان اپیزود غیرفعال می‌گردد. این تابع همچنین می‌تواند یک seed جدید دریافت کند. در صورت ارائه seed، مولد تصادفی محلی محیط دوباره مقداردهی می‌شود.

قابلیت تعیین seed در reset باعث می‌شود یک دنباله انتقال تصادفی مشخص را بتوان در اجرای مجدد بازسازی کرد. این ویژگی برای آزمایش پایداری و آزمون‌های واحد اهمیت دارد.

۱۶-۴. تابع step

تابع step رابط اصلی تعامل الگوریتم‌های مدل آزاد با محیط است. ورودی این تابع عمل انتخاب‌شده عامل است و خروجی آن شامل حالت بعدی، پاداش، وضعیت پایان و اطلاعات رخداده است:

$$(s_{t+1}, r_{t+1}, \text{done}, \text{info}) = \text{step}(a_t)$$

ابتدا یک عدد تصادفی بین صفر و یک تولید می‌شود. اگر مقدار آن کمتر از ۰٫۸ باشد، همان عمل انتخاب‌شده اجرا می‌شود. اگر مقدار بین ۰٫۸ و ۰٫۹ باشد، جهت عمود اول و در غیر این صورت جهت عمود دوم اجرا می‌شود.

پس از تعیین جهت مؤثر، تابع داخلی transition_ فراخوانی می‌شود. سپس شمارنده گام افزایش می‌یابد و شرط سقف تعداد گام‌ها بررسی می‌شود.

دیکشنری info شامل اطلاعات زیر است:

- نوع رخداد؛
- عمل انتخاب شده؛
- عمل مؤثر پس از لغزش؛
- وضعیت truncation؛
- تعداد گام‌های سپری شده.

جداسازی عمل انتخاب شده از عمل مؤثر امکان تحلیل مستقیم اثر تصادفی بودن محیط را فراهم می‌کند.

۴-۱۷. تابع انتقال قطعی داخلی

تابع `transition` یک جهت مؤثر را به صورت قطعی روی حالت فعلی اعمال می‌کند. تصادفی بودن در تابع `step` تابع مدل `transitions` مدیریت می‌شود و `transition` تنها نتیجه یک شاخه مشخص را محاسبه می‌کند.

در آغاز، مختصات پیشنهادی از جمع موقعیت فعلی و تغییر مختصات عمل به دست می‌آید. سپس وضعیت خانه مقصد بررسی می‌شود.

اگر خانه مقصد خارج از مرز یا دیوار باشد، عامل در موقعیت قبلی باقی می‌ماند و رخداد `WALL_HIT` ثبت می‌شود. اگر مقصد در باشد و عامل کلید نداشته باشد، موقعیت تغییر نمی‌کند و رخداد `DOOR_LOCKED_ATTEMPT` ثبت می‌شود.

اگر عامل وارد خانه کلید شود و هنوز کلید را دریافت نکرده باشد، مقدار متغیر کلید به یک تغییر می‌کند. ورود به در پس از دریافت کلید با رخداد `DOOR_PASS` و ورود به خانه جریمه با رخداد `PENALTY_CELL` ثبت می‌شود.

در تمام حالات، حتی برخورد با دیوار یا در بسته، یک واحد انرژی مصرف می‌شود:

$$e_{t+1} = e_t - 1$$

پس از اجرای حرکت، ابتدا شرط رسیدن به هدف بررسی می‌شود. اگر عامل وارد هدف شده باشد، اپیزود با موفقیت پایان می‌یابد. در غیر این صورت، اگر انرژی به صفر رسیده باشد، اپیزود با شکست خاتمه پیدا می‌کند. این ترتیب باعث می‌شود رسیدن به هدف با آخرین واحد انرژی همچنان موفقیت محسوب شود.

۴-۱۸. دسترسی مدل محور با تابع `transitions`

الگوریتم Value Iteration باید به تمام انتقال‌های ممکن هر جفت حالت-عمل دسترسی داشته باشد. به همین دلیل، کلاس MazeEnv علاوه بر تابع نمونه‌برداری step، تابع transitions را ارائه می‌کند.

این تابع سه شاخه زیر را بررسی می‌کند:

$$\begin{aligned} &(0.8, a) \\ &(0.1, a_{\perp,1}) \\ &(0.1, a_{\perp,2}) \end{aligned}$$

برای هر شاخه، تابع transition_فراخوانی می‌شود و حالت بعدی، پاداش و وضعیت پایان محاسبه می‌گردند. اگر چند جهت مؤثر به نتیجه یکسانی منتهی شوند، احتمال‌های آن‌ها با یکدیگر جمع می‌شوند.

برای مثال، اگر دو جهت مختلف هر دو به دیوار برخورد کنند و حالت و پاداش یکسانی تولید کنند، به‌جای دو خروجی جداگانه با احتمال ۰.۱، یک خروجی با احتمال مجموع ذخیره می‌شود.

این تابع فقط توسط Value Iteration استفاده می‌شود. Q-Learning و SARSA(λ) باید از reset و step استفاده کنند و به مدل کامل انتقال دسترسی ندارند. این جداسازی برای حفظ ماهیت مدل‌آزاد این دو الگوریتم ضروری است.

۱۹-۴. تولید فضای حالت کامل

تابع all_states تمام حالت‌های غیرپایانی موردنیاز Value Iteration را تولید می‌کند. برای هر خانه قابل عبور، دو مقدار وضعیت کلید و تمام سطوح انرژی از یک تا ۵۰ بررسی می‌شوند. دو نوع حالت از این فهرست حذف می‌شوند:

۱. حالت‌هایی که عامل در خانه هدف قرار دارد، زیرا هدف پایانی است؛

۲. حالت قرارگرفتن روی در بدون کلید، زیرا این حالت با قوانین محیط قابل دستیابی نیست.

تابع is_terminal نیز یک حالت را در صورت قرارگرفتن روی هدف یا صفرشدن انرژی، پایانی تشخیص می‌دهد.

برخی از حالت‌های تولیدشده ممکن است از حالت اولیه عملاً قابل دسترس نباشند، اما نگهداری فضای حالت ساختاری کامل، پیاده‌سازی Value Iteration را ساده و مستقل از مسیرهای نمونه‌برداری شده می‌کند.

۴-۲۰. محاسبه فاصله‌های BFS برای شکل‌دهی پاداش

در هنگام ایجاد محیط، دو نقشه فاصله BFS محاسبه می‌شوند:

- فاصله تمام خانه‌های قابل عبور تا کلید در حالت بسته‌بودن در؛
- فاصله تمام خانه‌های قابل عبور تا هدف در حالت بازبودن در.

این فاصله‌ها فقط برای تابع پاداش شکل‌دهی شده استفاده می‌شوند. هنگامی که عامل کلید ندارد، فاصله تا کلید و پس از دریافت کلید، فاصله تا هدف به‌عنوان پتانسیل استفاده می‌شود.

این محاسبات در آغاز ساخت محیط انجام می‌شوند تا در هر گام نیازی به اجرای مجدد BFS نباشد. در نتیجه، هزینه محاسباتی شکل‌دهی پاداش در طول آموزش کاهش می‌یابد.

BFS در این بخش نیز سیاست عامل را تعیین نمی‌کند. تنها یک مقدار کمکی برای تولید بازخورد پاداش است. تحلیل کامل آن در فصل پنجم ارائه می‌شود.

۴-۲۱. ثبت رخدادها با MazeLogger

برای ذخیره جزئیات انتقال‌ها، کلاس MazeLogger در فایل `maze.py` پیاده‌سازی شده است. این کلاس برای هر گام یک ردیف در فایل CSV ثبت می‌کند.

ستون‌های ثبت‌شده شامل موارد زیر هستند:

- شماره اپیزود؛
- شماره گام؛
- سطر و ستون حالت فعلی؛
- وضعیت کلید؛
- انرژی فعلی؛
- عمل انتخاب‌شده؛
- عمل مؤثر؛
- مختصات حالت بعدی؛
- وضعیت کلید بعدی؛
- انرژی بعدی؛

- پاداش؛
- نوع رخداد؛
- وضعیت پایان.

اگر فایل خروجی وجود نداشته باشد، عنوان ستون‌ها به صورت خودکار نوشته می‌شود. در صورت وجود فایل، داده‌ها به انتهای آن افزوده می‌شوند.

ثبت هم‌زمان عمل انتخاب‌شده و عمل مؤثر اهمیت ویژه‌ای دارد، زیرا اجازه می‌دهد موارد لغزش از تصمیم واقعی عامل تفکیک شوند. همچنین، با استفاده از ستون رخداد می‌توان تعداد برخورد با دیوار، ورود به خانه جریمه، دریافت کلید و سایر رفتارها را مستقیماً محاسبه کرد.

۴-۲۲. بازتولیدپذیری انتقال‌های تصادفی

کلاس MazeEnv از یک نمونه محلی random.Random استفاده می‌کند. بنابراین، تصادفی‌بودن محیط از وضعیت تصادفی سراسری پایتون مستقل است.

اگر دو محیط با نقشه، seed و دنباله اعمال یکسان اجرا شوند، دنباله یکسانی از جهت‌های مؤثر، حالت‌ها، پاداش‌ها و رخدادهای تولید می‌کنند. با تغییر seed، دنباله لغزش‌ها نیز تغییر خواهد کرد.

این طراحی برای مقایسه علمی الگوریتم‌ها مهم است. در آزمایش‌های کنترل‌شده می‌توان از seed های یکسان استفاده کرد تا الگوریتم‌ها در شرایط تصادفی قابل مقایسه قرار گیرند. در آزمایش پایداری نیز می‌توان با تغییر نظام‌مند seed، حساسیت نتایج را ارزیابی کرد.

۴-۲۳. آزمون‌های واحد محیط

فایل tests/test_environment.py مجموعه‌ای از آزمون‌های واحد را برای بررسی صحت تولیدکننده و محیط شامل می‌شود. در نسخه فعلی مخزن، ۱۷ آزمون تعریف شده است.

مهم‌ترین موارد بررسی‌شده عبارت‌اند از:

- محاسبه صحیح seed و اندازه نقشه؛
- رعایت حداقل تعداد دیوارها و خانه‌های جریمه؛
- دسترسی از شروع به کلید؛

- دسترسی از کلید به هدف پس از بازشدن در؛
- غیرقابل دسترس بودن هدف بدون کلید؛
- تولید یکسان نقشه در اجراهای تکراری؛
- مجموع احتمال‌های انتقال برابر یک؛
- کاهش انرژی در تمام گام‌ها؛
- ثابت ماندن موقعیت هنگام برخورد با دیوار؛
- تغییر صحیح وضعیت کلید؛
- مسدود بودن در بدون کلید؛
- امکان عبور از در پس از دریافت کلید؛
- پایانی بودن هدف؛
- پایان اپیزود با اتمام انرژی؛
- وابستگی مدل فقط به حالت و عمل فعلی؛
- سازگاری پاداش شکل‌دهی شده با پاداش تنک؛
- عملکرد صحیح محدودیت تعداد گام‌ها؛
- بازتولیدپذیری اپیزود با seed یکسان.

۴-۲۴. نحوه استفاده محیط در پروژه

هر اسکریپت آزمایشی یک نمونه از MazeEnv ایجاد می‌کند و نقشه مربوط را از فایل JSON می‌خواند. Value Iteration از تابع transitions و فهرست all_states استفاده می‌کند، درحالی‌که Q-Learning و SARSA فقط با reset و step تعامل دارند.

در رابط گرافیکی نیز همین کلاس محیط استفاده می‌شود. بنابراین، رفتار عامل در GUI با رفتاری که هنگام اجرای اسکریپت‌های آزمایشی مشاهده می‌شود، از یک منطق مشترک پیروی می‌کند.

فایل main.py پیش از اجرای رابط گرافیکی بررسی می‌کند که آیا source_map.json وجود دارد یا خیر. اگر فایل موجود نباشد، تولیدکننده به صورت خودکار اجرا و نقشه ذخیره می‌شود. سپس برنامه رابط کاربری را آغاز می‌کند.

۴-۲۵. ارزیابی طراحی محیط

طراحی انجام شده چند ویژگی مثبت دارد. نخست، نقشه به صورت قطعی و قابل بازتولید تولید می شود. دوم، اعتبار مسیرها پیش از اجرای آزمایش ها بررسی می گردد. سوم، منطق محیط از الگوریتم های یادگیری مستقل است. چهارم، یک API جداگانه برای دسترسی مدل محور در نظر گرفته شده است و الگوریتم های مدل آزاد به این مدل دسترسی ندارند.

استفاده از فایل JSON نیز موجب می شود تمام روش ها روی محیط یکسان اجرا شوند. ثبت رخدادها در فایل CSV امکان تحلیل جزئی رفتار عامل را فراهم می کند و آزمون های واحد، بخش مهمی از خطاهای مربوط به محیط را پیش از اجرای آموزش شناسایی می کنند.

با این حال، فضای حالت به دلیل قرارگرفتن انرژی در حالت نسبتاً بزرگ است. این مسئله در Value Iteration موجب افزایش تعداد حالت های مورد پردازش می شود. همچنین، نقشه در طول یک اپیزود از نظر موقعیت دیوارها ثابت است و پویایی محیط عمدتاً از انرژی، وضعیت کلید و انتقال تصادفی ناشی می شود. افزودن موانع متحرک یا درهای دوره ای می تواند پویایی بیشتری ایجاد کند، اما ابعاد فضای حالت و پیچیدگی مدل را نیز افزایش می داد.

فصل پنجم: طراحی و ارزیابی تابع پاداش

۵-۱. چارچوب طراحی تابع پاداش

تابع پاداش واسطه‌ای است که هدف کلی مسئله را به بازخوردهای عددی قابل استفاده برای عامل تبدیل می‌کند. در محیط این پروژه، عامل باید ضمن دستیابی به کلید و هدف، از مسیرهای طولانی، برخورد با دیوار، تلاش برای عبور از در قفل‌شده، ورود به خانه‌های جریمه و پایان انرژی اجتناب کند. از این‌رو، تابع پاداش باید هم موفقیت نهایی را تشویق کند و هم میان رفتارهای مطلوب و نامطلوب تمایز ایجاد نماید.

دو نسخه از تابع پاداش در محیط پیاده‌سازی شده است:

۱. پاداش تنک یا sparse که عمدتاً بر رویدادهای کلید و هدف متکی است؛
۲. پاداش شکل‌دهی‌شده یا shaped که علاوه بر پاداش تنک، از تغییر فاصله عامل تا زیرهدف جاری استفاده می‌کند.

در این فصل، ابتدا اجزای هر دو تابع معرفی می‌شوند، سپس منطق انتخاب مقادیر پاداش توضیح داده می‌شود و در پایان، اثر آن‌ها بر روند یادگیری Q-Learning و احتمال ایجاد رفتارهای ناخواسته تحلیل خواهد شد.

۵-۲. اهداف طراحی تابع پاداش

در طراحی تابع پاداش، چهار هدف اصلی در نظر گرفته شده است. نخست، رسیدن به هدف باید مطلوب‌ترین نتیجه ممکن باشد و نسبت به دریافت کلید به‌تنهایی ارزش بیشتری داشته باشد. دوم، عامل باید برای تکمیل مأموریت در تعداد گام کمتر تشویق شود. سوم، رفتارهایی مانند برخورد مکرر با دیوار یا ورود به خانه‌های جریمه باید از نظر عددی نامطلوب باشند. چهارم، پایان انرژی بدون دستیابی به هدف باید به‌عنوان شکست در نظر گرفته شود.

در کنار این اهداف، مقادیر پاداش نباید آن‌قدر بزرگ انتخاب شوند که اثر سایر اجزای مسئله از بین برود. برای مثال، اگر پاداش کلید بیش از حد زیاد باشد، عامل ممکن است دریافت کلید را تقریباً معادل تکمیل مأموریت تلقی کند. برعکس، اگر هزینه هر گام بیش از حد شدید باشد، پاداش دوردست هدف ممکن است نتواند از طریق به‌روزرسانی‌های زمانی به حالت‌های ابتدایی منتقل شود.

تمام پاداش‌ها در کلاس MazeEnv و در ساختار DEFAULT_REWARDS تعریف شده‌اند. این موضوع موجب می‌شود هر دو حالت تنک و شکل‌دهی‌شده از اجزای پایه یکسانی استفاده کنند و تنها مؤلفه هدایت‌کننده مبتنی بر فاصله در نسخه شکل‌دهی‌شده به آن‌ها افزوده شود.

۳-۵. تابع پاداش تنک

در نسخه تنک، هر عمل یک هزینه پایه برابر با منفی یک دارد. سپس، با توجه به رخداد حاصل از انتقال، پاداش یا جریمه اضافی به آن افزوده می‌شود. تعریف فشرده این تابع به صورت زیر است:

$$r_{\text{sparse}} = -1 - 4I_{\text{wall}} - 4I_{\text{locked}} - 9I_{\text{penalty}} + 50I_{\text{key}} + 100I_{\text{goal}} - 20I_{\text{energy}}$$

در این رابطه، هر متغیر I یک تابع شاخص است که در صورت وقوع رویداد متناظر مقدار یک و در غیر این صورت مقدار صفر می‌گیرد. برای مثال، هنگام برخورد با دیوار، $I_{\text{wall}}=1$ است و سایر شاخص‌های غیرمرتبط صفر باقی می‌مانند.

مقادیر پایه و پاداش مؤثر رویدادهای اصلی در جدول ۱-۵ ارائه شده‌اند.

رویداد	هزینه پایه گام	مؤلفه اضافی	پاداش مؤثر معمول
حرکت عادی	-1	0	-1
برخورد با دیوار یا مرز	-1	-4	-5
تلاش برای عبور از در قفل‌شده	-1	-4	-5
ورود به خانه جریمه	-1	-9	-10
دریافت کلید	-1	+50	+49
عبور از در پس از دریافت کلید	-1	0	-1
رسیدن به هدف	-1	+100	+99
پایان انرژی در یک حرکت عادی	-1	-20	-21

جدول ۱-۵: اجزای تابع پاداش تنک

مؤلفه‌های جدول قابلیت جمع‌شدن دارند. برای مثال، اگر انرژی عامل هنگام برخورد با دیوار به پایان برسد، هزینه گام، جریمه دیوار و جریمه پایان انرژی همگی در پاداش نهایی اثر می‌گذارند. در مقابل، رسیدن به هدف در اولویت قرار دارد؛ بنابراین، اگر عامل با آخرین واحد انرژی وارد هدف شود، اپیزود موفق در نظر گرفته می‌شود و جریمه پایان انرژی اعمال نمی‌گردد.

واژه «تنک» در این پروژه به معنای نبود کامل پاداش‌های میانی نیست. عامل در هر گام هزینه دریافت می‌کند و رویدادهای نامطلوب نیز جریمه دارند؛ باین‌حال، دو بازخورد مثبت اصلی تنها هنگام دریافت کلید و رسیدن به هدف ایجاد می‌شوند. در نتیجه، عامل در مراحل ابتدایی آموزش اطلاعات مستقیمی درباره مناسب‌بودن جهت حرکت در خانه‌های عادی دریافت نمی‌کند.

۵-۴. منطق انتخاب مقادیر پاداش

۵-۴-۱. هزینه هر گام

هزینه پایه هر عمل برابر 1- انتخاب شده است. این مقدار باعث می‌شود، در صورت یکسان‌بودن سایر شرایط، مسیر کوتاه‌تر نسبت به مسیر طولانی‌تر بازده بیشتری داشته باشد.

این هزینه به‌اندازه‌ای کوچک است که موفقیت مأموریت را تحت‌الشعاع قرار نمی‌دهد. برای مثال، کوتاه‌ترین مسیر قطعی کامل شامل ۳۰ گام است. در صورت نبود برخورد و خانه جریمه، بازده ثبت‌شده این مسیر برابر است با:

$$R_{shortest} = 50 + 100 - 30 = 120$$

بنابراین، حتی پس از کسر هزینه تمام گام‌های مسیر، دریافت کلید و هدف همچنان بازده مثبت قابل‌توجهی ایجاد می‌کند.

۵-۴-۲. جریمه برخورد با دیوار و در قفل‌شده

برخورد با دیوار و تلاش برای عبور از در بسته، هر دو پاداش مؤثر 5- دارند. این مقدار پنج برابر هزینه یک حرکت عادی است و به عامل نشان می‌دهد که انتخاب عمل نامعتبر از یک حرکت ساده نامطلوب‌تر است.

برای این دو رخداد مقدار یکسانی انتخاب شده است؛ زیرا پیامد فیزیکی آن‌ها مشابه است: عامل جابه‌جا نمی‌شود، اما یک گام و یک واحد انرژی را از دست می‌دهد. جریمه در بسته همچنین عامل را تشویق می‌کند پیش از حرکت به سمت محفظه هدف، ابتدا کلید را دریافت کند.

۵-۴-۳. جریمه خانه‌های خطرناک

ورود به یک خانه جریمه، پاداش مؤثر 10- ایجاد می‌کند. این خانه‌ها برخلاف دیوار قابل عبور هستند؛ بنابراین، عامل باید میان کوتاهی مسیر و هزینه عبور از آن‌ها تصمیم‌گیری کند.

مقدار 10- باعث می شود عبور از یک خانه جریمه تقریباً معادل هزینه ده حرکت عادی باشد. در نتیجه، یک مسیر اندکی طولانی تر که از خانه های جریمه عبور نمی کند می تواند نسبت به کوتاه ترین مسیر هندسی بازده بیشتری داشته باشد.

۵-۴-۴. پاداش کلید

دریافت کلید دارای پاداش اضافی 50+ است. کوتاه ترین فاصله شروع تا کلید در نقشه مبدأ ۱۲ گام است؛ بنابراین، رسیدن مستقیم به کلید بدون برخورد، بازده تقریبی زیر را ایجاد می کند:

$$R_{start_to_key} = 50 - 12 = 38$$

این مقدار مثبت کمک می کند زیرهدف کلید در مراحل آموزش قابل یادگیری باشد. با این حال، پاداش کلید تنها یک بار و هنگام تغییر وضعیت کلید از صفر به یک پرداخت می شود. بازگشت مجدد عامل به همان خانه، پاداش دیگری ایجاد نمی کند.

۵-۴-۵. پاداش هدف

پاداش اضافی هدف برابر 100+ و دو برابر پاداش کلید است. این تفاوت نشان می دهد دریافت کلید فقط یک مرحله میانی است و موفقیت اصلی با ورود به هدف حاصل می شود.

همچنین، هدف یک حالت پایانی است. در نتیجه، عامل نمی تواند با باقی ماندن در خانه هدف یا ورود مکرر به آن، پاداش را چند بار دریافت کند.

۵-۴-۶. جریمه پایان انرژی

اگر انرژی عامل پیش از دستیابی به هدف پایان یابد، جریمه اضافی 20- اعمال می شود. برای درک اثر این مقدار، فرض شود عامل تمام ۵۰ واحد انرژی را بدون دریافت کلید مصرف کند و هیچ جریمه دیگری نیز نداشته باشد. بازده اپیزود برابر خواهد بود با:

$$R_{failure_without_key} = -50 - 20 = -70$$

اگر عامل کلید را دریافت کند اما پیش از رسیدن به هدف انرژی آن پایان یابد، بازده تقریبی آن در حالت بدون جریمه دیگر برابر است با:

$$R_{failure_after_key} = -50 + 50 - 20 = -20$$

در مقابل، موفقیت در ۵۰ گام و بدون برخورد یا خانه جریمه بازده زیر را ایجاد می‌کند:

$$R_{success_in_50_steps} = -50 + 50 + 100 = 100$$

بنابراین، ترتیب ارزش‌گذاری به‌صورت منطقی برقرار است: تکمیل مأموریت از دریافت کلید و شکست پس از آن مطلوب‌تر است، و دریافت کلید نیز نسبت به شکست کامل اطلاعاتی درباره پیشرفت عامل ارائه می‌کند.

۵-۵. محدودیت پاداش تنک

در پاداش تنک، یک حرکت عادی در جهت درست و یک حرکت عادی در جهت نادرست هر دو پاداش 1- دارند، مگر آنکه به یک رویداد خاص مانند دریافت کلید، برخورد با مانع یا ورود به خانه جریمه منتهی شوند.

در نتیجه، عامل در آغاز آموزش نمی‌داند کدام حرکت آن را به کلید یا هدف نزدیک‌تر می‌کند. پاداش مثبت تنها پس از پیمودن بخش قابل‌توجهی از مسیر مشاهده می‌شود و باید از طریق به‌روزرسانی‌های مکرر به حالت‌های قبلی منتقل گردد. این مسئله به‌ویژه در محیط حاضر اهمیت دارد؛ زیرا عامل فقط ۵۰ واحد انرژی دارد و اعمال نیز تصادفی هستند. بنابراین، احتمال موفقیت تصادفی در اپیزودهای ابتدایی پایین است.

برای بررسی اینکه آیا بازخورد میانی می‌تواند این مشکل را کاهش دهد، نسخه دوم تابع پاداش براساس شکل‌دهی مبتنی بر پتانسیل طراحی شده است.

۵-۶. تابع پاداش شکل‌دهی شده

پاداش شکل‌دهی شده از پاداش تنک به‌عنوان پایه استفاده می‌کند و یک جمله هدایت‌کننده به آن می‌افزاید:

$$r_{shaped} = r_{sparse} + \eta F(s, s')$$

ضریب شکل‌دهی در آزمایش‌ها برابر یک است:

$$\eta = 1$$

جمله شکل‌دهی مبتنی بر پتانسیل به‌صورت زیر تعریف می‌شود:

$$F(s, s') = \gamma_{sh} \Phi(s') - \Phi(s)$$

ضریب مورد استفاده در شکل‌دهی برابر ضریب تنزیل آموزش است:

$$\gamma_{sh} = 0.99$$

تابع پتانسیل از منفی فاصله BFS عامل تا زیرهدف جاری به دست می‌آید:

$$\Phi(s) = -d_{BFS}(s, g_{sub})$$

تا پیش از دریافت کلید، زیرهدف جاری برابر خانه کلید است:

$$g_{sub} = Keyfork = 0$$

پس از دریافت کلید، زیرهدف به خانه هدف تغییر می‌کند:

$$g_{sub} = Goalfork = 1$$

در محاسبه فاصله تا کلید، در بسته در نظر گرفته می‌شود. پس از دریافت کلید، فاصله تا هدف با فرض قابل عبور بودن در محاسبه می‌گردد. بدین ترتیب، جمله شکل‌دهی با ترتیب واقعی مأموریت سازگار است و عامل را مستقیماً به سمت هدف نهایی هدایت نمی‌کند تا زمانی که کلید را به دست آورده باشد.

BFS در این بخش فقط برای محاسبه یک مقدار پتانسیل استفاده می‌شود و عمل عامل را تعیین نمی‌کند. الگوریتم Q-Learning همچنان سیاست خود را از تجربه‌های محیط می‌آموزد.

۷-۵. اثر حرکت بر پاداش شکل‌دهی

فرض شود فاصله فعلی عامل تا زیرهدف برابر d باشد.

اگر عامل یک خانه به زیرهدف نزدیک شود، فاصله بعدی برابر $d-1$ است و جمله شکل‌دهی به‌صورت زیر خواهد بود:

$$F_{closer} = (1 - \gamma_{sh})d + \gamma_{sh}$$

با قرار دادن $\gamma_{sh} = 0.99$:

$$F_{closer} = 0.01d + 0.99$$

این مقدار مثبت است. بنابراین، حرکت در جهت کاهش فاصله بازخوردی بهتر از هزینه پایه $1-d$ دریافت می‌کند.

اگر عامل یک خانه از زیرهدف دور شود:

$$F_{farther} = (1 - \gamma_{sh})d - \gamma_{sh}$$

و در تنظیمات پروژه:

$$F_{farther} = 0.01d - 0.99$$

این مقدار برای فاصله‌های موجود در نقشه منفی است و حرکت در جهت نامناسب را جریمه می‌کند.

برای نمونه، فاصله شروع تا کلید برابر ۱۲ است. اگر عامل در یک گام این فاصله را از ۱۲ به ۱۱ کاهش دهد:

$$F = 0.99(-11) - (-12) = 1.11$$

در نتیجه، پاداش حرکت عادی برابر خواهد بود با:

$$r_{shaped} = -1 + 1.11 = 0.11$$

اگر فاصله از ۱۲ به ۱۳ افزایش یابد:

$$F = 0.99(-13) - (-12) = -0.87$$

و پاداش نهایی حرکت برابر است با:

$$r_{shaped} = -1 - 0.87 = -1.87$$

این تفاوت، اطلاعاتی جهت‌دار در اختیار عامل قرار می‌دهد که در نسخه تنک وجود ندارد.

۸-۵. حفظ سیاست بهینه در شکل‌دهی مبتنی بر پتانسیل

یکی از دلایل استفاده از فرم مبتنی بر پتانسیل، امکان حفظ سیاست بهینه مسئله اصلی است. هنگامی که ضریب شکل‌دهی با ضریب تنزیل الگوریتم برابر باشد، مجموع تنزیل شده جملات شکل‌دهی در طول اپیزود خاصیت تلسکوپی دارد:

$$\sum_{t=0}^{T-1} \gamma^t F(s_t, s_{t+1}) = -\Phi(s_0) + \gamma^T \Phi(s_T)$$

در پیاده‌سازی، پتانسیل حالت پایانی برابر صفر قرار داده شده است:

$$\Phi(s_T) = 0$$

بنابراین:

$$\sum_{t=0}^{T-1} \gamma^t F(s_t, s_{t+1}) = -\Phi(s_0)$$

از آنجا که همه اپیزودها از حالت اولیه یکسان آغاز می‌شوند، این مقدار نسبت به مسیر انتخاب‌شده ثابت است. در نتیجه، جمله شکل‌دهی ترتیب بازده سیاست‌های کامل را تغییر نمی‌دهد و تنها نحوه توزیع بازخورد در طول مسیر را اصلاح می‌کند. این ویژگی همان شرط اصلی شکل‌دهی مبتنی بر پتانسیل برای حفظ سیاست بهینه است.

این نتیجه به برابری ضریب شکل‌دهی و ضریب تنزیل آموزش وابسته است. در پروژه هر دو مقدار برابر ۰٫۹۹ انتخاب شده‌اند. استفاده از ضریب متفاوت می‌توانست تضمین حفظ سیاست را از بین ببرد.

۵-۹. مقایسه تجربی دو تابع پاداش

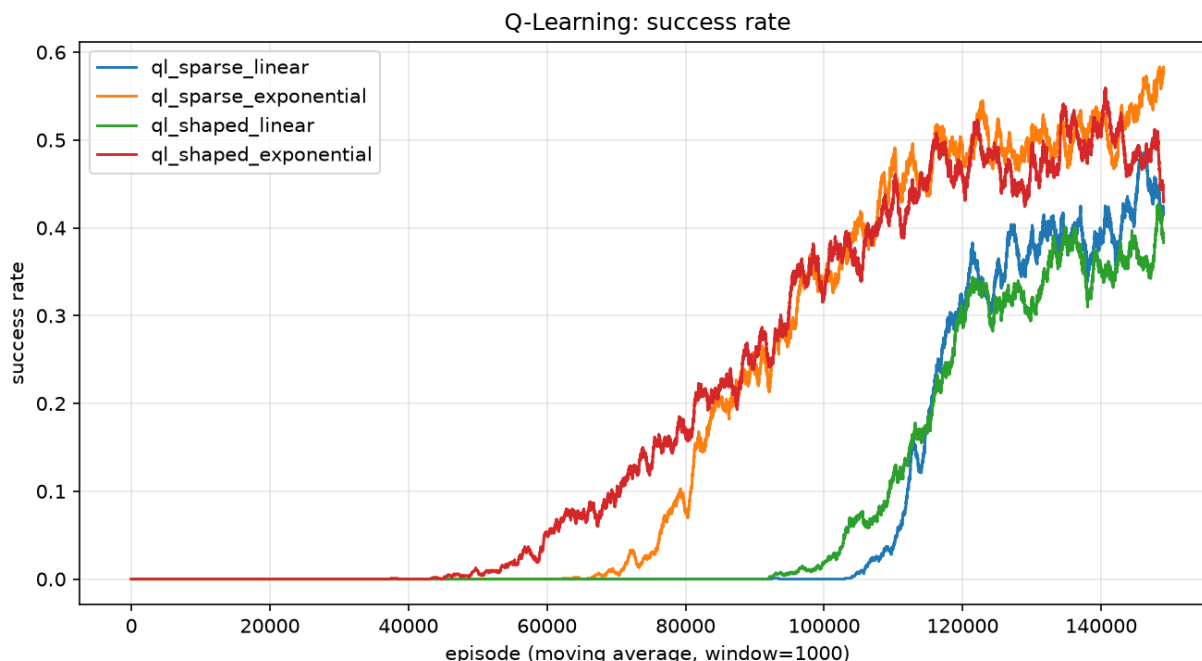
۵-۹-۱. تنظیمات مقایسه

برای مقایسه پاداش تنک و شکل‌دهی‌شده، Q-Learning در چهار پیکربندی آموزش داده شده است:

- پاداش تنک با کاهش خطی ϵ ؛
- پاداش تنک با کاهش نمایی ϵ ؛
- پاداش شکل‌دهی‌شده با کاهش خطی ϵ ؛
- پاداش شکل‌دهی‌شده با کاهش نمایی ϵ .

در تمام اجراها، نقشه، تعداد اپیزودها، نرخ یادگیری، ضریب تنزیل، seed و انتهای ϵ یکسان نگه داشته شده‌اند. تعداد اپیزودها ۱۵۰۰۰۰، نرخ یادگیری ۰٫۲ و ضریب تنزیل ۰٫۹۹ بوده است. هر سیاست نهایی نیز در ۳۰۰ اپیزود مستقل و با حذف اکتشاف ارزیابی شده است.

برای جداکردن اثر تابع پاداش از اثر برنامه کاهش ϵ ، مقایسه اصلی باید میان اجراهای دارای برنامه یکسان انجام شود؛ یعنی نسخه تنک خطی با شکل‌دهی‌شده خطی و نسخه تنک نمایی با شکل‌دهی‌شده نمایی مقایسه گردد.



شکل ۵-۲: تغییرات نرخ موفقیت Q-Learning با پاداش تنک و شکل‌دهی شده تحت برنامه‌های خطی و نمایی کاهش ϵ منحنی‌ها با میانگین متحرک ۱۰۰۰ اپیزودی رسم شده‌اند.

۵-۹-۲. سرعت آغاز یادگیری

شکل ۵-۱ نشان می‌دهد که پاداش شکل‌دهی شده، به‌ویژه در برنامه‌نمایی، موفقیت‌های اولیه را زودتر ایجاد کرده است. میانگین متحرک نرخ موفقیت اجرای shaped_exponential حدود اپیزود ۵۰ هزار از صفر فاصله می‌گیرد، درحالی‌که همین رخداد برای sparse_exponential تقریباً نزدیک اپیزود ۷۰ هزار مشاهده می‌شود.

در برنامه خطی نیز نسخه شکل‌دهی شده اندکی زودتر از نسخه تنک به موفقیت‌های قابل مشاهده می‌رسد. این نتیجه با نقش بازخورد فاصله‌ای سازگار است: عامل پیش از آنکه به‌طور تصادفی کل مأموریت را تکمیل کند، می‌تواند حرکتهای نزدیک‌کننده به کلید و هدف را از حرکتهای دورکننده متمایز سازد.

با این حال، مزیت اولیه شکل‌دهی در ادامه آموزش کاهش پیدا می‌کند. در نرخ‌های موفقیت بالاتر، منحنی‌ها به یکدیگر نزدیک می‌شوند و در بخش پایانی، اجرای تنک با کاهش نمایی بالاترین نرخ موفقیت آموزشی را به دست می‌آورد.

۵-۹-۳. ارزیابی سیاست نهایی

نتایج ارزیابی حریصانه در ۳۰۰ اپیزود به صورت زیر بوده‌اند:

- در برنامه خطی، نرخ موفقیت پاداش تنک برابر ۵۴ درصد و پاداش شکل‌دهی شده برابر ۴۰ درصد بوده است.
- در برنامه نمایی، نرخ موفقیت پاداش تنک برابر ۶۴٫۶۷ درصد و پاداش شکل‌دهی شده برابر ۵۵٫۶۷ درصد بوده است.
- میانگین طول اپیزودها در چهار اجرا بین ۴۸٫۳ و ۴۹٫۲ گام قرار داشته است.

بنابراین، شکل‌دهی باعث آغاز زودتر یادگیری شده است، اما در بودجه آموزشی فعلی موجب بهبود سیاست نهایی نشده است. بهترین نتیجه نهایی به ترکیب پاداش تنک و کاهش نمایی ϵ تعلق دارد.

این موضوع با تضمین حفظ سیاست بهینه تناقض ندارد. تضمین نظری بیان می‌کند که در صورت همگرایی کامل، سیاست بهینه دو مسئله یکسان است؛ اما تضمین نمی‌کند که هر دو روش با تعداد نمونه محدود، نرخ یادگیری ثابت و برنامه اکتشاف مشخص، با سرعت یا پایداری یکسان به آن سیاست برسند.

۵-۹-۴. محدودیت مقایسه بازده عددی

نمودار `ql_learning_curves_reward.png` نشان می‌دهد بازده ثبت‌شده در اجرای شکل‌دهی شده زودتر افزایش می‌یابد. با این حال، بازده خام معیار مناسبی برای مقایسه مستقیم دو تابع پاداش نیست؛ زیرا نسخه شکل‌دهی شده در هر گام یک مؤلفه عددی اضافی دریافت می‌کند.

به همین دلیل، ارزیابی اصلی باید براساس معیارهای مستقل از مقیاس پاداش، مانند نرخ موفقیت، تعداد گام، برخورد با دیوار و ورود به خانه‌های جریمه انجام شود. نمودار بازده می‌تواند برای بررسی روند یادگیری درون هر تابع پاداش استفاده شود، اما بزرگ‌تر بودن مقدار آن لزوماً به معنای بهتر بودن سیاست در مسئله اصلی نیست.

۵-۱۰. تحلیل رفتارهای ناخواسته

۵-۱۰-۱. حرکت‌های حلقه‌ای

یک خطر متداول در طراحی پاداش، ایجاد چرخه‌ای است که عامل بتواند با تکرار آن پاداش مثبت دریافت کند. در محیط حاضر، هر گام هزینه 1- دارد و یک واحد انرژی مصرف می‌کند؛ بنابراین، تکرار حرکت میان چند خانه به کاهش بازده و پایان انرژی منتهی می‌شود.

جمله شکل‌دهی مبتنی بر پتانسیل نیز در بازده تنزیل‌شده خاصیت تلسکوپی دارد. در نتیجه، رفت‌وبرگشت میان دو حالت نمی‌تواند به‌طور نامحدود پاداش خالص تولید کند.

۵-۱۰-۲. دریافت مکرر پاداش کلید

پاداش کلید فقط هنگامی پرداخت می‌شود که وضعیت کلید از صفر به یک تغییر کند. پس از آن، بازگشت به خانه کلید یک حرکت عادی محسوب می‌شود. بنابراین، عامل نمی‌تواند با ترک‌کردن و ورود مجدد به خانه کلید، پاداش +50 را تکرار کند.

۵-۱۰-۳. تکرار پاداش هدف

ورود به هدف بلافاصله اپیزود را پایان می‌دهد. در نتیجه، امکان باقی‌ماندن روی هدف یا ورود چندباره به آن وجود ندارد و پاداش +100 تنها یک بار پرداخت می‌شود.

۵-۱۰-۴. ثابت‌ماندن در یک خانه

اگر فاصله عامل تا زیرهدف تغییر نکند، جمله شکل‌دهی برابر است با:

$$F_{same} = (1 - \gamma_s h)d = 0.01d$$

بنابراین، این جمله می‌تواند مقدار مثبت کوچکی داشته باشد. برای نمونه، در فاصله ۱۲ مقدار آن برابر ۰.۱۲ است. بالاین‌حال، برخورد با دیوار همچنان پاداشی نزدیک به -4.88 ایجاد می‌کند:

$$r_{wallshaped} = -5 + 0.12 = -4.88$$

این مقدار به‌وضوح منفی است. علاوه بر آن، جمع تنزیل‌شده شکل‌دهی در کل اپیزود ثابت است؛ ازاین‌رو، ثابت‌ماندن یا برخورد مکرر با دیوار یک راهبرد سودآور محسوب نمی‌شود.

۵-۱۰-۵. ترجیح کوتاه‌ترین مسیر بدون توجه به خطر

فاصله BFS فقط تعداد خانه‌های باقی‌مانده تا زیرهدف را اندازه می‌گیرد و خانه‌های جریمه یا ریسک لغزش در نزدیکی دیوارها را مستقیماً در نظر نمی‌گیرد. بنابراین، شکل‌دهی ممکن است در ابتدا عامل را به سمت کوتاه‌ترین مسیر توپولوژیکی هدایت کند، حتی اگر آن مسیر از نظر تصادفی بودن حرکت یا خانه‌های جریمه پرخطرتر باشد.

جریمه‌های پایه باید این محدودیت را اصلاح کنند. عامل در طول آموزش یاد می‌گیرد که کاهش یک واحد فاصله همیشه مطلوب نیست؛ زیرا ممکن است احتمال ورود به خانه جریمه یا برخورد با دیوار را افزایش دهد. این اختلاف میان «کوتاه‌ترین مسیر BFS» و «مسیر دارای بیشترین بازده مورد انتظار» یکی از دلایل احتمالی است که شکل‌دهی، با وجود تسریع موفقیت‌های اولیه، بهترین عملکرد نهایی را ایجاد نکرده است.

۵-۱۰-۶. تمرکز بیش از حد بر کلید

پاداش مثبت کلید می‌تواند باعث شود عامل دریافت زیرهدف اول را سریع‌تر از ادامه مسیر بیاموزد. داده‌های بخش پایانی آموزش نشان می‌دهند عامل‌ها تقریباً همیشه کلید را دریافت می‌کنند، درحالی‌که نرخ موفقیت نهایی کمتر از صد درصد باقی مانده است. بنابراین، بخش دشوارتر مسئله پس از یادگیری کلید، عبور مطمئن از مسیر دوم تحت محدودیت انرژی و لغزش تصادفی است.

بزرگ‌تر بودن پاداش هدف نسبت به کلید و اعمال جریمه پایان انرژی، مانع از آن می‌شود که دریافت کلید به‌تنهایی نتیجه مطلوب نهایی تلقی شود.

۵-۱۰-۷. تفسیر پاداش انتقال پایانی

در انتقال پایانی، پتانسیل حالت بعدی صفر قرار داده می‌شود. در نتیجه، جمله شکل‌دهی همان گام ممکن است به‌صورت محلی مثبت به نظر برسد. بااین‌حال، این مقدار نباید جدا از شکل‌دهی مراحل قبلی تفسیر شود؛ زیرا مجموع تنزیل‌شده تمام جملات شکل‌دهی از حالت اولیه ثابت است. بنابراین، پایان دادن عمدی اپیزود با اتمام انرژی، مزیت خالصی نسبت به ادامه یک سیاست بهتر ایجاد نمی‌کند.

۵-۱۱. انتخاب تابع پاداش مرجع

پاداش شکل‌دهی شده برای مطالعه اثر بازخورد میانی در آزمایش Q-Learning استفاده شده است. نتایج نشان دادند این روش شروع یادگیری را سریع‌تر می‌کند، اما در ارزیابی نهایی از نسخه تنک بهتر نیست.

به همین دلیل، پاداش تنک به‌عنوان تعریف مرجع در آزمایش‌های Value Iteration، $SARSA(\lambda)$ ، پایداری و انتقال یادگیری استفاده شده است. این انتخاب چند مزیت دارد:

- هدف اصلی مسئله را بدون افزودن اطلاعات فاصله‌ای نمایش می‌دهد؛
- امکان مقایسه مستقیم الگوریتم‌ها با یک تابع پاداش مشترک را فراهم می‌کند؛
- بازده عددی الگوریتم‌های مختلف در یک مقیاس واحد قرار می‌گیرد؛

- بهترین نتیجه نهایی Q-Learning نیز در میان اجراهای بررسی شده با همین پاداش به دست آمده است.

نسخه شکل دهی شده همچنان به عنوان یک آزمایش تحلیلی مهم باقی می ماند؛ زیرا نشان می دهد بازخورد بیشتر لزوماً به عملکرد نهایی بهتر منجر نمی شود و اثر آن به اکتشاف، تعداد نمونه ها و جزئیات ساختار محیط وابسته است.

فصل ششم: الگوریتم Value Iteration و تحلیل نتایج

۱-۶. چارچوب اجرای Value Iteration

الگوریتم **Value Iteration** در این پروژه به عنوان روش مدل محور و سیاست مرجع مورد استفاده قرار گرفته است. برخلاف Q-Learning و SARSA که باید رفتار محیط را از طریق نمونه‌های تجربی بیاموزند، Value Iteration به مدل کامل انتقال دسترسی دارد و برای هر حالت و عمل، تمام پیامدهای ممکن و احتمال وقوع آن‌ها را در محاسبات خود وارد می‌کند.

با توجه به اینکه مدل MDP و تابع انتقال در فصل‌های قبل تعریف شده‌اند، در این فصل تمرکز اصلی بر نحوه اجرای الگوریتم و، مهم‌تر از آن، تحلیل نتایج حاصل از اجرای آن است.

به‌روزرسانی تابع ارزش براساس معادله بهینگی بلمن انجام می‌شود:

$$V_{(k+1)}(s) = \max_a \sum (s')P(s'|s, a)[R(s, a, s') + \gamma V_k(s')]$$

در حالت پایانی، جمله مربوط به ادامه مسیر صفر در نظر گرفته می‌شود. در هر sweep، تمام ۲۰,۸۵۰ حالت غیرپایانی بررسی شده و برای هر حالت، مقدار مورد انتظار چهار عمل ممکن محاسبه می‌شود. به‌روزرسانی‌ها به‌صورت هم‌زمان انجام می‌شوند؛ به این معنا که مقادیر sweep جدید ابتدا در یک ساختار جداگانه محاسبه و پس از پایان کامل پیمایش جایگزین تابع ارزش قبلی می‌شوند.

پس از همگرایی، سیاست نهایی با انتخاب عمل دارای بیشترین مقدار مورد انتظار استخراج می‌شود:

$$\pi^*(s) = \operatorname{argmax}_a \sum (s')P(s'|s, a)[R(s, a, s') + \gamma V^*(s')]$$

در صورت برابری چند عمل، پیاده‌سازی اولین عمل دارای بیشترین مقدار را انتخاب می‌کند.

۲-۶. تنظیمات آزمایش

آزمایش Value Iteration روی نقشه مبدأ و با تابع پاداش sparse اجرا شده است. سه مقدار متفاوت برای ضریب تنزیل در نظر گرفته شده‌اند:

$$\gamma \in \{0.90, 0.95, 0.99\}$$

آستانه همگرایی برابر است با:

$$\theta = 10^{(-6)}$$

و حداکثر تعداد sweep مجاز نیز برابر ۵۰۰ تعیین شده است. مقدار اولیه تابع ارزش برای تمام حالت‌ها صفر است.

برای هر مقدار γ ، مدل حاصل به صورت مستقل ذخیره شده و فایل‌های خام مربوط به روند همگرایی، زمان اجرا، مقدار حالت اولیه و میزان شباهت سیاست‌ها نیز در پوشه results/raw_data ثبت شده‌اند.

نتایج اصلی اجرای الگوریتم در جدول ۶-۱ خلاصه شده‌اند.

جدول ۶-۱. نتایج Value Iteration برای مقادیر مختلف ضریب تنزیل

γ	تعداد sweep تا همگرایی	زمان اجرا (ثانیه)	$V(s_0)$	شباهت سیاست با $\gamma=0.99$
0.90	51	13.930	-0.4781	93.29%
0.95	51	13.927	14.1845	95.88%
0.99	51	13.834	64.7176	100%

این جدول اثر γ را هم بر مقدار تابع ارزش و هم بر سیاست حاصل نشان می‌دهد.

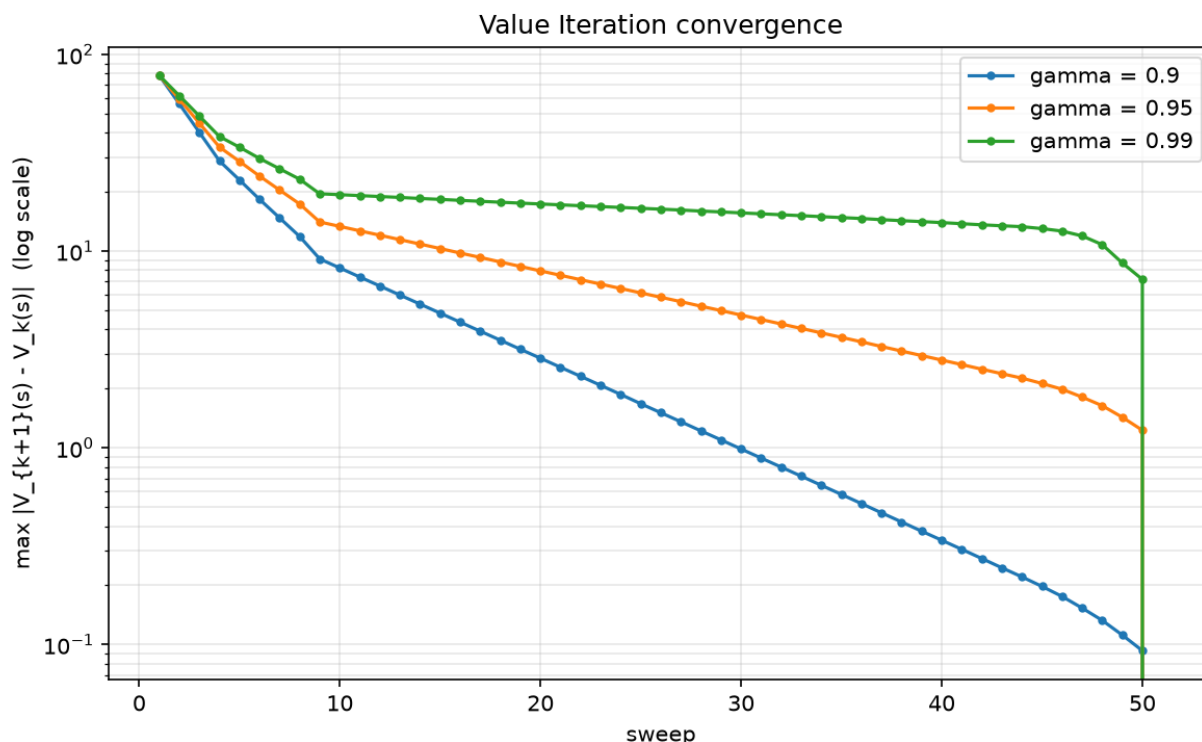
۶-۳. تحلیل همگرایی

برای بررسی روند همگرایی، در پایان هر sweep بیشترین تغییر مطلق تابع ارزش محاسبه شده است:

$$\Delta_k = \max_s |V_{(k+1)}(s) - V_k(s)|$$

معیار توقف زمانی برقرار است که:

$$\Delta_k < \theta$$



شکل ۶-۱: روند همگرایی Value Iteration برای سه مقدار ضریب تنزیل. محور عمودی بیشترین تغییر تابع ارزش در هر sweep را در مقیاس لگاریتمی نشان می‌دهد.

نمودار همگرایی نشان می‌دهد که رفتار سه مقدار γ در مراحل ابتدایی یکسان نیست. برای $\gamma=0.90$ ، اختلاف تابع ارزش با سرعت بیشتری کاهش می‌یابد، در حالی که در $\gamma=0.99$ مقدار Δ مدت بیشتری در سطح بالا باقی می‌ماند. این رفتار طبیعی است، زیرا افزایش ضریب تنزیل موجب می‌شود اثر پاداش‌های دورتر با شدت بیشتری در حالت‌های قبلی منتشر شود.

برای نمونه، مقدار بیشترین تغییر در sweep پنجاهم برابر است با:

$$\begin{aligned} \Delta_{50} &= 0.093157 & \text{for } \gamma &= 0.90 \\ \Delta_{50} &= 1.229469 & \text{for } \gamma &= 0.95 \\ \Delta_{50} &= 7.188269 & \text{for } \gamma &= 0.99 \end{aligned}$$

بنابراین، اگر تنها شکل منحنی را در نظر بگیریم، $\gamma=0.90$ به مراتب سریع‌تر به یک مقدار کوچک نزدیک شده است. با این حال، نکته قابل توجه این است که هر سه اجرا دقیقاً در sweep شماره ۵۱ همگرا شده‌اند.

این رفتار به ساختار ویژه فضای حالت مربوط است. انرژی بخشی از حالت است و هر انتقال دقیقاً یک واحد انرژی را کاهش می‌دهد:

$$e_{(t+1)} = e_t - 1$$

در نتیجه، هیچ انتقالی از یک سطح انرژی به همان سطح یا سطح بالاتر وجود ندارد. با انرژی اولیه ۵۰، وابستگی حالت‌ها در امتداد بعد انرژی یک ساختار جهت‌دار و متناهی ایجاد می‌کند. مقدار یک حالت با انرژی e تنها به حالت‌هایی با انرژی کمتر وابسته است و اطلاعات مربوط به حالت‌های پایانی در sweep های متوالی در کل فضای انرژی انتشار می‌یابد.

پس از ۵۰ مرحله انتشار، تمام سطوح انرژی مقدار نهایی خود را دریافت کرده‌اند و در sweep بعدی هیچ مقدار جدیدی تغییر نمی‌کند. به همین دلیل، در داده خام برای هر سه مقدار ضریب تنزیل داریم:

$$\Delta_{51} = 0$$

این نتیجه همچنین توضیح می‌دهد که چرا آستانه بسیار کوچک ۱۰-۶ باعث افزایش تعداد sweep ها نشده است. الگوریتم پیش از آنکه به صورت تدریجی از آستانه عبور کند، در sweep شماره ۵۱ به مقدار دقیقاً ثابت می‌رسد.

۴-۶. اثر ضریب تنزیل بر مقدار حالت اولیه

یکی از واضح‌ترین اثرهای γ در مقدار حالت اولیه مشاهده می‌شود. مقادیر ثبت شده عبارت‌اند از:

$$V_{(\gamma=0.90)}(s_0) = -0.4781$$

$$V_{(\gamma=0.95)}(s_0) = 14.1845$$

$$V_{(\gamma=0.99)}(s_0) = 64.7176$$

افزایش شدید $V(s_0)$ با افزایش ضریب تنزیل ناشی از فاصله زمانی میان شروع اپیزود و دریافت پاداش‌های اصلی است. عامل برای دریافت پاداش کلید و به‌ویژه هدف باید تعداد قابل توجهی گام طی کند. در $\gamma=0.90$ ، اثر این پاداش‌های دوردست به شدت تنزیل می‌شود و هزینه‌های منفی گام‌های ابتدایی سهم بیشتری در ارزش حالت شروع پیدا می‌کنند. به همین دلیل، مقدار $V(s_0)$ حتی اندکی منفی شده است.

در مقابل، با $\gamma=0.99$ ، پاداش‌های آینده بخش عمده ارزش خود را حفظ می‌کنند. بنابراین، عامل از حالت شروع نیز ارزش بالایی برای امکان رسیدن به کلید و هدف در آینده در نظر می‌گیرد.

منفی بودن $V(s_0)$ برای $\gamma=0.90$ به معنای ناموفق بودن الگوریتم یا نبود مسیر مناسب نیست؛ بلکه نشان می‌دهد تعریف بازده تنزیل شده در این مقدار، پاداش‌های دیر هنگام را بسیار کمتر از هزینه‌های نزدیک وزن می‌دهد.

۵-۶. اثر ضریب تنزیل بر سیاست نهایی

برای بررسی اینکه تغییر γ فقط مقدار تابع ارزش را تغییر داده یا بر تصمیم‌های عامل نیز اثر گذاشته است، سیاست‌های حاصل با سیاست $\gamma=0.99$ مقایسه شده‌اند.

از مجموع ۲۰,۸۵۰ حالت موجود، سیاست $\gamma=0.90$ در ۱۹,۴۵۰ حالت همان عمل سیاست مرجع را انتخاب می‌کند. در نتیجه:

$$Agreement_{(0.90,0.99)} = 19450/20850 = 93.29\%$$

تعداد حالت‌های دارای تصمیم متفاوت برابر ۱۴۰۰ است.

برای $\gamma=0.95$ ، تعداد حالت‌های مشابه ۱۹,۹۹۲ است:

$$Agreement_{(0.95,0.99)} = 19992/20850 = 95.88\%$$

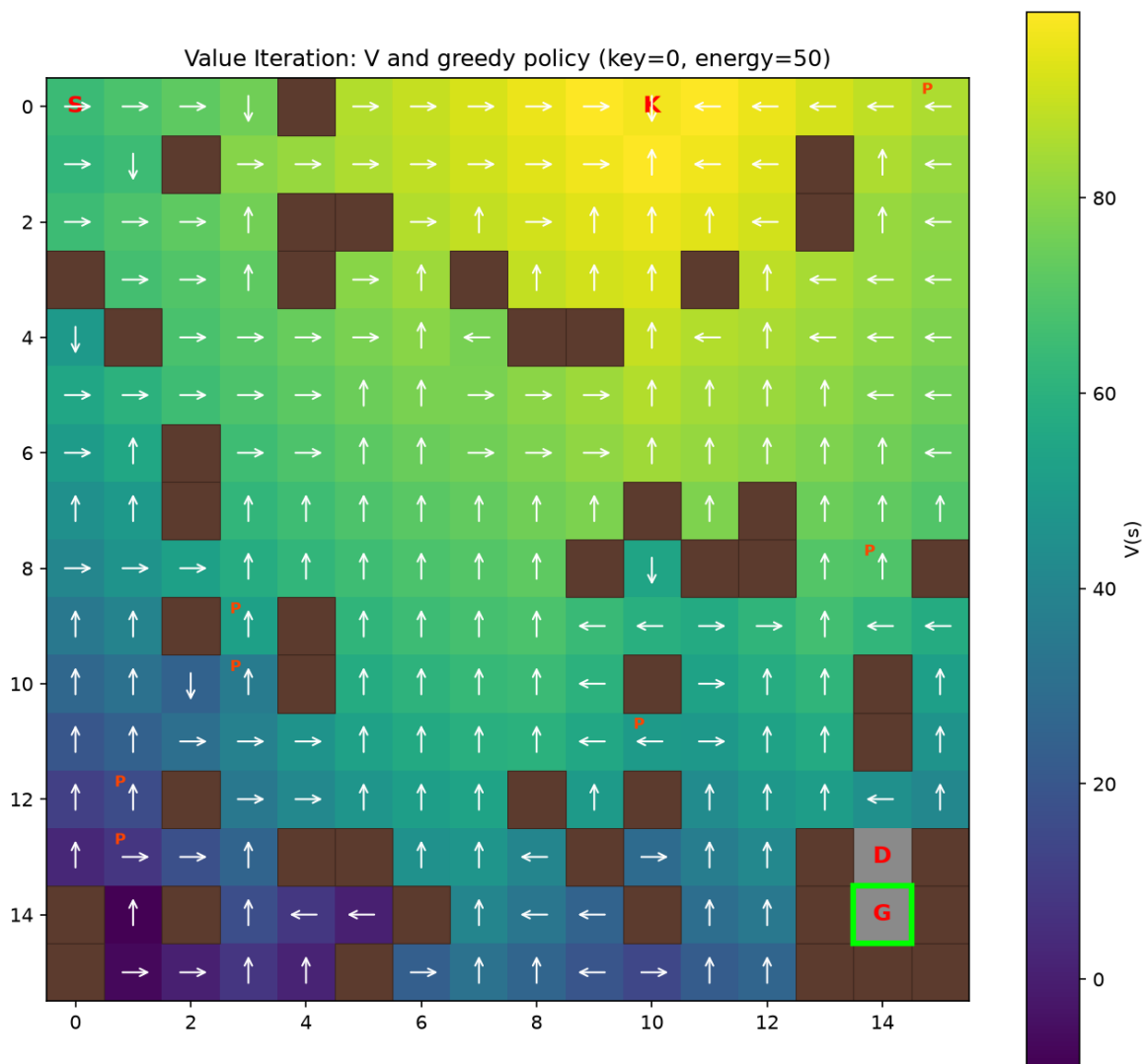
و تنها ۸۵۸ حالت تصمیم متفاوت دارند.

بنابراین، افزایش ضریب تنزیل از ۰,۹۰ به ۰,۹۵ سیاست را به سیاست مرجع ۰,۹۹ نزدیک‌تر کرده است. با این حال، حتی در $\gamma=0.90$ بیش از ۹۳ درصد تصمیم‌ها با مدل مرجع یکسان هستند. این نتیجه نشان می‌دهد ساختار کلی مسیرها نسبت به تغییر γ نسبتاً پایدار است، ولی تصمیم‌های محلی در بخشی از فضای حالت به میزان اهمیت پاداش‌های آینده حساس هستند.

اختلاف سیاست‌ها الزاماً در مسیر اصلی مورد استفاده از حالت شروع رخ نمی‌دهد. بخش زیادی از فضای ۲۰,۸۵۰ حالت به ترکیب‌های مختلف موقعیت، وضعیت کلید و انرژی است که ممکن است در یک اجرای معمول کمتر مورد بازدید قرار گیرند. تحلیل دقیق اختلاف سیاست الگوریتم‌های مدل آزاد با Value Iteration در فصل نهم ارائه خواهد شد.

۶-۶. تحلیل Heatmap پیش از دریافت کلید

برای تحلیل بصری تابع ارزش، مدل $\gamma=0.99$ به عنوان مدل مرجع انتخاب شده است. از آنجا که انرژی و وضعیت کلید بخشی از حالت هستند، نمایش کل تابع ارزش چهاربعدی در یک تصویر ممکن نیست. بنابراین، Heatmapها برای برش‌های مشخصی از فضای حالت رسم شده‌اند.



شکل ۶-۲. تابع ارزش و سیاست حریصانه Value Iteration با $\gamma=0.99$ (الف) پیش از دریافت کلید در انرژی ۵۰

این تصویر مربوط به شرایط زیر است:

$$k = 0, e = 50$$

یعنی عامل هنوز کلید را دریافت نکرده و در سطح انرژی اولیه قرار دارد.

در این Heatmap، رنگ هر خانه مقدار $V(s)$ و پیکان سفید عمل حریصانه سیاست را نشان می‌دهد. دیوارها با رنگ مجزا و عناصر اصلی نقشه با حروف S، K، D و G مشخص شده‌اند.

در حالت بدون کلید، بالاترین مقادیر تابع ارزش عمدتاً در ناحیه منتهی به کلید مشاهده می‌شوند. پیکان‌های سیاست نیز در بخش‌های مختلف نقشه به‌گونه‌ای سازمان یافته‌اند که عامل را به سمت ناحیه کلید هدایت کنند. این نتیجه با ساختار مسئله سازگار است، زیرا تا پیش از دریافت کلید، حرکت مستقیم به سمت هدف سودی ندارد و عبور از در نیز ممکن نیست.

نکته مهم آن است که پیکان‌ها الزاماً در تمام خانه‌ها کوتاه‌ترین جهت هندسی به سمت کلید را انتخاب نمی‌کنند. Value Iteration احتمال لغزش، دیوارها و خانه‌های جریمه را در امید ریاضی انتقال وارد می‌کند. بنابراین، در برخی موقعیت‌ها یک جهت با فاصله هندسی اندکی بیشتر می‌تواند به دلیل ریسک کمتر، ارزش مورد انتظار بیشتری داشته باشد.

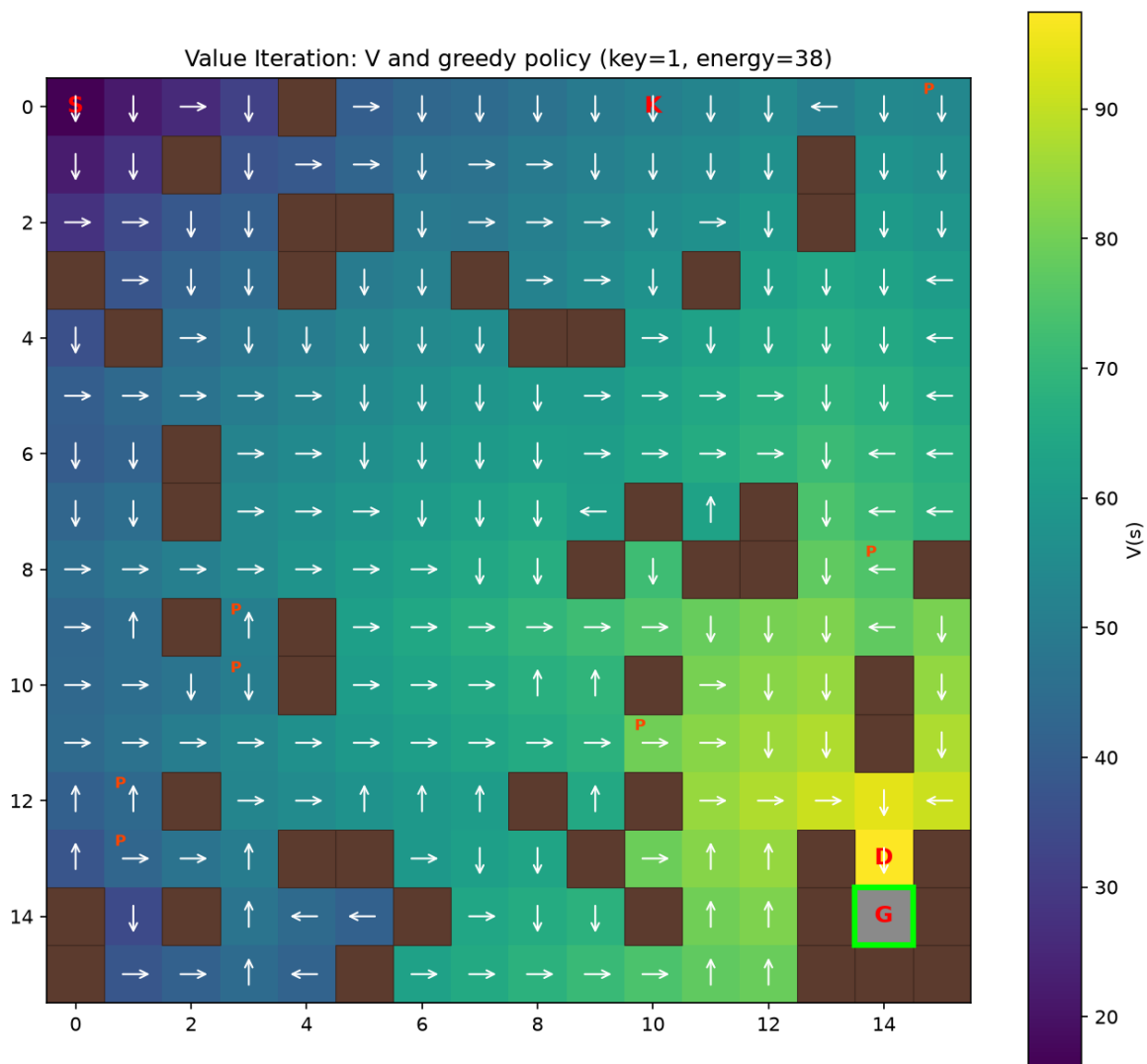
۷-۶. تغییر تابع ارزش پس از دریافت کلید

پس از دریافت کلید، هدف تصمیم‌گیری تغییر می‌کند. برای نشان دادن این وضعیت، برش دوم تابع ارزش در شرایط زیر رسم شده است:

$$k = 1, \quad e = 38$$

مقدار انرژی ۳۸ به این دلیل انتخاب شده است که کوتاه‌ترین مسیر شروع تا کلید ۱۲ گام است و در یک مسیر ایده‌آل پس از رسیدن به کلید:

$$e = 50 - 12 = 38$$



شکل ۶-۲. تابع ارزش و سیاست حریمانه Value Iteration با $\gamma=0.99$ (ب) پس از دریافت کلید در انرژی ۳۸.

در شکل دوم، توزیع ارزش‌ها به‌طور محسوسی تغییر کرده است. پس از دریافت کلید، مقادیر بالا از ناحیه کلید به سمت مسیر منتهی به در و هدف منتقل می‌شوند. پیکان‌ها نیز به‌جای همگرایی به سمت کلید، عمدتاً مسیر حرکت به بخش پایین و راست نقشه و سپس در را دنبال می‌کنند.

این تغییر یکی از نتایج مهم Heatmap است، زیرا نشان می‌دهد اضافه‌کردن متغیر کلید به فضای حالت فقط یک انتخاب نظری نبوده است. برای یک مختصات مکانی یکسان، سیاست مطلوب قبل و بعد از دریافت کلید می‌تواند متفاوت باشد. بنابراین، Value Iteration عملاً دو ساختار تصمیم‌گیری متفاوت را در دو لایه $k=0$ و $k=1$ آموخته است.

همچنین، مقادیر بسیار بالا در اطراف در و هدف در لایه $k=1$ نشان می‌دهند که پس از بازشدن دسترسی، پاداش نهایی به شکل مستقیم‌تری در ارزش حالت‌های نزدیک مقصد منعکس شده است.

۸-۶. ارزیابی سیاست مرجع

علاوه بر محاسبه تابع ارزش، سیاست مرجع Value Iteration در محیط تصادفی نیز ارزیابی شده است. فایل `vi_policy_evaluation.csv` شامل نتایج ارزیابی ۳۰۰ اپیزود با تابع پاداش تنک است.

نرخ موفقیت ثبت‌شده برابر است با:

Success Rate=93%

میانگین بازده اپیزودها:

Mean Return=88.9033

و میانگین تعداد گام در تمام اپیزودها:

Mean Steps=40.71

در اپیزودهای موفق، میانگین طول مسیر برابر ۴۰٫۰۱ گام بوده است. این مقدار از کوتاه‌ترین مسیر قطعی ۳۰ گام بیشتر است که با ماهیت تصادفی محیط سازگار است. حتی زمانی که سیاست بهینه عمل مناسب را انتخاب می‌کند، لغزش‌های ۰.۱/۰.۱ می‌توانند عامل را از مسیر موردنظر منحرف کنند و تعداد گام‌های موردنیاز را افزایش دهند.

در ۷ درصد اپیزودها، انرژی عامل پیش از رسیدن به هدف پایان یافته است:

Energy Depleted Rate=7%

میانگین طول این اپیزودهای ناموفق دقیقاً ۵۰ گام بوده است که با بودجه انرژی اولیه مطابقت دارد. هیچ اپیزودی نیز به سقف ۶۳۰ گام نرسیده است:

Step Limit Rate=0%

این نتیجه تأیید می‌کند که در تنظیمات فعلی، محدودیت واقعی طول اپیزود همان انرژی ۵۰ واحدی است و سقف `max_steps` در عمل فعال نمی‌شود.

از سوی دیگر، نرخ موفقیت ۹۳ درصد و نه ۱۰۰ درصد نکته مهمی است. Value Iteration مدل انتقال را به طور کامل می شناسد و سیاست آن از نظر بازده مورد انتظار بهینه است، اما محیط همچنان تصادفی است. یک سیاست بهینه نمی تواند نتیجه تمام لغزش های تصادفی را کنترل کند؛ در برخی اپیزودها، توالی نامطلوبی از انحراف ها و حرکات اضافی می تواند انرژی را پیش از رسیدن به هدف تمام کند.

به همین دلیل، نرخ موفقیت کمتر از ۱۰۰ درصد را نباید به عنوان عدم همگرایی Value Iteration تفسیر کرد. تابع ارزش به طور کامل همگرا شده است، اما نتیجه اجرای سیاست در یک MDP تصادفی ذاتاً قطعی نیست.

۹-۶. زمان محاسباتی

زمان اجرای Value Iteration برای سه مقدار ضریب تنزیل تقریباً یکسان بوده است:

13.930 s	for	$\gamma=0.90$
13.927 s	for	$\gamma=0.95$
13.834 s	for	$\gamma=0.99$

اختلاف کمتر از یک دهم ثانیه است و از نظر عملی معنادار نیست. این شباهت قابل انتظار است، زیرا هر سه اجرا دقیقاً ۲۰،۸۵۰ حالت، چهار عمل و ۵۱ sweep را پردازش کرده اند. تغییر ضریب تنزیل هزینه هر Bellman backup را تغییر نمی دهد.

Value Iteration برای دستیابی به این سیاست به هیچ اپیزود آموزشی نیاز ندارد، اما در عوض در هر sweep باید مدل تمام حالت ها را بررسی کند. بنابراین، هزینه آن بیشتر به اندازه فضای حالت و تعداد انتقال های ممکن وابسته است تا تعداد نمونه های تعامل با محیط. این تفاوت در فصل نهم هنگام مقایسه Sample Runtime، Memory و Efficiency میان سه الگوریتم بررسی خواهد شد.

۱۰-۶. انتخاب سیاست مرجع

بر اساس آزمایش انجام شده، مدل $\gamma=0.99$ برای ادامه پروژه به عنوان سیاست مرجع انتخاب شده است. مهم ترین دلایل این انتخاب عبارت اند از حفظ اثر پاداش های دور دست، بیشترین مقدار حالت اولیه و سازگاری با ضریب تنزیل مورد استفاده در Q-Learning و $SARSA(\lambda)$.

این مدل در فایل results/models/vi_gamma_0.99.pkl ذخیره شده است و شامل تابع ارزش، سیاست حریصانه و تاریخچه همگرایی است.

در مراحل بعدی پروژه، سیاست‌های یادگرفته‌شده توسط Q-Learning و SARSA(λ) با همین سیاست مقایسه می‌شوند. بنابراین، Value Iteration علاوه بر یک الگوریتم مستقل، نقش benchmark مدل‌محور پروژه را نیز دارد.

۱۱-۶. جمع‌بندی نتایج

نتایج Value Iteration نشان داد که الگوریتم برای هر سه مقدار ضریب تنزیل پس از ۵۱ sweep به مقدار دقیق ثابت می‌رسد. این رفتار بیش از آنکه ناشی از مقدار θ باشد، نتیجه ساختار متناهی بعد انرژی است؛ زیرا هر عمل یک واحد انرژی مصرف می‌کند و وابستگی تابع ارزش فقط به سطوح انرژی پایین‌تر ادامه می‌یابد.

افزایش γ تأثیر قابل‌توجهی بر مقدار تابع ارزش داشت. مقدار حالت اولیه از -0.4781 در $\gamma=0.90$ به 64.7176 در $\gamma=0.99$ افزایش یافت. با این حال، سیاست‌های حاصل نسبتاً پایدار باقی ماندند و حتی سیاست $\gamma=0.90$ در بیش از ۹۳ درصد حالت‌ها با سیاست مرجع یکسان بود.

Heatmapهای مدل مرجع نیز تغییر روشن سیاست پس از دریافت کلید را نشان دادند: پیش از کلید، تابع ارزش و اعمال به سمت زیرهدف کلید سازمان می‌یابند و پس از دریافت آن، تمرکز سیاست به سمت در و هدف منتقل می‌شود.

در نهایت، ارزیابی سیاست $\gamma=0.99$ در ۳۰۰ اپیزود تصادفی نرخ موفقیت ۹۳ درصد و میانگین بازده $۸۸,۹۰$ را نشان داد. این مدل از این نقطه به بعد به عنوان سیاست مرجع برای ارزیابی الگوریتم‌های مدل آزاد استفاده خواهد شد.

فصل هفتم: الگوریتم Q-Learning و تحلیل نتایج

۷-۱. چارچوب اجرای Q-Learning

در این مرحله، الگوریتم **Q-Learning** به عنوان روش مدل آزاد پروژه روی همان محیط مبدأ آموزش داده شد. برخلاف Value Iteration، عامل در طول آموزش به تابع انتقال کامل دسترسی ندارد و تنها از طریق اجرای عمل، مشاهده حالت بعدی و دریافت پاداش، مقادیر Q را به روزرسانی می کند.

از آنجا که مدل MDP، قوانین محیط و تابع پاداش در فصل های قبل تشریح شده اند، در این فصل تنها ویژگی های ضروری پیاده سازی Q-Learning بیان شده و تمرکز اصلی بر روند یادگیری و نتایج حاصل قرار می گیرد.

قاعده به روزرسانی استفاده شده به صورت زیر است:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{(t+1)} + \gamma \max_{(a')} Q(s_{(t+1)}, a') - Q(s_t, a_t)]$$

برای یک انتقال پایانی، مقدار جمله مربوط به حالت بعدی صفر در نظر گرفته می شود.

این الگوریتم **off-policy** است؛ زیرا مقدار هدف بر اساس بهترین عمل موجود در حالت بعدی محاسبه می شود، مستقل از اینکه عامل در گام بعدی بر اثر سیاست اکتشافی واقعاً چه عملی را انتخاب کند.

جدول Q در ابتدای آموزش با صفر مقداردهی شده است. از آنجا که بیشتر پاداش های عادی محیط منفی هستند، مقدار صفر در ابتدای کار نوعی مقداردهی خوش بینانه محسوب می شود و اعمالی که هنوز تجربه نشده اند موقتاً جذاب تر از اعمالی هستند که پیامد منفی آن ها مشاهده شده است.

۷-۲. تنظیمات آموزش

تمام اجزای اصلی Q-Learning شامل ۱۵۰,۰۰۰ اپیزود بوده اند. پارامترهای پایه به صورت زیر تنظیم شده اند:

$$\begin{aligned}\alpha &= 0.2 \\ \gamma &= 0.99 \\ \varepsilon_{\text{start}} &= 1.0 \\ \varepsilon_{\text{min}} &= 0.05\end{aligned}$$

کاهش ε طی ۸۰ درصد نخست آموزش انجام می شود؛ بنابراین:

$$N_{decay} = 0.8 \times 150000 = 120000$$

پس از اپیزود ۱۲۰,۰۰۰، مقدار ϵ تا پایان آموزش برابر ۰.۰۵ باقی می‌ماند.

برای بررسی اثر اکتشاف، دو برنامه کاهش ϵ پیاده‌سازی شد: خطی و نمایی. هر دو برنامه از مقدار یک آغاز می‌شوند، در اپیزود ۱۲۰,۰۰۰ به ۰.۰۵ می‌رسند و فقط شکل کاهش آن‌ها متفاوت است.

در برنامه خطی:

$$\epsilon(ep) = \epsilon_{start} - (\epsilon_{start} - \epsilon_{min}) \min(1, ep/N_{decay})$$

و در برنامه نمایی:

$$\epsilon(ep) = \max(\epsilon_{min}, \epsilon_{start} \rho^e p)$$

که در آن:

$$\rho = (\epsilon_{min}/\epsilon_{start})^{(1/N_{decay})}$$

این طراحی امکان مقایسه منصفانه دو برنامه را فراهم می‌کند؛ زیرا نقطه شروع، مقدار نهایی و زمان رسیدن به کف ϵ در هر دو یکسان است.

همان‌طور که در فصل پنجم بیان شد، چهار اجرای اصلی از ترکیب دو برنامه اکتشاف و دو تابع پاداش sparse و shaped تشکیل شده‌اند. در ادامه، تمرکز اصلی بر تفاوت برنامه‌های اکتشاف و مدل منتخب Q-Learning خواهد بود تا تحلیل تابع پاداش فصل قبل تکرار نشود.

۳-۷. تفاوت عملی برنامه‌های خطی و نمایی

اگرچه هر دو برنامه در اپیزود ۱۲۰,۰۰۰ به مقدار ۰.۰۵ می‌رسند، نرخ کاهش آن‌ها در بخش ابتدایی آموزش بسیار متفاوت است. برای نمونه، در اپیزود ۲۵,۰۰۰ داریم:

$$\epsilon_{linear} \approx 0.802$$

در حالی که:

$$\epsilon_{exponential} \approx 0.536$$

در اپیزود ۷۵,۰۰۰ اختلاف بیشتر قابل مشاهده است:

$$\begin{aligned}\epsilon_{linear} &\approx 0.406 \\ \epsilon_{exponential} &\approx 0.154\end{aligned}$$

بنابراین، برنامه‌نمایی پس از مرحله اولیه آموزش، سریع‌تر از اکتشاف گسترده به سمت بهره‌برداری از اطلاعات آموخته‌شده حرکت می‌کند.

این تفاوت مستقیماً در منحنی نرخ موفقیت مشاهده شد. همان‌طور که در شکل ۵-۱ نشان داده شد، اجرای‌نمایی بسیار زودتر از اجرای خطی شروع به دستیابی پایدار به هدف کرد.

برای جلوگیری از تکرار همان شکل، در این فصل از درج مجدد نمودار نرخ موفقیت خودداری شده و تنها تحلیل عددی داده‌های آن ارائه می‌شود.

۴-۷. روند یادگیری با پاداش تنک

برای بررسی اثر خالص برنامه اکتشاف، مناسب‌ترین مقایسه میان دو اجرای `sparse_linear` و `sparse_exponential` است؛ زیرا تابع پاداش در هر دو کاملاً یکسان است.

در اجرای خطی، نرخ موفقیت در چهار بازه نخست ۲۵,۰۰۰ اپیزودی عملاً صفر باقی ماند. تنها در بازه ۱۰۰,۰۰۰ تا ۱۲۵,۰۰۰ اپیزود، میانگین موفقیت به حدود ۱۳,۹ درصد رسید و در ۲۵,۰۰۰ اپیزود پایانی به ۳۹,۷ درصد افزایش یافت.

در مقابل، اجرای‌نمایی زودتر وارد مرحله یادگیری موفق شد. نرخ موفقیت در بازه ۵۰,۰۰۰ تا ۷۵,۰۰۰ اپیزود حدود ۰,۴ درصد بود، سپس در بازه ۷۵,۰۰۰ تا ۱۰۰,۰۰۰ به حدود ۱۹,۹ درصد، در بازه ۱۰۰,۰۰۰ تا ۱۲۵,۰۰۰ به ۴۴,۳ درصد و در ۲۵,۰۰۰ اپیزود آخر به ۵۱,۳ درصد رسید.

بررسی میانگین متحرک ۱۰۰۰ اپیزودی نیز همین تفاوت را نشان می‌دهد. اجرای‌نمایی برای نخستین بار تقریباً در اپیزود ۸۰,۲۱۲ به نرخ موفقیت ۱۰ درصد رسید، درحالی‌که اجرای خطی حدود اپیزود ۱۱۳,۰۰۸ به این سطح رسید.

برای آستانه ۳۰ درصد نیز اختلاف قابل توجه است:

$$\begin{aligned}Episode_{(30\%, exponential)} &\approx 96972 \\ Episode_{(30\%, linear)} &\approx 119177\end{aligned}$$

بنابراین، در تنظیمات فعلی، کاهش نمایی € تقریباً ۲۲ هزار اپیزود زودتر به نرخ موفقیت ۳۰ درصد رسیده است. این نتیجه نشان می‌دهد که در این مسئله، باقی‌ماندن طولانی‌مدت در سطح بالای اکتشاف الزاماً مفید نبوده است. پس از آنکه عامل مسیرهای اولیه به سمت کلید را آموخت، برنامه نمایی اجازه داد زودتر از اطلاعات موجود بهره‌برداری کند و تجربه بیشتری از بخش دوم مأموریت، یعنی مسیر کلید تا هدف، به دست آورد.

۵-۷. نتایج ارزیابی نهایی

پس از پایان آموزش، سیاست هر چهار مدل با حذف اکتشاف و در ۳۰۰ اپیزود مستقل ارزیابی شد. بنابراین، در این مرحله $\epsilon=0$ بوده و عامل فقط از سیاست حریصانه حاصل از جدول Q استفاده کرده است.

نتایج در جدول ۷-۱ ارائه شده‌اند.

پیکربندی	نرخ موفقیت	میانگین بازده	میانگین گام	تعداد حالت‌های ذخیره‌شده
Sparse + Linear	54.00%	31.97	49.0	8865
Sparse + Exponential	64.67%	43.14	48.3	8986
Shaped + Linear	40.00%	31.29	49.2	7819
Shaped + Exponential	55.67%	49.04	48.9	7734

جدول ۷-۱: ارزیابی نهایی پیکربندی‌های Q-Learning

در میان چهار اجرا، پاداش تنک همراه با کاهش نمایی € بالاترین نرخ موفقیت را به دست آورده است. در این پیکربندی، ۱۹۴ اپیزود از ۳۰۰ اپیزود ارزیابی با موفقیت به هدف رسیده‌اند:

$$194/300 = 64.67\%$$

در مقابل، پیکربندی تنک-خطی در ۱۶۲ اپیزود موفق بوده است:

$$162/300 = 54\%$$

در نتیجه، تغییر برنامه کاهش € بدون تغییر هیچ پارامتر اصلی دیگری، نرخ موفقیت ارزیابی را حدود ۱۰٫۷ واحد درصد افزایش داده است.

میانگین بازده پاداش تنک نیز از ۳۱٫۹۷ به ۴۳٫۱۴ افزایش یافته که نتیجه بهبود نرخ موفقیت و کاهش برخی حرکت‌های نامطلوب است.

مقادیر بازده نسخه shaped را نباید مستقیماً با بازده نسخه sparse مقایسه کرد؛ زیرا مقیاس دو تابع پاداش متفاوت است. برای مقایسه بین این دو حالت، نرخ موفقیت معیار معتبرتری است؛ موضوعی که در فصل پنجم نیز بررسی شد.

۶-۷. تفاوت عملکرد آموزش و ارزیابی

در ۱۰۰۰ اپیزود پایانی اجرای sparse_exponential، نرخ موفقیت آموزشی برابر حدود ۵۷٫۷ درصد بوده است، درحالی که ارزیابی نهایی سیاست حریصانه نرخ موفقیت ۶۴٫۶۷ درصد را نشان داده است.

این اختلاف قابل انتظار است. در مرحله آموزش، حتی پس از رسیدن به کف برنامه کاهش، مقدار ϵ برابر ۰٫۰۰۵ باقی می ماند. بنابراین، تقریباً پنج درصد تصمیم ها همچنان با هدف اکتشاف و به صورت تصادفی انتخاب می شوند. در محیطی با بودجه انرژی محدود، حتی تعداد کمی حرکت تصادفی می تواند باعث انحراف از مسیر و پایان انرژی شود.

در مرحله ارزیابی، این اکتشاف حذف شده است. بنابراین، سیاست آموخته شده بدون دخالت اعمال اکتشافی مورد سنجش قرار می گیرد.

البته محیط همچنان تصادفی است و لغزش حرکتی حذف نمی شود؛ به همین دلیل، حتی سیاست حریصانه نیز موفقیت صددرصدی ندارد.

۷-۷. پوشش فضای حالت

مدل منتخب Q-Learning در طول آموزش ۸۹۸۶ حالت متفاوت را تجربه و در جدول خود ذخیره کرده است. در مقابل، فضای حالت غیرپایانی بررسی شده در Value Iteration شامل ۲۰،۸۵۰ حالت بود. بنابراین، پوشش Q-Learning تقریباً برابر است با:

$$8986/20850 \approx 43.1\%$$

این تفاوت یکی از ویژگی های مهم روش مدل آزاد است. Value Iteration تمام فضای حالت تعریف شده را بدون توجه به احتمال مشاهده عملی آن بررسی می کند، اما Q-Learning فقط درباره حالت هایی اطلاعات به دست می آورد که در مسیر تعامل واقعاً مشاهده شده اند.

جالب آنکه اجرای نمایی با وجود کاهش سریع تر اکتشاف، اندکی بیش از اجرای خطی حالت ذخیره کرده است؛ ۸۹۸۶ در برابر ۸۸۶۵ حالت. یکی از توضیحات محتمل این است که یادگیری سریع تر مسیرهای مؤثر باعث شده عامل زودتر به نواحی بعد از کلید دسترسی پیدا کند و حالت‌های جدیدی از بخش دوم مأموریت را تجربه نماید. بنابراین، تعداد حالت‌های بازدید شده صرفاً تابع مقدار ϵ نیست، بلکه به میزان پیشرفت عامل در محیط نیز وابسته است.

۸-۷. تحلیل مقادیر Q و سیاست مدل منتخب

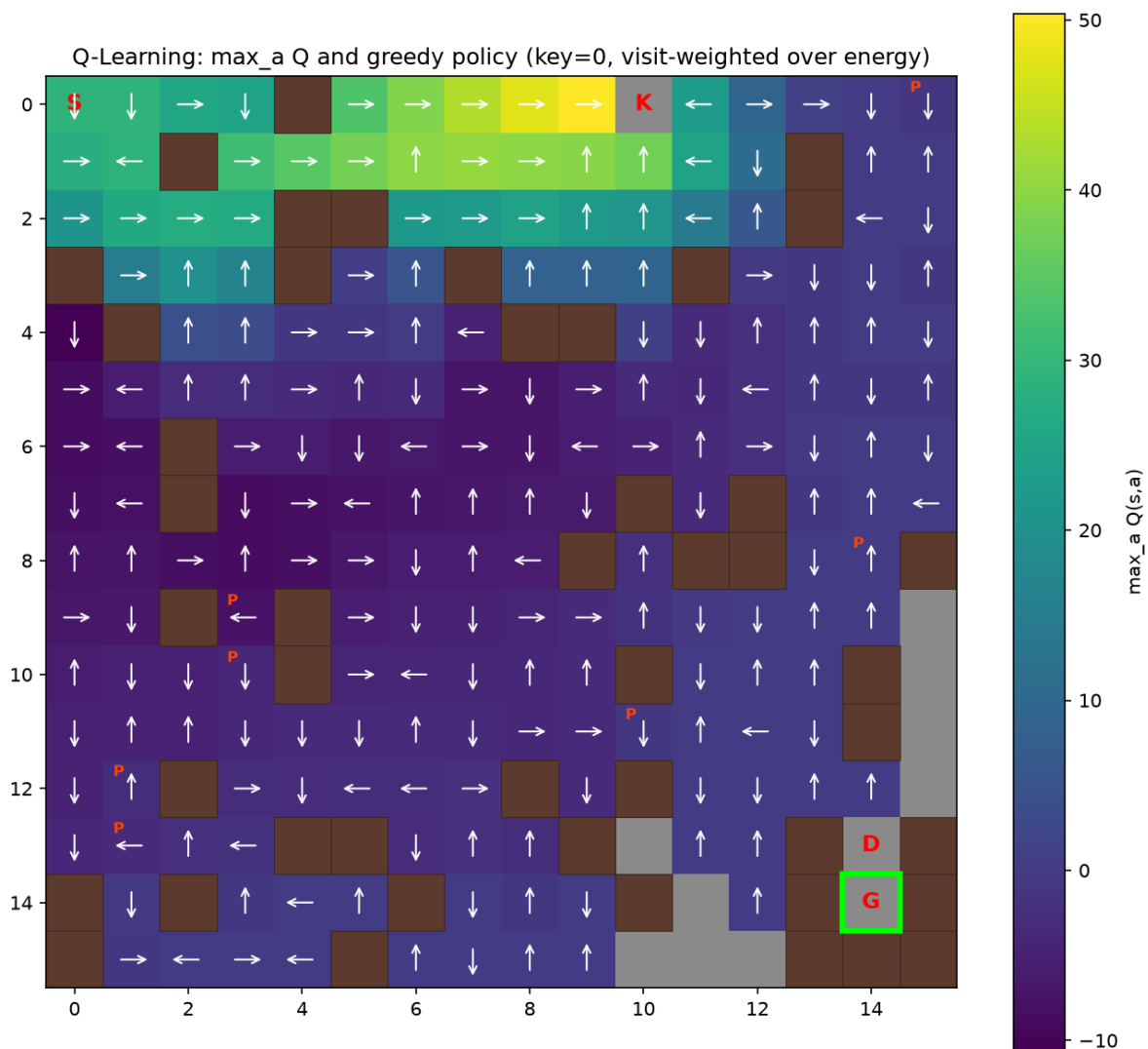
برای تحلیل فضایی مدل نهایی، از مدل `ql_sparse_exponential_q.pkl` استفاده شده است. این مدل علاوه بر داشتن بیشترین نرخ موفقیت، در مراحل بعدی پروژه نیز به عنوان مدل Q-Learning مرجع استفاده شده است.

از آنجا که انرژی بخشی از حالت است، یک Heatmap دوبعدی نمی‌تواند مقدار Q را برای تمام حالت‌های (r, c, k, e) مستقیماً نمایش دهد. برخلاف Value Iteration، انتخاب یک مقدار ثابت انرژی نیز برای Q-Learning مناسب نیست؛ زیرا عامل تنها ترکیب‌های محدودی از موقعیت و انرژی را در عمل تجربه کرده است.

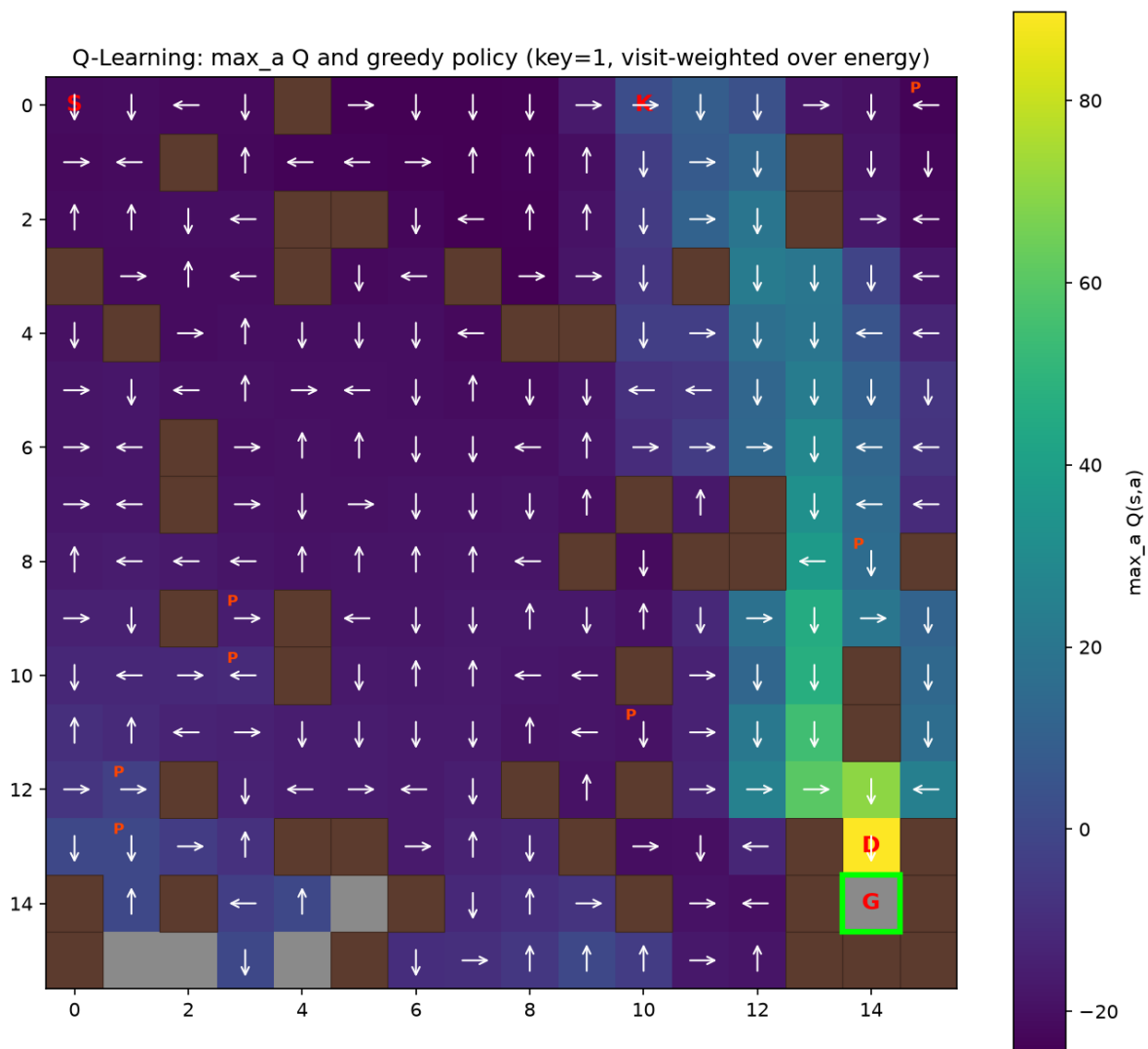
بنابراین، در Heatmap های Q-Learning مقادیر مربوط به انرژی‌های مشاهده شده با وزن تعداد بازدید تجمیع شده‌اند. برای هر خانه، مقدار رنگ از میانگین وزن دار زیر به دست می‌آید:

$$Q_{cell} = \left(\sum_e N(s_e) \max_a Q(s_e, a) \right) / \left(\sum_e N(s_e) \right)$$

و پیکان سیاست از حالت انرژی‌ای انتخاب می‌شود که در آن خانه بیشترین تعداد بازدید را داشته است.



شکل ۷-۱: مقادیر بیشینه Q و سیاست حریصانه مدل منتخب Q -Learning با پاداش تنک و کاهش نمایی ϵ (الف) پیش از دریافت کلید



شکل ۷-۱: مقادیر پیشنهادی Q و سیاست حریصانه مدل منتخب Q-Learning با پاداش تنک و کاهش نمایی ϵ . (ب) پس از دریافت کلید. مقادیر هر خانه به صورت وزن‌دهی شده بر اساس تعداد بازدید از سطوح مختلف انرژی تجمیع شده‌اند.

۷-۸-۱. وضعیت پیش از دریافت کلید

در بخش $k=0$ ، بالاترین مقادیر Q عمدتاً در ناحیه بالایی نقشه و مسیر منتهی به کلید مشاهده می‌شوند. مقدارها با نزدیک شدن به کلید افزایش می‌یابند و بسیاری از پیکان‌های سیاست در ناحیه پرتدد، عامل را به سمت این زیرهدف هدایت می‌کنند.

در مقابل، نواحی دور از مسیر اصلی دارای مقادیر پایین‌تر و در برخی خانه‌ها حتی منفی هستند. این الگو نشان می‌دهد که تجربه‌های موفق عامل در طول آموزش به‌طور مؤثر ارزش مسیر منتهی به کلید را افزایش داده‌اند.

خانه‌های خاکستری بیانگر نواحی‌ای هستند که برای ترکیب موردنظر از وضعیت کلید و انرژی داده کافی در جدول Q وجود نداشته است. این مسئله باید در تفسیر شکل در نظر گرفته شود؛ خاکستری بودن یک خانه به معنای ارزش پایین آن نیست، بلکه به معنای نبود تجربه کافی است.

۲-۸-۷. وضعیت پس از دریافت کلید

پس از تغییر k به یک، توزیع مقادیر Q به شدت تغییر می‌کند. در این وضعیت، مقادیر بالا در امتداد مسیر سمت راست نقشه و به‌ویژه در نزدیکی در و هدف متمرکز می‌شوند.

بیشترین مقادیر در ناحیه انتهایی مسیر و اطراف در مشاهده می‌شوند که نتیجه مستقیم انتشار پاداش هدف از انتقال‌های موفق است. بنابراین، جدول Q توانسته است دو وظیفه متوالی مسئله را از یکدیگر تفکیک کند: پیش از کلید، حرکت به سمت کلید و پس از آن، حرکت به سمت در و هدف.

همچنین در بخش‌هایی که کمتر مورد بازدید قرار گرفته‌اند، پیکان‌های سیاست پراکندگی بیشتری دارند. این رفتار با ماهیت نمونه‌محور Q -Learning سازگار است؛ زیرا کیفیت مقادیر آموخته‌شده در حالت‌های کم‌تجربه پایین‌تر از مسیرهای پرتکرار است.

۹-۷. بازسازی دستی یک به‌روزرسانی واقعی Q

برای امکان بررسی مستقیم عملکرد الگوریتم، جزئیات به‌روزرسانی Q در اپیزودهای منتخب در فایل CSV ذخیره شده است. در این بخش، یکی از به‌روزرسانی‌های واقعی اجرای `sparse_exponential` از آخرین اپیزود آموزش بازسازی می‌شود.

در گام صفر اپیزود ۱۴۹۰۹۹۹، حالت فعلی عبارت بوده است از:

$$s_t = (0,0,0,50)$$

عمل انتخاب‌شده:

$$a_t = DOWN$$

و پس از اجرای عمل:

$$s_{(t+1)} = (1,0,0,49)$$

پاداش این انتقال برابر است با:

$$r_{(t+1)} = -1$$

مقادیر ثبت شده پیش از به روزرسانی عبارت اند از:

$$Q_{old} = 32.597166$$
$$\max Q(s_{(t+1)}, a') = 33.923928$$

با $\gamma=0.99$ ، هدف TD برابر می شود با:

$$TD_{target} = -1 + 0.99 \times 33.923928$$
$$TD_{target} = 32.584689$$

در نتیجه، خطای TD برابر است با:

$$\delta = 32.584689 - 32.597166$$
$$\delta = -0.012477$$

با نرخ یادگیری $\alpha=0.2$:

$$Q_{new} = 32.597166 + 0.2 \times (-0.012477)$$
$$Q_{new} = 32.594670$$

این مقدار دقیقاً با مقدار ثبت شده در فایل `ql_sparse_exponential_qupdates.csv` مطابقت دارد.

کوچک بودن خطای TD در این نمونه نیز معنادار است. این انتقال از آخرین اپیزود آموزش انتخاب شده و مقدار فعلی Q تقریباً با هدف جدید سازگار شده است؛ بنابراین، به روزرسانی فقط تغییر بسیار کوچکی در مقدار جدول ایجاد می کند.

در مقابل، در انتقال نهایی همین اپیزود، عامل از خانه در به هدف رسیده و پاداش ۹۹ دریافت کرده است. چون حالت بعدی پایانی است:

$$\max Q(s_{(t+1)}, a') = 0$$

و در نتیجه مقدار هدف مستقیماً برابر ۹۹ می شود. این رفتار نشان می دهد که در حالت های پایانی bootstrap انجام نمی شود.

۷-۱۰. میزان توافق با سیاست Value Iteration

برای سنجش شباهت سیاست Q-Learning با سیاست مدل محور، تنها حالت‌هایی مقایسه شده‌اند که Q-Learning واقعاً آن‌ها را بازدید کرده است. مقایسه حالت‌های هرگز مشاهده نشده با Value Iteration از نظر تجربی معنا ندارد.

برای مدل منتخب `sparse_exponential`، توافق سیاست با Value Iteration در ۸۹۸۶ حالت مشترک برابر بوده است با:

$$Policy Agreement = 50.18\%$$

این مقدار در نگاه اول نسبتاً پایین‌تر از نرخ موفقیت مدل به نظر می‌رسد، اما نباید آن را مستقیماً معادل کیفیت سیاست دانست. همه حالت‌ها اهمیت یکسانی در اجرای واقعی ندارند. اختلاف در یک حالت بسیار کم‌بازدید با اختلاف در یکی از حالت‌های اصلی مسیر شروع تا هدف اثر یکسانی بر عملکرد ندارد.

علاوه بر این، مقادیر چند عمل در برخی حالت‌ها می‌توانند بسیار نزدیک یا برابر باشند و انتخاب متفاوت میان آن‌ها لزوماً اثر محسوسی بر بازده ایجاد نکند.

تحلیل فضایی کامل اختلاف سیاست Q-Learning و Value Iteration و بررسی نمونه‌های مشخص اختلاف در فصل نهم انجام خواهد شد.

۷-۱۱. انتخاب مدل نهایی Q-Learning

با در نظر گرفتن نتایج ارزیابی، مدل `Sparse + Exponential` به عنوان مدل اصلی Q-Learning پروژه انتخاب شد. این مدل دارای نرخ موفقیت ۶۴٫۶۷ درصد، میانگین بازده ۴۳٫۱۴ و بیشترین تعداد حالت ذخیره شده در میان چهار اجرای اصلی است.

فایل مدل نهایی `results/models/ql_sparse_exponential_q.pkl` است.

همین مدل در تحلیل‌های بعدی، آزمون پایداری و مرحله انتقال یادگیری به عنوان مدل مبدأ مورد استفاده قرار گرفته است. بنابراین، انتخاب مدل صرفاً براساس یک نمودار آموزشی انجام نشده، بلکه عملکرد آن در ارزیابی مستقل نیز مبنای تصمیم قرار گرفته است.

۷-۱۲. جمع‌بندی نتایج

نتایج Q-Learning نشان دادند که نحوه کاهش اکتشاف تأثیر قابل توجهی بر سرعت و کیفیت یادگیری دارد. با وجود یکسان بودن ابتدا، انتها و مدت زمان کاهش ϵ ، برنامه‌نمایی بسیار زودتر از برنامه خطی به نرخ‌های موفقیت قابل توجه دست یافت. در حالت پاداش تنک، رسیدن به نرخ موفقیت متحرک ۳۰ درصد در برنامه‌نمایی حدود ۲۲ هزار اپیزود زودتر رخ داد.

در ارزیابی نهایی، مدل Sparse + Exponential با موفقیت ۶۴٫۶۷ درصد بهترین پیکربندی Q-Learning بود و نسبت به Sparse + Linear بیش از ۱۰ واحد درصد بهبود نشان داد. Heatmap های این مدل نیز نشان دادند که Q-Learning ساختار دو مرحله‌ای مسئله را آموخته است؛ به گونه‌ای که پیش از دریافت کلید مقادیر بالا در مسیر کلید و پس از آن در مسیر منتهی به در و هدف متمرکز می‌شوند.

بالین حال، Q-Learning تنها حدود ۴۳ درصد از فضای حالت کامل مورد بررسی Value Iteration را تجربه کرده و میزان توافق حالت به حالت سیاست آن با مرجع حدود ۵۰ درصد است. بنابراین، با وجود دستیابی به یک سیاست کاربردی، هنوز فاصله قابل توجهی میان سیاست مدل آزاد آموخته شده و سیاست مدل محور مرجع وجود دارد. بررسی دقیق این فاصله در فصل مقایسه الگوریتم‌ها انجام خواهد شد.

فصل هشتم: الگوریتم $SARSA(\lambda)$ و تحلیل نتایج

۸-۱. چارچوب اجرای $SARSA(\lambda)$

در این مرحله، الگوریتم $SARSA(\lambda)$ به عنوان روش مدل آزاد و on-policy پروژه بررسی شد. تفاوت اصلی این روش با Q-Learning در نحوه تشکیل هدف TD است. در Q-Learning، بهترین عمل ممکن در حالت بعدی مبنای بهروزرسانی قرار می‌گیرد؛ در حالی که در $SARSA$ ، همان عملی استفاده می‌شود که سیاست رفتاری عامل واقعاً برای گام بعد انتخاب کرده است. بنابراین، آثار اکتشاف نیز مستقیماً وارد فرآیند یادگیری می‌شوند.

خطای TD در پیاده‌سازی به صورت زیر محاسبه شده است:

$$\delta_t = r_{(t+1)} + \gamma Q(s_{(t+1)}, a_{(t+1)}) - Q(s_t, a_t)$$

و مقادیر Q با استفاده از رد صلاحیت یا Eligibility Trace بهروزرسانی می‌شوند:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \delta_t E(s, a)$$

پس از هر بهروزرسانی، ردهای فعال با ضریب زیر کاهش می‌یابند:

$$E(s, a) \leftarrow \gamma \lambda E(s, a)$$

در حالت پایانی، مقدار Q مربوط به حالت بعدی صفر در نظر گرفته می‌شود.

برای جلوگیری از تکرار مباحث فصل‌های پیشین، جزئیات محیط، تابع پاداش و برنامه کاهش اکتشاف مجدداً تشریح نمی‌شوند. تمام آزمایش‌های اصلی $SARSA$ با همان پاداش تنک، نرخ یادگیری 0.2، ضریب تنزیل 0.99، برنامه نمایی کاهش ϵ و ۱۵۰,۰۰۰ اپیزود اجرا شده‌اند. انتخاب برنامه نمایی نیز بر اساس نتیجه مرحله Q-Learning انجام شده است تا اثر پارامتر λ با تغییر هم‌زمان سایر عوامل مخدوش نشود.

۸-۲. Eligibility Trace و نقش پارامتر λ

هدف Eligibility Trace آن است که خطای TD یک گام فقط بر جفت حالت-عمل فعلی اثر نگذارد، بلکه بخشی از آن به تصمیم‌های اخیر همان اپیزود نیز منتقل شود. مقدار λ شدت و طول این انتقال اعتبار را کنترل می‌کند.

در پروژه، مقادیر اصلی مورد بررسی عبارت‌اند از:

$$\lambda \in \{0, 0.3, 0.7, 0.9\}$$

هنگامی که:

$$\lambda = 0$$

ردهای قبلی بلافاصله حذف می‌شوند و الگوریتم دقیقاً به **SARSA** یک‌مرحله‌ای تبدیل می‌شود. با افزایش λ ، تصمیم‌های قدیمی‌تر مدت بیشتری در به‌روزرسانی‌های بعدی مشارکت می‌کنند.

نوع اصلی رد در آزمایش‌ها replacing trace بوده است. در این حالت، هنگام مشاهده یک جفت حالت-عمل، مقدار رد آن برابر یک قرار داده می‌شود:

$$E(s_t, a_t) \leftarrow 1$$

ردهایی که مقدار آن‌ها کمتر از آستانه زیر شود از ساختار فعال حذف می‌شوند:

$$E(s, a) < 10^{-4}$$

از این‌رو، طول تقریبی ماندگاری یک رد را می‌توان به‌صورت زیر محاسبه کرد:

$$L \approx \ln(10^{-4}) / \ln(\gamma\lambda)$$

افزایش λ مستقیماً باعث افزایش تعداد ردهای هم‌زمان فعال و در نتیجه افزایش گستره انتقال اعتبار به گذشته می‌شود.

۳-۸. نتایج اصلی برای مقادیر مختلف λ

پس از آموزش، هر مدل در ۳۰۰ اپیزود مستقل و با سیاست حریصانه ارزیابی شد. نتایج چهار مقدار اصلی λ در جدول ۸-۱ ارائه شده‌اند.

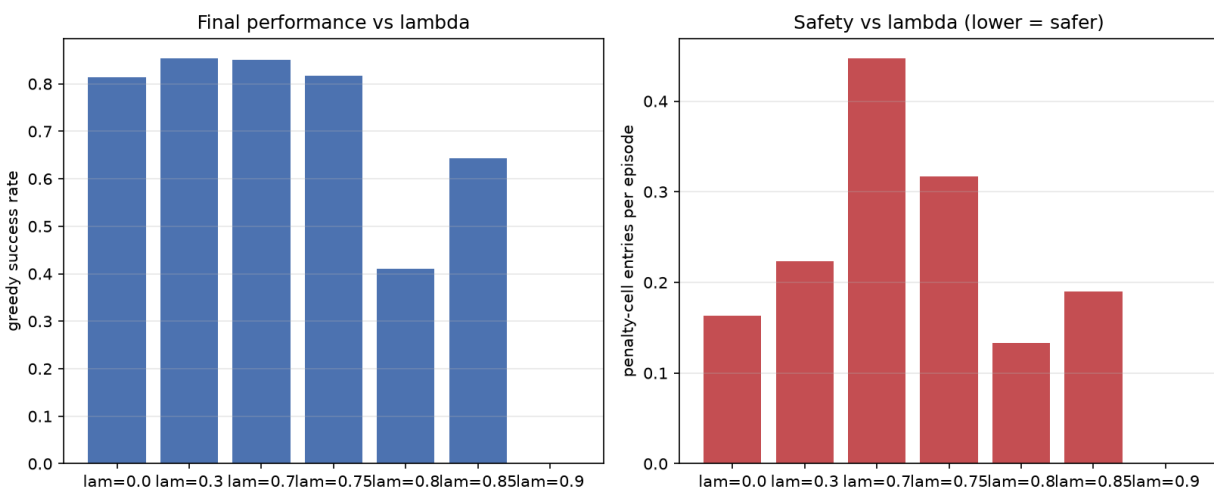
λ	طول تقریبی رد (گام)	حداکثر رد فعال	زمان آموزش (ثانیه)	نرخ موفقیت ارزیابی	میانگین بازده	میانگین گام اپیزودهای موفق
0.0	0	1	23.51	81.33%	64.98	46.4
0.3	7.6	8	31.81	85.33%	69.75	45.5

45.7	65.01	85.00%	47.38	26	25.1	0.7
—	-26.57	0.00%	55.28	50	79.8	0.9

جدول ۸-۱: نتایج نهایی λ SARSA با replacing trace

مهم‌ترین نتیجه این جدول آن است که افزایش λ به‌طور یکنواخت عملکرد را بهبود نمی‌دهد. افزودن یک رد نسبتاً کوتاه در $\lambda=0.3$ نرخ موفقیت را از ۸۱٫۳۳ درصد در SARSA یک‌مرحله‌ای به ۸۵٫۳۳ درصد افزایش داده است. مقدار $\lambda=0.7$ نیز عملکرد بسیار نزدیکی، برابر ۸۵ درصد، ایجاد کرده است؛ اما افزایش بیشتر λ به ۰٫۹ باعث شکست کامل سیاست نهایی شده است.

بنابراین، در این محیط یک مقدار میانی λ مؤثرتر از هر دو حالت بدون رد و رد بسیار بلند بوده است.



شکل ۸-۱: مقایسه عملکرد نهایی SARSA برای مقادیر مختلف λ نمودار سمت چپ نرخ موفقیت سیاست حریم‌ناهی و نمودار سمت راست میانگین ورود به خانه‌های جریمه را نشان می‌دهد.

در این شکل علاوه بر چهار مقدار اصلی، چند مقدار تکمیلی نیز برای بررسی رفتار ناحیه بین ۰٫۷ و ۰٫۹ نمایش داده شده‌اند.

۴-۸. سرعت یادگیری برای مقادیر مختلف λ

مقایسه منحنی‌های آموزشی نشان می‌دهد Eligibility Trace فقط نتیجه نهایی را تغییر نداده، بلکه زمان آغاز یادگیری مؤثر را نیز تحت تأثیر قرار داده است.

برای $\lambda=0$ ، میانگین متحرک نرخ موفقیت برای نخستین بار حدود اپیزود ۶۸۰۸۰۰ از ۱۰ درصد عبور کرد و رسیدن به سطح ۳۰ درصد تقریباً در اپیزود ۷۷۰۷۰۰ رخ داد.

در $\lambda=0.3$ ، همین دو سطح تقریباً در اپیزودهای ۶۱،۱۰۰ و ۶۹،۰۰۰ به دست آمدند. مقدار $\lambda=0.7$ حتی در بخش ابتدایی سریع‌تر بود و حدود اپیزود ۵۸،۷۰۰ به نرخ ۱۰ درصد و اپیزود ۶۵،۵۰۰ به نرخ ۳۰ درصد رسید.

بنابراین، افزایش λ از صفر تا محدوده متوسط باعث شده پاداش‌های مشاهده‌شده سریع‌تر به تصمیم‌های قبلی منتقل شوند و عامل زودتر از SARSA یک مرحله‌ای مسیر قابل استفاده‌ای را کشف کند.

بالین‌حال، در مراحل بعدی تفاوت میان $\lambda=0.3$ و $\lambda=0.7$ تغییر می‌کند. مدل $\lambda=0.3$ تقریباً در اپیزود ۹۸،۵۰۰ به نرخ موفقیت متحرک ۷۰ درصد رسیده است، درحالی‌که مدل $\lambda=0.7$ حدود اپیزود ۱۰۳،۳۰۰ به همین سطح رسیده است. در ارزیابی نهایی نیز $\lambda=0.3$ اندکی بهتر عمل کرده است.

در مقابل، $\lambda=0.9$ در هیچ مرحله‌ای به نرخ موفقیت متحرک ۱۰ درصد نرسیده است. بنابراین، انتقال اعتبار طولانی‌تر از یک حد مشخص به جای تسریع یادگیری موجب ناپایداری آن شده است.

۵-۸. تحلیل ایمنی و کیفیت مسیر

نرخ موفقیت تنها معیار مورد بررسی نبوده است. تعداد ورود به خانه‌های جریمه نیز در ارزیابی ثبت شده است. میانگین تعداد ورود به خانه جریمه در هر اپیزود برای مقادیر اصلی به‌ترتیب برابر بوده است با:

$$\lambda = 0.0 : 0.163$$

$$\lambda = 0.3 : 0.223$$

$$\lambda = 0.7 : 0.447$$

$$\lambda = 0.9 : 0.000$$

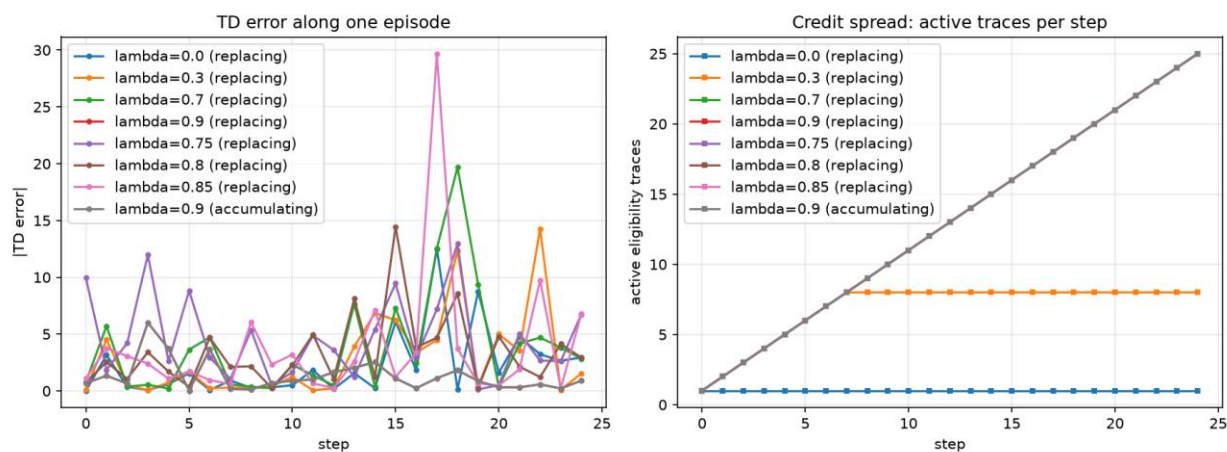
مقدار صفر در $\lambda=0.9$ را نباید به‌عنوان ایمن‌ترین سیاست تفسیر کرد. این مدل هیچ اپیزود ارزیابی موفق‌ی نداشته و اصولاً وارد بخش‌های مؤثر مسیر نشده است. بنابراین، کاهش تعداد برخورد با خانه جریمه در این حالت ناشی از عملکرد ضعیف عامل است، نه یادگیری یک مسیر ایمن.

بین مدل‌های موفق، SARSA یک مرحله‌ای کمترین ورود به خانه‌های جریمه را داشته، اما نرخ موفقیت آن پایین‌تر از دو مقدار میانی λ است. $\lambda=0.3$ با افزایش اندک مواجهه با خانه‌های جریمه، بهترین نرخ موفقیت و بالاترین بازده متوسط را به دست آورده است.

در مقابل، $\lambda=0.7$ تقریباً دو برابر $\lambda=0.3$ وارد خانه‌های جریمه شده، درحالی‌که بهبود بیشتری در نرخ موفقیت ایجاد نکرده است. از این منظر نیز مقدار ۰٫۳ تعادل مناسب‌تری میان انتقال اعتبار و کیفیت رفتار ایجاد کرده است.

۸-۶. پویایی Eligibility Trace

برای مشاهده مستقیم اثر λ ، مقدار خطای TD و تعداد ردهای فعال طی چند گام متوالی یک اپیزود ثبت شده است.



شکل ۸-۲: رفتار خطای TD و Eligibility Trace در $SARSA(\lambda)$ نمودار سمت چپ قدرمطلق خطای TD و نمودار سمت راست تعداد ردهای فعال را طی گام‌های متوالی یک اپیزود نشان می‌دهد.

سمت راست شکل تفاوت مقادیر λ را به‌وضوح نشان می‌دهد. در $\lambda=0$ ، در هر لحظه فقط یک رد فعال باقی می‌ماند. بنابراین، خطای TD تنها جفت حالت-عمل جاری را اصلاح می‌کند.

برای $\lambda=0.3$ ، ضریب کاهش رد برابر است با:

$$\gamma\lambda = 0.99 \times 0.3 = 0.297$$

ردها نسبتاً سریع کاهش پیدا می‌کنند و تعداد آن‌ها پس از چند گام در حدود هشت مقدار فعال ثابت می‌شود.

برای $\lambda=0.7$:

$$\gamma\lambda = 0.693$$

ردها بسیار آهسته‌تر کاهش می‌یابند و حداکثر ۲۶ جفت حالت-عمل به‌طور هم‌زمان تحت تأثیر یک خطای TD قرار گرفته‌اند.

در $\lambda=0.9$:

$$\gamma\lambda = 0.891$$

طول نظری رد حدود ۷۹٫۸ گام است، درحالی‌که کل بودجه انرژی اپیزود تنها ۵۰ گام است. در نتیجه، رد یک تصمیم می‌تواند عملاً تا پایان کل اپیزود فعال باقی بماند و حداکثر تعداد ردهای فعال به ۵۰ رسیده است.

این رفتار هم از نظر یادگیری و هم از نظر هزینه محاسباتی اهمیت دارد. زمان آموزش از ۲۳٫۵۱ ثانیه در $\lambda=0$ به ۵۵٫۲۸ ثانیه در $\lambda=0.9$ افزایش یافته است؛ زیرا در هر گام باید تعداد بیشتری مقدار Q به‌روزرسانی شود.

۷-۸. بررسی عددی چند گام متوالی

برای بررسی دقیق‌تر رفتار ردها، بخشی از اپیزود پایانی مدل منتخب $\lambda=0.3$ در جدول ۲-۸ ارائه شده است.

گام	خطای TD، δ	تعداد ردهای فعال	مجموع ردها پس از کاهش
0	0.109141	1	0.2970
1	4.540409	2	0.3852
2	0.509991	3	0.4114
3	0.078892	4	0.4192
4	0.735425	5	0.4215
7	0.298228	8	0.4224
8	0.244729	8	0.4224

جدول ۲-۸: تغییر خطای TD و تعداد ردهای فعال در چند گام متوالی برای $\lambda=0.3$

در گام صفر، مقدار Q پیش از به‌روزرسانی برابر بوده است با:

$$Q(s_t, a_t) = 36.242116$$

مقدار عمل انتخاب‌شده در حالت بعدی:

$$Q(s_{(t+1)}, a_{(t+1)}) = 37.728543$$

و پاداش انتقال برابر 1- بوده است. بنابراین:

$$\delta = -1 + 0.99 \times 37.728543 - 36.242116$$

که حاصل آن برابر است با:

$$\delta = 0.109141$$

از آنجا که رد جفت حالت-عمل فعلی ابتدا برابر یک قرار می‌گیرد:

$$E(s_t, a_t) = 1$$

مقدار Q جدید برابر می‌شود با:

$$Q_{new} = 36.242116 + 0.2 \times 0.109141$$

و در نتیجه:

$$Q_{new} = 36.263945$$

پس از انجام این به‌روزرسانی، رد با ضریب ۰٫۲۹۷ کاهش یافته و برای گام بعد باقی می‌ماند.

در گام بعد، یک رد جدید برای جفت حالت-عمل فعلی ایجاد می‌شود، درحالی‌که رد تصمیم قبلی هنوز فعال است. به همین ترتیب، خطای TD جدید نه تنها عمل جاری، بلکه تعدادی از تصمیم‌های گذشته را نیز اصلاح می‌کند. پس از حدود هشت گام، ردهای قدیمی‌تر به زیر آستانه ۴-۱۰ رسیده و حذف می‌شوند؛ در نتیجه تعداد ردهای فعال تقریباً ثابت باقی می‌ماند.

این رفتار توضیح می‌دهد چرا $\lambda=0.3$ می‌تواند انتقال اعتبار را نسبت به SARSA یک مرحله‌ای سریع‌تر کند، بدون آنکه یک خطای محلی به تقریباً کل اپیزود انتشار یابد.

۸-۸. علت افت شدید عملکرد در $\lambda=0.9$

نتیجه $\lambda=0.9$ یکی از مهم‌ترین مشاهدات این مرحله است. این مدل در ارزیابی نهایی نرخ موفقیت صفر داشته و میانگین بازده آن به ۲۶.۵۷- کاهش یافته است.

داده‌های آموزشی نشان می‌دهند این شکست به معنای یادنگرفتن کامل مسئله نیست. در ۱۰۰۰ اپیزود پایانی، عامل در حدود ۹۹٫۴ درصد اپیزودها کلید را دریافت کرده است، اما هیچ‌یک از آن‌ها را با رسیدن به هدف تکمیل نکرده است. بنابراین، زیرهدف نخست آموخته شده، اما انتقال موفق از کلید به هدف پایدار نشده است.

در $\lambda=0.9$ ، عمر رد حدود ۸۰ گام است و از حداکثر طول عملی یک اپیزود بیشتر می‌شود. به همین دلیل، یک خطای TD می‌تواند تصمیم‌هایی را که ده‌ها گام قبل گرفته شده‌اند اصلاح کند. در بخش قابل توجهی از آموزش نیز سیاست هنوز ϵ -greedy است و در مسیر عامل اعمال اکتشافی، لغزش و برخوردهای تصادفی وجود دارند.

بنابراین، یک پیامد محلی مثبت یا منفی ممکن است به تعداد زیادی تصمیم گذشته نسبت داده شود که همه آنها واقعاً مسئول همان پیامد نیستند. در چنین شرایطی مسئله **credit assignment** بیش از حد گسترده شده و مقدارهای Q ناپایدارتر می‌شوند.

برای بررسی دقیق‌تر این ناحیه، سه مقدار تکمیلی نیز آزمایش شده‌اند. نرخ موفقیت ارزیابی برای آنها عبارت بوده است از ۸۱٫۶۷ درصد برای $\lambda=0.75$ ، ۴۱ درصد برای $\lambda=0.8$ و ۶۴٫۳۳ درصد برای $\lambda=0.85$. این نتایج نشان می‌دهند افت عملکرد در ناحیه λ بزرگ کاملاً یکنواخت نیست؛ بلکه یادگیری در این محدوده به‌شدت حساس و ناپایدار می‌شود. بنابراین، مناسب‌تر است نتیجه گرفته شود که λ بسیار بزرگ در این مسئله قابل اتکا نیست، نه اینکه یک آستانه عددی قطعی برای شکست وجود داشته باشد.

۹-۸. Accumulating Trace و Replacing

علاوه بر **replacing trace**، برای $\lambda=0.9$ یک اجرای جداگانه با **accumulating trace** نیز انجام شده است. در حالت **accumulating**، هنگام مشاهده مجدد یک جفت حالت-عمل مقدار رد افزایش می‌یابد:

$$E(s_t, a_t) \leftarrow E(s_t, a_t) + 1$$

در مقابل، در **replacing trace** مقدار آن صرفاً برابر یک قرار می‌گیرد.

در محیط فعلی، نتیجه این دو اجرا از نظر مقادیر Q ، دنباله اپیزودها، بازدید حالت‌ها و نرخ موفقیت کاملاً یکسان بوده است. تنها زمان اجرای اندازه‌گیری شده اندکی متفاوت است: ۵۵٫۲۸ ثانیه برای **replacing** و ۵۷٫۵۹ ثانیه برای **accumulating**.

این برابری یک دلیل مشخص دارد. انرژی بخشی از حالت است:

$$s = (r, c, k, e)$$

و در هر گام مقدار انرژی یک واحد کاهش می‌یابد. بنابراین، حتی اگر عامل چند بار در یک مختصات مکانی یکسان قرار گیرد، مقدار e متفاوت است و از دید جدول Q یک حالت متفاوت محسوب می‌شود. به همین دلیل، یک جفت دقیق (s, a) در طول همان اپیزود عملاً دوباره مشاهده نمی‌شود و عمل $E=1$ و $E=E+1$ نتیجه یکسانی تولید می‌کنند.

از این رو، replacing trace به عنوان حالت پیش فرض حفظ شده است، اما باید توجه داشت که در تعریف حالت فعلی پروژه، تفاوت عملی replacing و accumulating قابل مشاهده نیست. اگر انرژی در حالت وجود نداشت یا ردها فقط بر اساس موقعیت تعریف می شدند، این دو روش می توانستند رفتار متفاوتی داشته باشند.

این نکته برای تفسیر صحیح نتایج اهمیت دارد و مانع نسبت دادن اختلافی به نوع trace می شود که در داده های واقعی پروژه وجود ندارد.

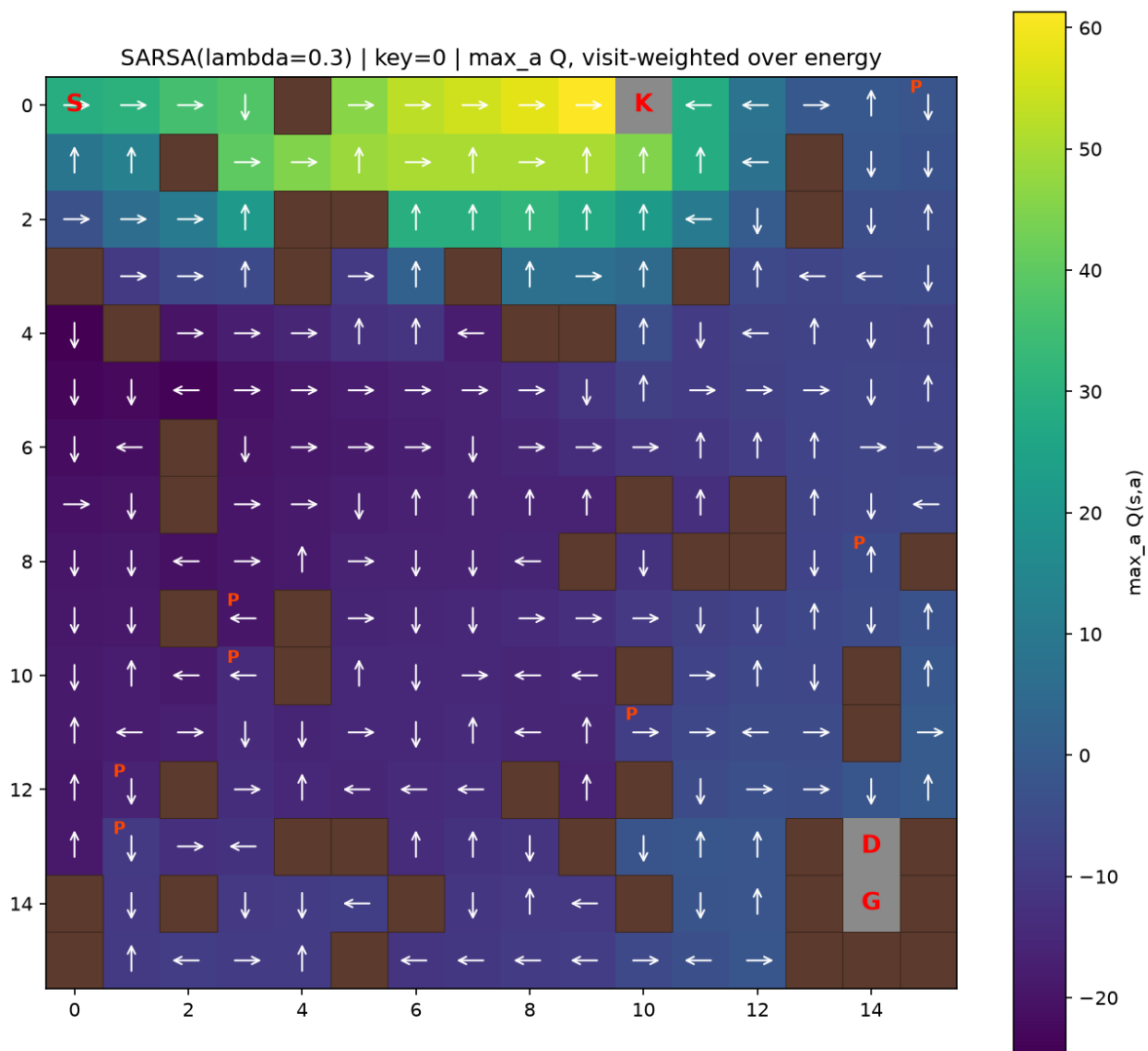
۸-۱۰. سیاست حاصل از مدل منتخب

بر اساس نرخ موفقیت ارزیابی، مدل زیر به عنوان مدل اصلی SARSA انتخاب شده است:

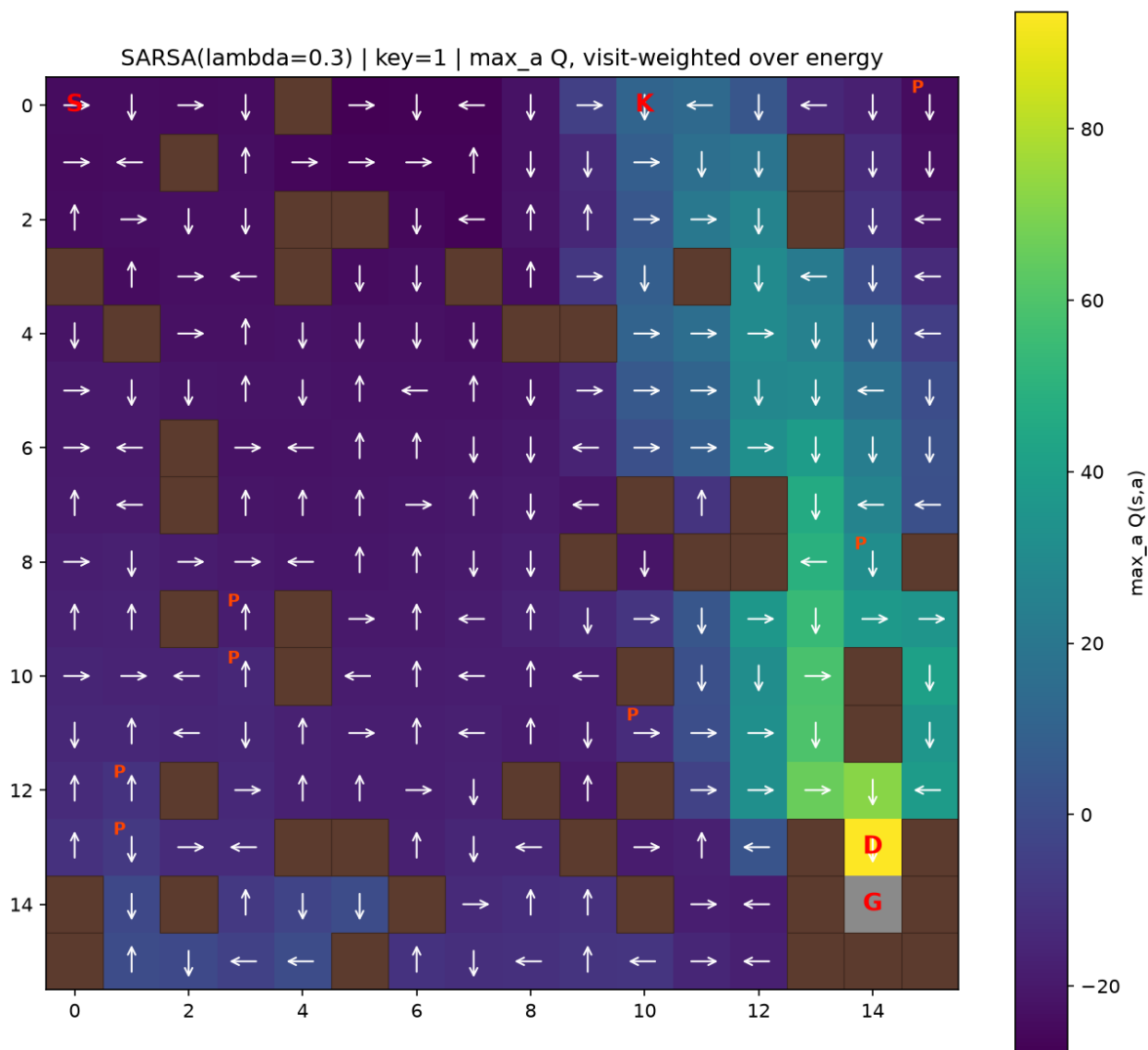
```
results/models/sarsa_lam0.3_replacing_q.pkl
```

این مدل نرخ موفقیت ۸۵٫۳۳ درصد و میانگین بازده ۶۹٫۷۵ را به دست آورده است.

برای آن Heatmap های جداگانه قبل و بعد از دریافت کلید نیز در مخزن تولید شده اند:



شکل ۸-۳: مقادیر بیشینه Q و سیاست حریصانه مدل منتخب $SARSA$: (الف) پیش از دریافت کلید



شکل ۸-۳: مقادیر بیشینه Q و سیاست حریصانه مدل منتخب SARSA: (ب) پس از دریافت کلید

این تصاویر همانند Heatmap های فصل Q-Learning نشان می دهند که پیش از دریافت کلید، مقادیر Q بالا در مسیر منتهی به کلید متمرکز هستند و پس از دریافت آن، منطقه دارای ارزش بالا به سمت در و هدف منتقل می شود.

میزان توافق سیاست مدل منتخب با Value Iteration در حالت های مشترک بازدید شده برابر ۴۸،۳۹ درصد بوده است. مقدار $\lambda=0.7$ با ۴۸،۸۵ درصد اندکی توافق بیشتری با Value Iteration دارد، با این حال نرخ موفقیت و بازده $\lambda=0.3$ بهتر بوده اند. بنابراین، مشابه Q-Learning، درصد توافق خام سیاست به تنهایی معیار کافی برای انتخاب بهترین مدل نیست. تحلیل دقیق اختلاف سیاست ها در فصل بعد انجام می شود.

۸-۱۱. جمع‌بندی نتایج

آزمایش‌های $SARSA(\lambda)$ نشان دادند که Eligibility Trace در مقادیر متوسط می‌تواند یادگیری را نسبت به $SARSA$ یک مرحله‌ای بهبود دهد. مدل $\lambda=0.3$ با نرخ موفقیت ۸۵٫۳۳ درصد بهترین نتیجه را به دست آورد و نسبت به $\lambda=0$ حدود چهار واحد درصد بهبود ایجاد کرد. $\lambda=0.7$ نیز عملکرد نزدیک به آن داشت و در مراحل ابتدایی یادگیری اندکی سریع‌تر پیشرفت کرد، اما هزینه محاسباتی و تعداد ردهای فعال آن بیشتر بود.

افزایش λ به ۰٫۹ اثر معکوس داشت. طول رد در این حالت از بودجه انرژی یک اپیزود بیشتر شد و خطاهای TD به بخش بزرگی از مسیر گذشته منتقل شدند. مدل حاصل، با وجود یادگیری تقریباً کامل زیرهدف کلید، نتوانست مسیر نهایی را به صورت پایدار تکمیل کند و نرخ موفقیت ارزیابی آن به صفر رسید.

تحلیل مستقیم ردها نیز نشان داد که در $\lambda=0$ فقط یک جفت حالت-عمل به روزرسانی می‌شود، در $\lambda=0.3$ حدود هشت رد فعال نگه داشته می‌شود و در $\lambda=0.9$ این مقدار می‌تواند به تمام ۵۰ گام اپیزود برسد. این افزایش علاوه بر اثر بر یادگیری، زمان محاسباتی را نیز بیش از دو برابر کرده است.

در مجموع، نتایج این مرحله نشان می‌دهند که در محیط مورد بررسی انتقال اعتبار محدود اما چندمرحله‌ای مفید است، درحالی‌که انتقال بیش از حد بلندمدت می‌تواند موجب افزایش واریانس و کاهش پایداری یادگیری شود. بر این اساس، مدل $SARSA$ با $\lambda=0.3$ و replacing trace به عنوان مدل منتخب این الگوریتم برای مقایسه‌های فصل بعد انتخاب شده است.

فصل نهم: مقایسه جامع الگوریتم‌ها و تحلیل سیاست‌ها

۹-۱. چارچوب مقایسه

در این فصل، سه روش اصلی پروژه یعنی **Q-Learning, Value Iteration** و **SARSA(λ)** تحت شرایط مشترک مقایسه می‌شوند. برای جلوگیری از اثر عوامل جانبی، مقایسه بر روی نقشه مبدأ، تابع پاداش sparse و ضریب تنزیل 0.99 انجام شده است. برای Q-Learning، مدل دارای کاهش نمایی ϵ و برای SARSA مدل اصلی با $\lambda=0.3$ و replacing trace در نظر گرفته شده است.

تمرکز مقایسه صرفاً بر مقدار پاداش نهایی نیست، بلکه زمان اجرا، میزان تعامل موردنیاز با محیط، پایداری بین اجراها، حافظه، کیفیت مسیر، حساسیت پارامتری و شباهت سیاست‌های مدل آزاد با سیاست مرجع Value Iteration نیز بررسی می‌شود.

نتایج اصلی سه مدل در جدول ۹-۱ خلاصه شده‌اند.

معیار	Value Iteration	Q-Learning	SARSA($\lambda=0.3$)
نوع روش	Model-based	Model-free, Off-policy	Model-free, On-policy
واحد آموزش	Bellman Sweep	Episode	Episode
تعداد مراحل آموزش	51 sweep	150,000 episode	150,000 episode
انتقال‌های محیط در آموزش	نیاز ندارد	7,449,430	7,296,776
زمان آموزش	13.834 s	19.87 s	31.81 s
حالت‌های ذخیره‌شده	20,850	8,986	11,147
حافظه جدول مدل منتخب	549.2 KB	447.6 KB	555.3 KB
نرخ موفقیت ارزیابی اصلی	93.00%	64.67%	85.33%
میانگین بازده	88.90	43.14	69.75

توافق سیاست با VI	مرجع	50.18%	48.39%
-------------------	------	--------	--------

جدول ۹-۱: مقایسه مدل‌های اصلی سه الگوریتم

جدول نشان می‌دهد Value Iteration از نظر کیفیت نهایی سیاست بهترین عملکرد را دارد، اما این نتیجه با دسترسی کامل آن به مدل انتقال حاصل شده است. در میان روش‌های مدل آزاد، SARSA($\lambda=0.3$) به‌وضوح عملکرد بهتری از Q-Learning داشته و نرخ موفقیت آن حدود ۲۰،۷ واحد درصد بیشتر است:

$$85.33\% - 64.67\% = 20.66\%$$

بالاین حال، این بهبود با هزینه محاسباتی بالاتر حاصل شده است؛ زیرا Eligibility Trace باعث می‌شود در هر گام بیش از یک مقدار Q به‌روزرسانی شود.

۹-۲. مقایسه زمان اجرا و هزینه محاسباتی

در اجرای اصلی، Value Iteration با زمان ۱۳،۸۳۴ ثانیه سریع‌ترین روش بوده است. Q-Learning حدود ۱۹،۸۷ ثانیه و SARSA($\lambda=0.3$) حدود ۳۱،۸۱ ثانیه زمان نیاز داشته‌اند.

اختلاف Q-Learning و SARSA را می‌توان مستقیماً به نحوه به‌روزرسانی آن‌ها مرتبط دانست. Q-Learning در هر انتقال فقط یک جفت حالت-عمل را به‌روزرسانی می‌کند، درحالی‌که SARSA(λ) علاوه بر جفت فعلی، تمام Eligibility Trace‌های فعال را نیز اصلاح می‌کند. برای $\lambda=0.3$ ، حداکثر هشت رد به‌طور هم‌زمان فعال بوده‌اند.

زمان SARSA($\lambda=0.3$) نسبت به Q-Learning حدود ۶۰ درصد بیشتر است:

$$(31.81 - 19.87)/19.87 \times 100 \approx 60.1\%$$

این افزایش زمان در مقابل بهبود محسوس نرخ موفقیت قرار می‌گیرد. بنابراین، انتخاب میان دو روش فقط براساس سرعت اجرا مناسب نیست؛ بلکه باید کیفیت سیاست حاصل نیز در نظر گرفته شود.

مقایسه Value Iteration با دو روش دیگر نیازمند احتیاط بیشتری است. Value Iteration هیچ اپیزود آموزشی را در محیط نمونه‌برداری نمی‌کند، اما در هر sweep تمام حالت‌ها و اعمال را بررسی می‌کند. تعداد ارزیابی‌های حالت-عمل آن تقریباً برابر است با:

$$51 \times 20850 \times 4 = 4253400$$

یعنی بیش از ۴۰۲۵ میلیون Bellman backup روی مدل کامل انجام شده است. این مقدار با «نمونه محیط» در روش‌های مدل‌آزاد معادل نیست؛ زیرا Value Iteration نتیجه همه انتقال‌های احتمالی را مستقیماً از مدل دریافت می‌کند.

در مقابل، Q-Learning و SARSA هیچ دانش قبلی از احتمال انتقال ندارند و بیش از هفت میلیون انتقال واقعی شبیه‌سازی‌شده را طی آموزش تجربه کرده‌اند.

۳-۹. کارایی نمونه‌ای و سرعت یادگیری

صرف یکسان‌بودن تعداد ۱۵۰۰۰۰۰ اپیزود به معنای یکسان‌بودن سرعت یادگیری نیست. برای مقایسه کارایی نمونه‌ای، در آزمایش پایداری بررسی شد که میانگین متحرک نرخ موفقیت چه زمانی برای اولین بار به ۳۰ درصد می‌رسد.

میانگین این مقدار در پنج seed برای Q-Learning برابر بوده است با:

$$N_{30\%, QL} = 101310$$

درحالی‌که برای SARSA($\lambda=0.3$):

$$N_{30\%, SARSA} = 69687$$

ثبت شده است.

بنابراین، SARSA حدود ۳۱۰۶۰۰ اپیزود زودتر به این سطح از عملکرد رسیده است. این نتیجه با نقش Eligibility Trace سازگار است؛ زیرا یک خطای TD می‌تواند به چند تصمیم قبلی منتقل شود و پاداش‌های مهم سریع‌تر در مسیر انتشار یابند.

معیار دیگری که روند کامل آموزش را بهتر نشان می‌دهد، مساحت زیر منحنی موفقیت یا Success AUC است. میانگین این مقدار در پنج اجرا برای Q-Learning برابر ۰.۱۶۹ و برای SARSA($\lambda=0.3$) برابر ۰.۳۷۰۹ بوده است. بنابراین، SARSA نه فقط در پایان، بلکه در بخش عمده فرآیند آموزش نیز موفقیت بیشتری داشته است.

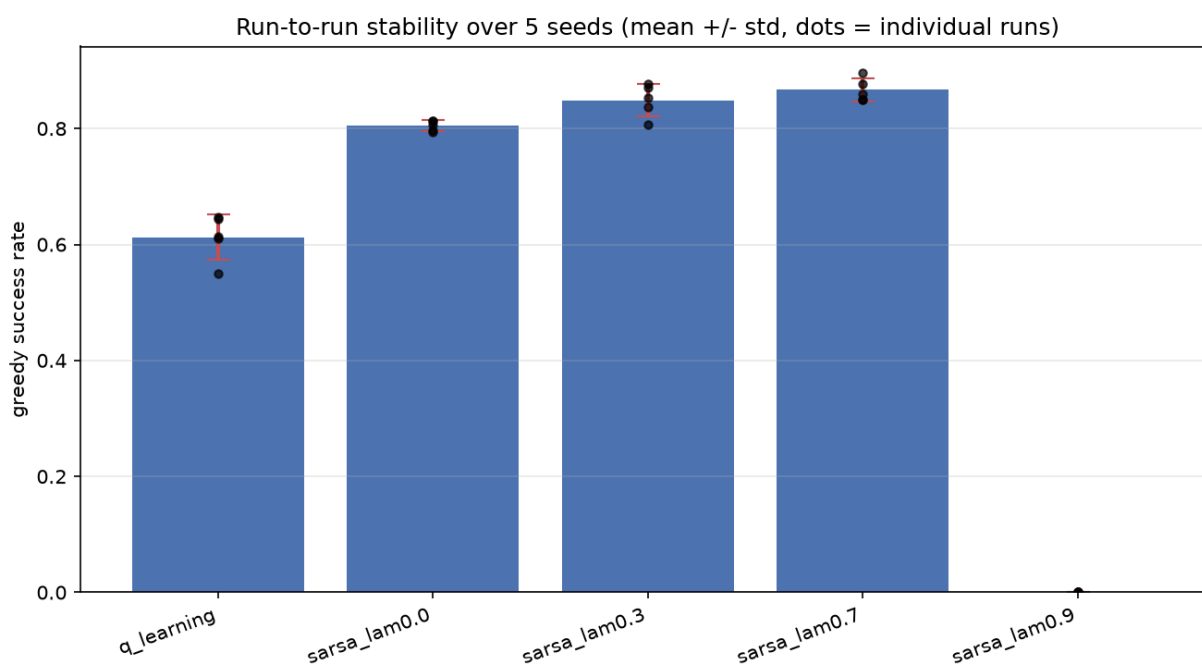
Value Iteration از نظر sample efficiency در دسته متفاوتی قرار دارد. این الگوریتم اصلاً به تجربه تصادفی نیاز ندارد و سیاست را مستقیماً از مدل استخراج می‌کند. چنین مزیتی تنها زمانی وجود دارد که تابع انتقال کامل محیط در دسترس باشد؛ شرطی که در بسیاری از مسائل واقعی برقرار نیست.

۴-۹. پایداری بین اجراهای مستقل

برای بررسی اینکه نتایج یک مدل به seed خاص وابسته نباشند، Q-Learning و چند پیکربندی SARSA با پنج seed زیر به‌طور مستقل آموزش داده شدند:

{9,17,23,42,101}

در تمام این اجراها، نقشه و تنظیمات اصلی ثابت بوده و تنها تصادفی‌بودن عامل و محیط تغییر کرده است.



شکل ۹-۱: پایداری نرخ موفقیت مدل‌های Q-Learning و SARSA برای پنج seed مستقل. ارتفاع ستون‌ها میانگین، خط خطا انحراف معیار و نقاط سیاه نتایج تک‌تک اجراها را نشان می‌دهند.

نتایج عددی مهم در جدول ۲-۹ آمده‌اند:

روش	میانگین موفقیت	انحراف معیار	کمینه	بیشینه	میانگین اپیزود رسیدن به ۳۰٪
-----	----------------	--------------	-------	--------	-----------------------------

101,310	64.67%	55.00%	3.88%	61.27%	Q-Learning
77,813	81.33%	79.33%	0.93%	80.47%	SARSA($\lambda=0$)
69,687	87.67%	80.67%	2.81%	84.87%	SARSA($\lambda=0.3$)
67,489	89.67%	85.00%	2.00%	86.67%	SARSA($\lambda=0.7$)
-	0%	0%	0%	0%	SARSA($\lambda=0.9$)

جدول ۹-۲. پایداری عملکرد در پنج اجرای مستقل

این آزمایش چند نتیجه مهم دارد. نخست، برتری SARSA نسبت به Q-Learning فقط محدود به seed اصلی پروژه نیست. حتی SARSA یک مرحله‌ای با $\lambda=0$ به‌طور متوسط بیش از ۱۹ واحد درصد نرخ موفقیت بالاتری از Q-Learning دارد.

دوم، در آزمایش چند seed-، $\lambda=0.7$ با میانگین موفقیت ۸۶٫۶۷ درصد بهترین نتیجه نهایی را ایجاد کرده است. این موضوع با نتیجه اجرای اصلی که در آن $\lambda=0.3$ اندکی بهتر بود، نشان می‌دهد اختلاف این دو مقدار کوچک و وابسته به اجرای تصادفی است.

باین حال، $\lambda=0.7$ به‌طور متوسط حدود ۴۷٫۲ ثانیه زمان آموزش نیاز داشته، درحالی‌که مقدار متناظر برای $\lambda=0.3$ حدود ۳۱٫۶ ثانیه بوده است. بنابراین، اگر معیار تنها بالاترین میانگین موفقیت باشد، $\lambda=0.7$ مناسب‌تر است؛ اما از نظر تعادل میان عملکرد، سرعت و هزینه محاسباتی، $\lambda=0.3$ همچنان انتخاب مناسبی برای مدل اصلی پروژه است.

افت کامل $\lambda=0.9$ نیز تأیید می‌کند که افزایش Eligibility Trace به‌طور یکنواخت مفید نیست.

پایداری Value Iteration

در Value Iteration، با ثابت بودن نقشه و پارامترها، فرآیند آموزش کاملاً قطعی است؛ بنابراین، واریانس ناشی از آموزش برابر صفر است.

باین حال، اجرای سیاست نهایی همچنان در محیط تصادفی صورت می‌گیرد. ارزیابی سیاست Value Iteration تحت پنج مجموعه تحقق تصادفی نرخ موفقیت متوسط ۹۵٫۴ درصد با انحراف معیار حدود ۰٫۸ درصد ایجاد کرده است.

این اختلاف با نرخ ۹۳ درصد گزارش شده در فصل ششم ناشی از استفاده از مجموعه متفاوتی از اپیزودهای تصادفی در آزمایش پایداری است و تغییر در خود سیاست Value Iteration رخ نداده است.

۵-۹. مقایسه حافظه مصرفی

حافظه مورد نیاز الگوریتم‌ها با اندازه سریال شده جدول‌هایی که برای اجرای سیاست باید ذخیره شوند اندازه‌گیری شده است.

در آزمایش پنج seed-، میانگین حافظه Q-Learning برابر ۴۴۲٫۳ KB و $SARSA(\lambda=0.3)$ برابر ۵۵۴٫۲ KB بوده است. Value Iteration برای ذخیره تابع ارزش و سیاست تمام حالت‌ها حدود ۵۴۹٫۲ KB نیاز داشته است.

نکته قابل توجه آن است که SARSA با وجود بازدید از تعداد حالت کمتر از Value Iteration، حافظه تقریباً مشابه یا اندکی بیشتر نیاز دارد. دلیل آن نوع داده ذخیره شده است. Value Iteration برای هر حالت یک مقدار $V(s)$ و یک عمل سیاست را نگه می‌دارد، در حالی که روش‌های مدل آزاد باید برای هر حالت مشاهده شده چهار مقدار Q ذخیره کنند.

Q-Learning کمترین مصرف حافظه را دارد، زیرا در اجرای اصلی تنها ۸۹۸۶ حالت از مجموع ۲۰۹۸۵۰ حالت را مشاهده کرده است. این مقدار تقریباً برابر است با:

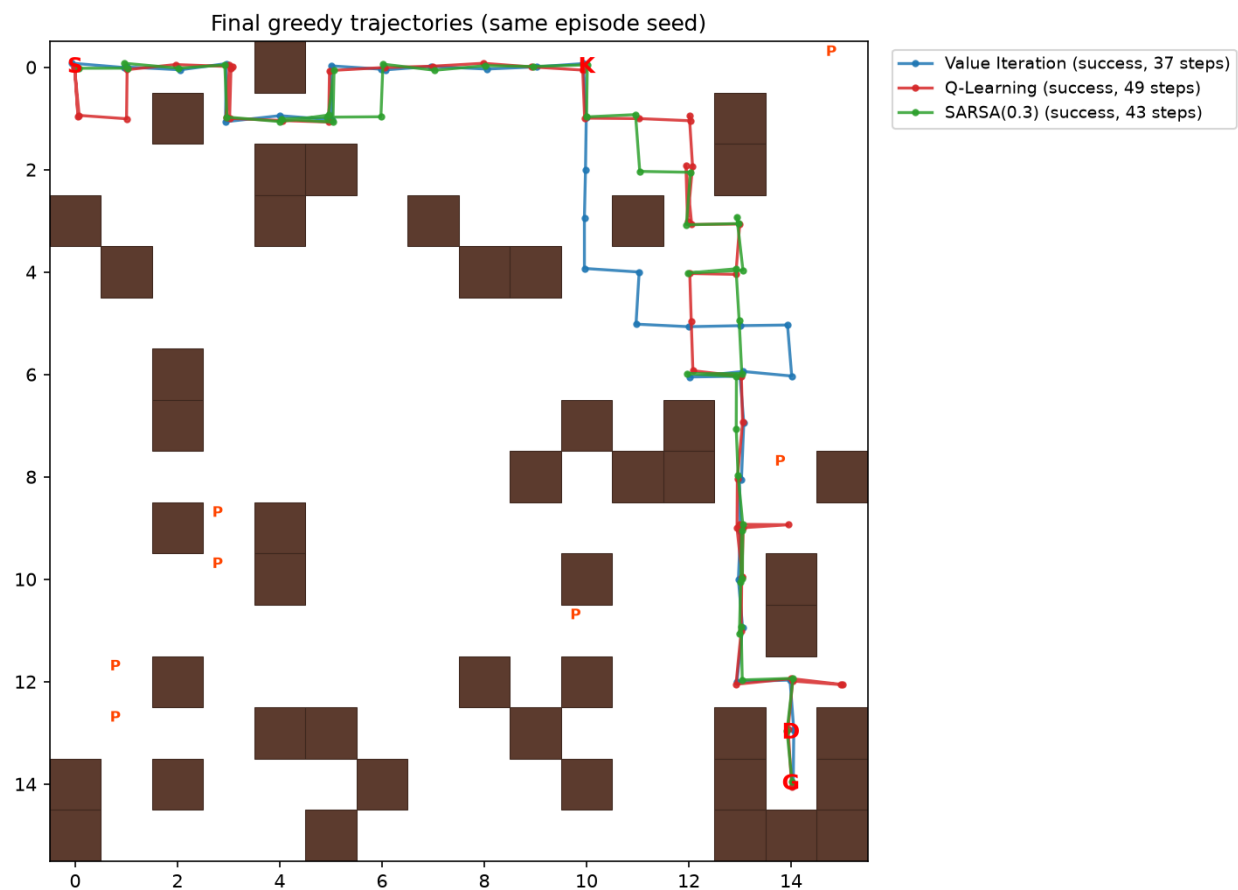
$$8986/20850 \times 100 \approx 43.1\%$$

$SARSA(\lambda=0.3)$ نیز ۱۱۰۱۴۷ حالت را بازدید کرده و در نتیجه پوشش بیشتری از فضای حالت دارد.

این تفاوت یکی از مزایای بالقوه روش‌های مدل آزاد در فضای حالت بسیار بزرگ است: لازم نیست حالت‌هایی که هرگز مشاهده نمی‌شوند حتماً پردازش یا ذخیره شوند. البته در مسئله فعلی فضای حالت به اندازه‌ای کوچک است که هر سه روش جدولی قابل استفاده هستند.

۶-۹. کیفیت مسیر حاصل

برای مقایسه بصری سیاست‌ها، هر سه مدل در یک تحقق تصادفی کاملاً یکسان از محیط اجرا شده‌اند. seed اپیزود در هر سه حالت ثابت بوده است؛ بنابراین، تفاوت مسیرها عمدتاً ناشی از سیاست آن‌هاست.



شکل ۹-۲: مسیر سیاست حریصانه Value Iteration، Q-Learning و SARSA($\lambda=0.3$) در یک تحقق تصادفی یکسان از محیط.

در این اپیزود هر سه الگوریتم موفق شده‌اند به هدف برسند، اما تعداد گام‌های آن‌ها متفاوت است:

روش	تعداد گام	برخورد با دیوار	ورود به خانه جریمه	نتیجه
Value Iteration	37	3	0	موفق
SARSA($\lambda=0.3$)	43	3	0	موفق
Q-Learning	49	3	0	موفق

Value Iteration تنها ۳۷ گام برای تکمیل مأموریت نیاز داشته است. SARSA با ۴۳ گام در رتبه بعد قرار می‌گیرد و Q-Learning با ۴۹ گام تقریباً تمام بودجه ۵۰ واحدی انرژی را مصرف کرده است.

این مثال به‌تنهایی معیار آماری نیست، اما تفاوت کیفی سیاست‌ها را به‌خوبی نشان می‌دهد. سیاست-Q-Learning در این تحقق موفق شده است، ولی تنها یک واحد انرژی حاشیه دارد؛ بنابراین، چند لغزش اضافی

می‌توانست آن را به شکست تبدیل کند. این شکنندگی با نرخ موفقیت پایین‌تر Q-Learning در ارزیابی گسترده نیز سازگار است.

SARSA مسیری نزدیک‌تر به Value Iteration ایجاد کرده و حاشیه انرژی بیشتری باقی گذاشته است. این نتیجه یکی از دلایل عملکرد بهتر آن در اپیزودهای تصادفی است.

۷-۹. میزان شباهت سیاست‌های مدل آزاد با Value Iteration

برای سنجش مستقیم سیاست‌ها، عمل حریصانه هر حالت مشاهده‌شده در Q-Learning و SARSA با عمل Value Iteration در همان حالت مقایسه شده است. این معیار در صورت پروژه نیز به‌عنوان یکی از معیارهای کیفیت سیاست درخواست شده است.

برای مدل منتخب Q-Learning، توافق برابر است با:

$$Agreement_{QL} = 50.18\%$$

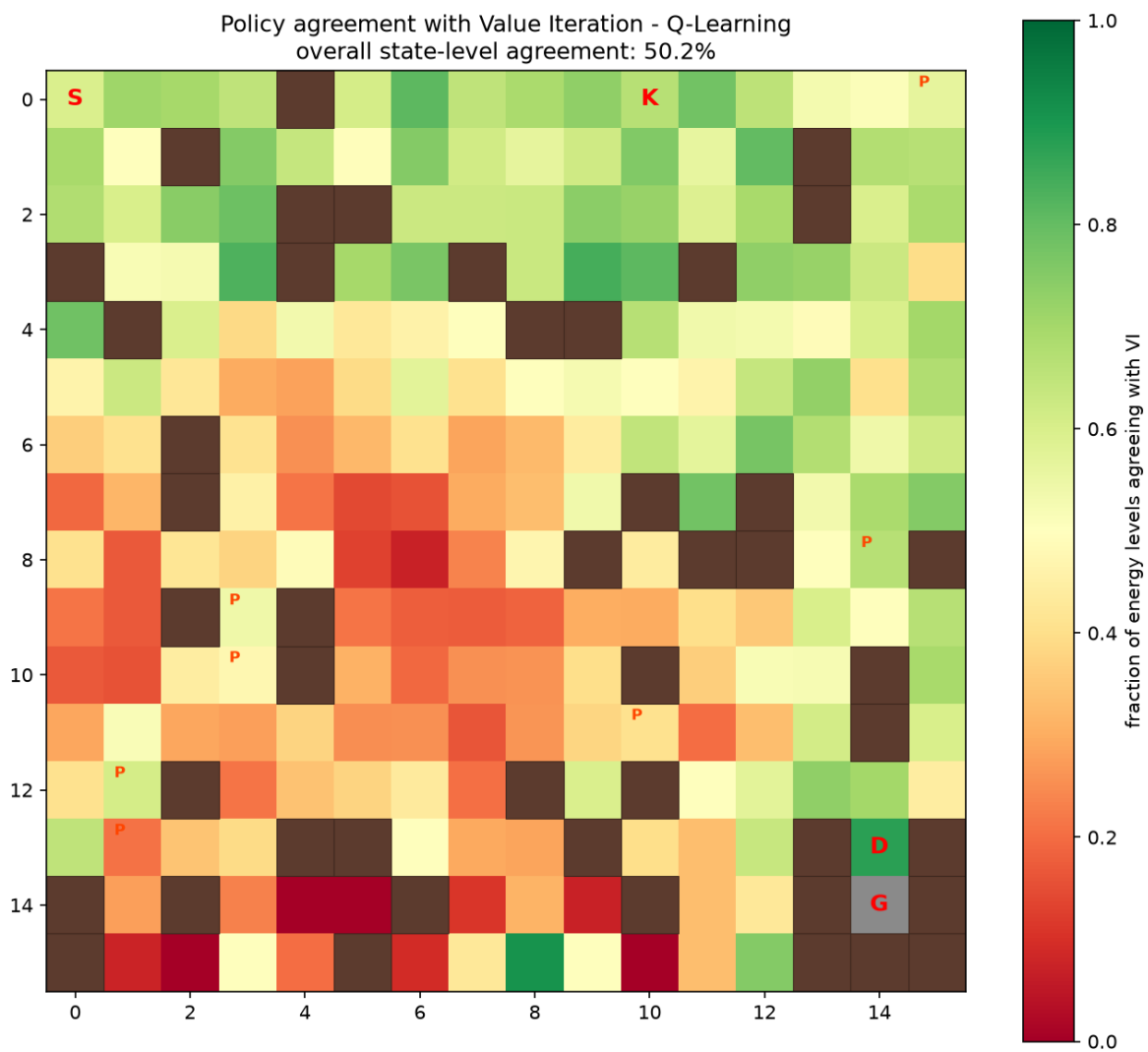
و برای SARSA($\lambda=0.3$):

$$Agreement_{SARSA} = 48.39\%$$

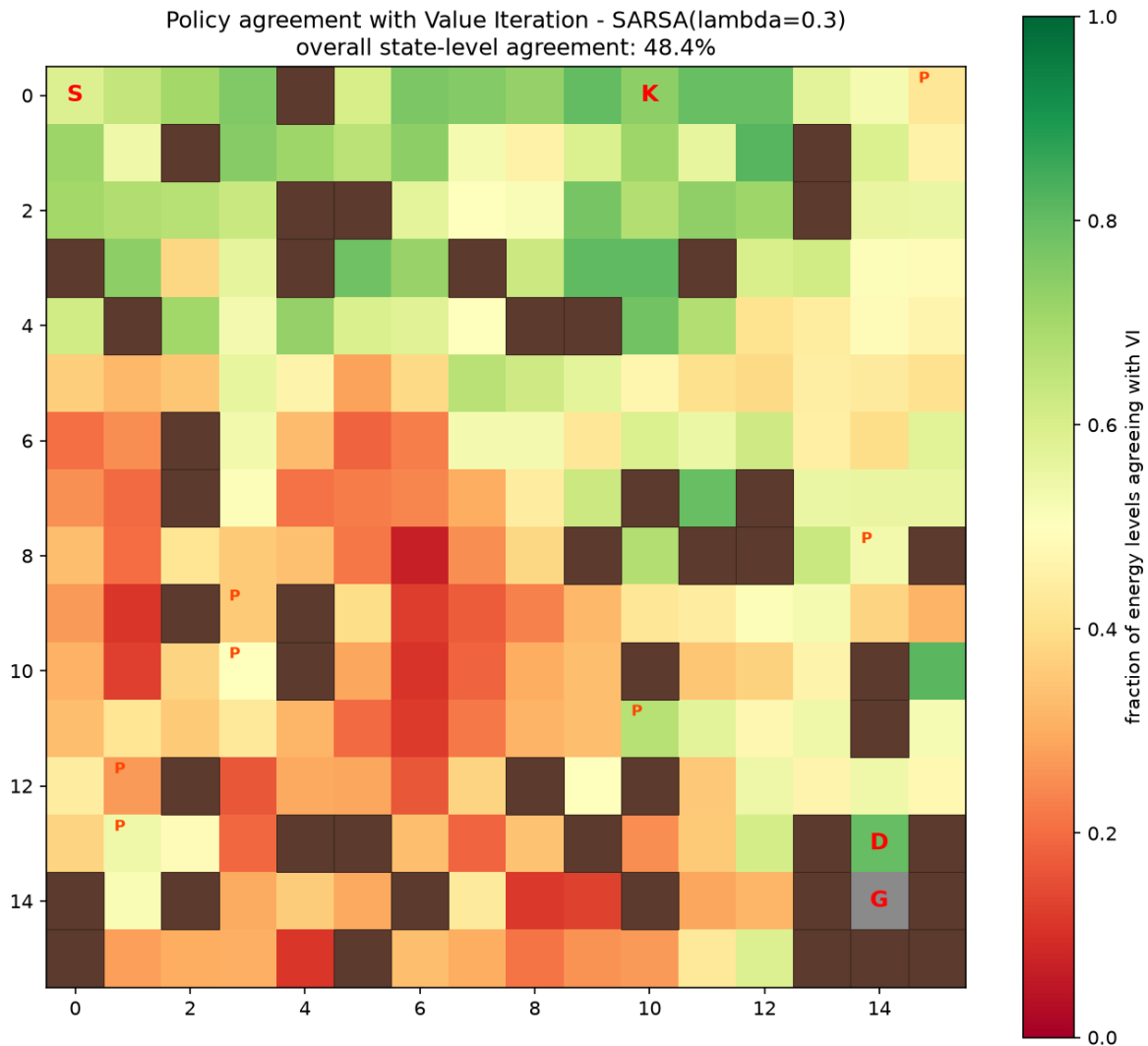
در نگاه نخست ممکن است انتظار رود SARSA به دلیل نرخ موفقیت بیشتر، توافق بالاتری با Value Iteration داشته باشد؛ اما داده‌ها چنین رابطه مستقیمی را نشان نمی‌دهند.

علت اصلی آن است که معیار agreement به تمام حالت‌های بازدیدشده وزن یکسان می‌دهد. اختلاف در یک حالت کم‌اهمیت یا میان دو عمل تقریباً هم‌ارزش، همان اندازه یک اختلاف مهم در نزدیکی هدف در درصد نهایی اثر دارد. علاوه بر این، SARSA بیش از Q-Learning فضای حالت را پوشش داده است و تصمیم‌های آن در تعداد بیشتری از حالت‌ها وارد مقایسه می‌شوند.

برای بررسی مکانی این اختلاف‌ها، نقشه‌های اختلاف سیاست رسم شده‌اند.



شکل ۹-۳: میزان توافق سیاست‌های مدل آزاد با Value Iteration. (الف) Q-Learning



میزان توافق سیاست‌های مدل آزاد با Value Iteration: (ب) SARSA($\lambda=0.3$) رنگ سبز بیانگر توافق بیشتر، قرمز بیانگر اختلاف بیشتر و خاکستری نشان‌دهنده نواحی بازدیدنشده است. مقدار هر خانه نسبت سطوح انرژی‌ای است که در آن عمل حریمانه با Value Iteration یکسان است.

نقشه‌ها نشان می‌دهند اختلاف سیاست‌ها یکنواخت نیست و در برخی نواحی و سطوح انرژی بیشتر متمرکز شده است. بنابراین، استفاده از درصد کلی بدون بررسی ساختار محلی می‌تواند گمراه‌کننده باشد.

۸-۹. تحلیل سه حالت نمونه دارای اختلاف سیاست

برای بررسی دقیق‌تر، سه حالت واقعی از مدل‌ها انتخاب شده‌اند. این سه نمونه عمده شامل یک اختلاف مهم در شروع، یک اختلاف تقریباً بی‌اثر و یک اختلاف مهم در نزدیکی هدف هستند.

۸-۹-۱. حالت اول: اختلاف Q-Learning در نقطه شروع

حالت مورد بررسی:

$$s_1 = (0,0,0,50)$$

این حالت دقیقاً نقطه شروع اپیزود است و در نتیجه در هر ۱۵۰۰۰۰ اپیزود آموزشی مشاهده شده است.

اعمال انتخابی سه سیاست عبارت‌اند از:

عمل	روش
RIGHT	Value Iteration
DOWN	Q-Learning
RIGHT	SARSA(0.3)

مقادیر مورد انتظار Value Iteration برای دو عمل اصلی برابرند با:

$$Q_{VI}(RIGHT) = 64.718$$

$$Q_{VI}(DOWN) = 59.892$$

بنابراین، Value Iteration ترجیح نسبتاً مشخصی برای حرکت به راست دارد. این جهت عامل را وارد مسیر بالایی منتهی به کلید می‌کند.

اما مقادیر یادگرفته‌شده توسط Q-Learning عبارت‌اند از:

$$Q_{QL}(DOWN) = 32.595$$

$$Q_{QL}(RIGHT) = 30.611$$

و در نتیجه Q-Learning حرکت پایین را انتخاب می‌کند.

این اختلاف را نمی‌توان به بازدید کم حالت نسبت داد، زیرا نقطه شروع بیشترین تعداد بازدید ممکن را دارد. محتمل‌تر است که ترکیب نمونه‌برداری تصادفی، نرخ یادگیری ثابت و ماهیت off-policy باعث شده برآورد تجربی دو مسیر در جدول Q با مقدار دقیق مدل متفاوت باقی بماند.

SARSA در همین حالت عمل RIGHT را انتخاب کرده و با سیاست مرجع هم‌جهت است. این اختلاف در ابتدای اپیزود می‌تواند یکی از عواملی باشد که مسیر Q-Learning را طولانی‌تر می‌کند.

۹-۸-۲. حالت دوم: اختلافی با اعمال تقریباً هم‌ارزش

حالت دوم عبارت است از:

$$s_2 = (1,0,0,49)$$

اعمال انتخابی:

عمل	روش
RIGHT	Value Iteration
RIGHT	Q-Learning
UP	SARSA(0.3)

مقادیر Value Iteration عبارت‌اند از:

$$Q_{VI}(RIGHT) = 60.931$$

$$Q_{VI}(UP) = 60.206$$

$$Q_{VI}(DOWN) = 60.209$$

اختلاف میان بهترین عمل و دو گزینه دیگر کمتر از یک واحد است. بنابراین، اگر SARSA به جای RIGHT، عمل UP را انتخاب کند، از دید معیار policy agreement یک «اختلاف کامل» ثبت می‌شود، درحالی‌که مقدار مورد انتظار دو عمل بسیار نزدیک است.

این نمونه نشان می‌دهد چرا توافق حدود ۴۸ تا ۵۰ درصد نباید مستقیماً به معنای «نیمی از سیاست اشتباه است» تفسیر شود. برخی از اختلاف‌ها میان اعمال تقریباً هم‌ارزش رخ می‌دهند و اثر آن‌ها بر نرخ موفقیت بسیار کم است.

۳-۸-۹. حالت سوم: اختلاف مهم در نزدیکی هدف

حالت سوم در موقعیت در، پس از دریافت کلید و با تنها سه واحد انرژی باقی‌مانده است:

$$s_3 = (13,14,1,3)$$

عامل در این حالت درست یک خانه بالاتر از هدف قرار دارد. عمل DOWN می‌تواند مستقیماً عامل را به هدف منتقل کند، درحالی‌که چپ و راست به دیوار منتهی می‌شوند.

اعمال سیاست‌ها عبارت‌اند از:

عمل	روش
DOWN	Value Iteration

DOWN	Q-Learning
RIGHT	SARSA(0.3)

مقادیر مورد انتظار Value Iteration :

$$Q_{VI}(DOWN) = 96.593$$

در حالی که:

$$Q_{VI}(RIGHT) = 84.673$$

است.

بنابراین، این بار اختلاف اعمال صرفاً یک tie یا انتخاب تقریباً هم‌ارزش نیست. حرکت DOWN به‌وضوح تصمیم مناسب‌تر است.

Q-Learning نیز این عمل را درست تشخیص داده است. اما جدول SARSA مقدار زیر را برای RIGHT آموخته است:

$$Q_{SARSA}(RIGHT) = 89.167$$

در مقابل:

$$Q_{SARSA}(DOWN) = 74.243$$

بنابراین، SARSA در این حالت رتبه‌بندی نادرستی میان دو عمل ایجاد کرده است.

یکی از دلایل محتمل آن، ترکیب انتقال تصادفی، نرخ یادگیری ثابت و **Eligibility Trace** است. حتی هنگام انتخاب RIGHT، لغزش عمودی ممکن است گاهی عامل را به سمت پایین و هدف منتقل کند و پاداش بالایی ایجاد نماید. Value Iteration این رویداد را دقیقاً با احتمال صحیح وزن می‌دهد، اما تخمین نمونه‌ای SARSA در طول آموزش همچنان می‌تواند تحت تأثیر نوسان‌های تصادفی باقی بماند.

با توجه به انرژی بسیار کم این حالت، چنین اختلافی می‌تواند مستقیماً موفقیت یا شکست یک اپیزود را تعیین کند. این نمونه در مقابل حالت دوم نشان می‌دهد که محل و شدت اختلاف مهم‌تر از تعداد خام اختلاف‌هاست.

۹-۹. حساسیت الگوریتم‌ها به پارامترها

نتایج فصل‌های قبلی نشان دادند هر سه الگوریتم به پارامترهای خود حساس هستند، اما شدت این حساسیت یکسان نیست.

Value Iteration نسبت به تغییر γ از ۰٫۹۰ تا ۰٫۹۹ تغییر بزرگی در مقدار تابع ارزش نشان داد، اما ساختار سیاست نسبتاً پایدار بود؛ حتی در $\gamma=0.90$ بیش از ۹۳ درصد اعمال با سیاست $\gamma=0.99$ یکسان بودند. بنابراین، در محدوده آزمایش‌شده، سیاست Value Iteration نسبتاً مقاوم بوده است.

در Q-Learning، شکل برنامه کاهش ϵ اثر قابل‌توجهی داشت. تنها تغییر از برنامه خطی به نمایی در پاداش تنک، نرخ موفقیت نهایی را از ۵۴ درصد به ۶۴٫۶۷ درصد افزایش داد. بنابراین، نحوه گذار از exploration به exploitation یکی از عوامل اصلی عملکرد Q-Learning بوده است.

SARSA بیشترین حساسیت را نسبت به λ نشان داد. مقادیر ۰٫۳ و ۰٫۷ عملکرد بسیار مناسب داشتند، اما افزایش آن به ۰٫۹ منجر به شکست کامل یادگیری شد. بنابراین، Eligibility Trace ابزاری مفید است، اما انتخاب بیش از حد بزرگ آن می‌تواند اثر معکوس ایجاد کند.

۹-۱۰. جمع‌بندی مقایسه

نتایج این فصل نشان می‌دهند هیچ معیار منفردی برای رتبه‌بندی سه الگوریتم کافی نیست.

Value Iteration بهترین سیاست نهایی را ایجاد کرد و در ارزیابی اصلی به موفقیت ۹۳ درصد رسید. همچنین، به دلیل دسترسی به مدل کامل، نتیجه آموزش آن قطعی است. محدودیت اصلی این روش وابستگی کامل به مدل انتقال و نیاز به پردازش تمام فضای حالت است.

Q-Learning ساده‌ترین و سریع‌ترین روش مدل‌آزاد بود و کمترین حافظه را مصرف کرد، اما در این مسئله از نظر نرخ موفقیت، سرعت یادگیری و کیفیت مسیر ضعیف‌تر از SARSA بود. مدل منتخب آن تنها ۶۴٫۶۷ درصد موفقیت داشت و در مسیر نمونه نیز ۴۹ واحد از ۵۰ واحد انرژی را مصرف کرد.

SARSA(λ) بهترین عملکرد را در میان روش‌های مدل‌آزاد ارائه داد. مدل $\lambda=0.3$ در اجرای اصلی نرخ موفقیت ۸۵٫۳۳ درصد به دست آورد و نسبت به Q-Learning حدود ۲۰٫۷ واحد درصد بهتر بود. در آزمایش پنج-seed، $\lambda=0.7$ بیشترین میانگین موفقیت را ایجاد کرد، اما هزینه محاسباتی آن به‌طور قابل‌توجهی بیشتر بود. بنابراین، $\lambda=0.3$ از نظر نسبت عملکرد به هزینه همچنان انتخاب متعادل‌تری است.

یکی از مهم‌ترین نتایج مقایسه سیاست‌ها آن است که درصد توافق خام با **Value Iteration** به‌تنهایی معیار کافی برای کیفیت سیاست نیست. **SARSA** با وجود توافق حالت‌به‌حالت کمی پایین‌تر از **Q-Learning**، نرخ موفقیت بسیار بیشتری به دست آورده است. تحلیل سه حالت نمونه نیز نشان داد برخی اختلاف‌ها تقریباً بی‌اثر هستند، درحالی‌که برخی دیگر در نواحی حساس مانند نقطه شروع یا نزدیکی هدف می‌توانند مستقیماً بر نتیجه اپیزود اثر بگذارند.

در مجموع، **Value Iteration** معیار بالای عملکرد را در صورت وجود مدل فراهم می‌کند؛ در نبود مدل، **SARSA** با **Eligibility Trace** متوسط در این محیط مؤثرترین راهکار تجربی بوده است. فصل بعد به بررسی این موضوع می‌پردازد که آیا دانش آموخته‌شده توسط **Q-Learning** در محیط مبدأ می‌تواند به محیط‌های جدید منتقل شود و چه زمانی این انتقال مفید یا مضر خواهد بود.

فصل دهم: انتقال یادگیری در Q-Learning و تحلیل نتایج

۱-۱۰. چارچوب آزمایش انتقال یادگیری

در این مرحله بررسی شد که آیا دانش آموخته شده توسط Q-Learning در محیط مبدأ می تواند فرایند یادگیری در یک محیط جدید را بهبود دهد یا خیر. انتقال یادگیری در این پروژه فقط برای Q-Learning انجام شده و دو محیط مقصد با میزان تغییر متفاوت در نظر گرفته شده اند: یک محیط مشابه با تغییرات محدود و یک محیط متفاوت با تغییرات ساختاری گسترده تر. مطابق تعریف پروژه، انتقال باید از نظر عملکرد اولیه، سرعت یادگیری، عملکرد نهایی و احتمال انتقال منفی به صورت جداگانه ارزیابی شود.

جدول Q مدل منتخب فصل هفتم، یعنی `ql_sparse_exponential_q.pkl`، به عنوان دانش مبدأ استفاده شده است. این جدول شامل ۸۹۸۶ حالت مشاهده شده در محیط مبدأ است.

برخلاف آموزش اولیه Q-Learning که ۱۵۰۰۰۰۰ اپیزود داشت، هر آزمایش انتقال فقط برای ۶۰۰۰۰۰ اپیزود اجرا شده است. این انتخاب عمدی است؛ زیرا هدف اصلی این بخش مشاهده اثر دانش اولیه بر شروع و سرعت سازگاری است، نه آموزش همه مدل ها تا نقطه اشباع کامل.

پارامترهای اصلی Q-Learning بدون تغییر باقی مانده اند:

$$\alpha = 0.2$$
$$\gamma = 0.99$$

و همان برنامه نمایی کاهش ϵ استفاده شده است. ارزیابی اولیه و نهایی هر مدل نیز در ۳۰۰ اپیزود با سیاست حریصانه انجام شده است.

۲-۱۰. محیط های مقصد

دو نقشه مقصد مستقیماً از نقشه مبدأ تولید و همانند محیط اصلی با BFS اعتبارسنجی شده اند. محیط «مشابه» باید تقریباً ۱۵ تا ۲۰ درصد موانع جابه جا شده داشته باشد، در حالی که در محیط «متفاوت» حداقل ۳۵ درصد موانع تغییر می کنند و موقعیت یکی از عناصر اصلی نیز تغییر می یابد.

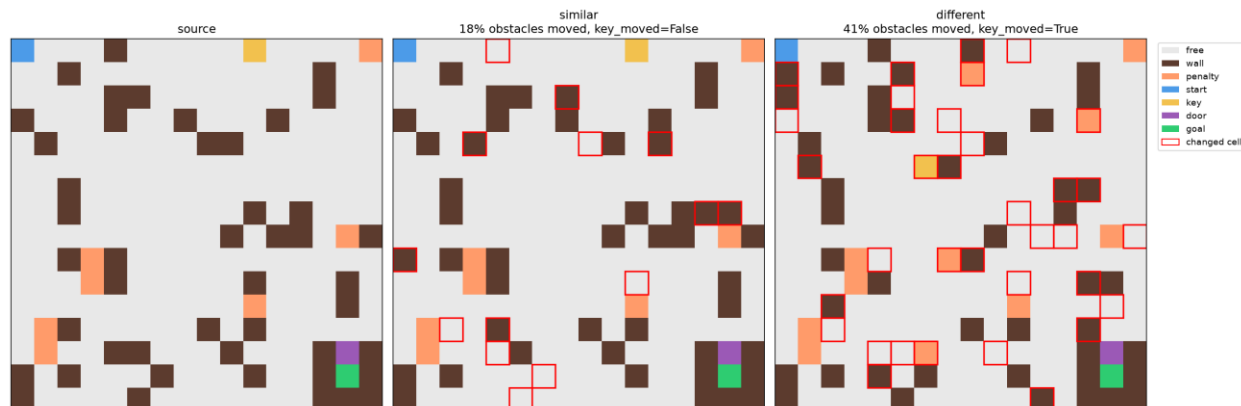
مشخصات نقشه‌های حاصل در جدول ۱-۱۰ آمده است.

ویژگی	مبدأ	مقصد مشابه	مقصد متفاوت
اندازه نقشه	16×16	16×16	16×16
تعداد دیوار	46	46	46
نسبت دیوار	17.97%	17.97%	17.97%
موانع جابه‌جا شده	—	7	16
نسبت تغییر موانع	—	17.95%	41.03%
کل خانه‌های تغییر کرده	—	14	36
موقعیت کلید	(0,10)	(0,10)	(5,6)
کلید جابه‌جا شده	—	خیر	بله
تعداد خانه جریمه	7	7	11
خانه جریمه جدید	—	0	4
کوتاه‌ترین مسیر شروع تا کلید	12	10	11
کوتاه‌ترین مسیر کلید تا هدف	18	22	17
طول کل مسیر کوتاه	30	32	28

جدول ۱-۱۰: مقایسه ساختاری محیط مبدأ و دو محیط مقصد

در محیط مشابه، نقطه شروع، کلید، در و هدف ثابت مانده‌اند و تنها هفت مانع جابه‌جا شده‌اند. در نتیجه، ۱۴ خانه از شبکه نسبت به مبدأ تغییر نوع داده‌اند.

در محیط متفاوت، ۱۶ مانع جابه‌جا شده‌اند، کلید از (0,10) به (5,6) منتقل شده و چهار خانه جریمه جدید نیز اضافه شده‌اند. مجموعاً ۳۶ خانه نسبت به نقشه مبدأ تغییر کرده‌اند.



شکل ۱۰-۱: مقایسه نقشه مبدأ با محیط‌های مقصد مشابه و متفاوت. خانه‌هایی که نسبت به محیط مبدأ تغییر کرده‌اند با کادر قرمز مشخص شده‌اند.

نکته قابل توجه آن است که اصطلاح «مشابه» و «متفاوت» میزان فاصله ساختاری از محیط مبدأ را نشان می‌دهد، نه دشواری مطلق مسئله. در واقع، کوتاه‌ترین مسیر محیط متفاوت ۲۸ گام است، درحالی‌که این مقدار در محیط مشابه ۳۲ گام است. همان‌طور که در نتایج مشاهده خواهد شد، آموزش از صفر نیز در محیط متفاوت بسیار موفق‌تر از محیط مشابه بوده است.

۳-۱۰. سناریوهای انتقال دانش

چهار روش اصلی انتقال مطابق الزامات پروژه بررسی شده‌اند: آموزش از صفر، انتقال کامل، انتقال مقیاس‌شده و انتقال انتخابی.

آموزش از صفر

در این حالت هیچ دانشی از محیط مبدأ استفاده نمی‌شود:

$$Q_T(s, a) = 0$$

این روش خط مبنای مقایسه است.

انتقال کامل

تمام جدول Q محیط مبدأ بدون تغییر به عامل مقصد داده می‌شود:

$$Q_T(s, a) = Q_S(s, a)$$

بنابراین، هر ۸۹۸۶ حالت ذخیره شده منتقل می شوند.

انتقال مقیاس شده

دانش مبدأ با ضریب β کاهش داده می شود:

$$Q_T(s, a) = \beta Q_S(s, a)$$

سه مقدار زیر آزمایش شده اند:

$$\beta \in \{0.25, 0.50, 0.75\}$$

ضریب مثبت β ترتیب حریصانه اعمال را در آغاز تغییر نمی دهد، اما میزان اعتماد عددی به مقادیر قدیمی را کاهش می دهد و بنابراین می تواند سرعت اصلاح Q در محیط جدید را تحت تأثیر قرار دهد.

انتقال انتخابی

در انتقال انتخابی، جدول Q فقط برای حالت هایی منتقل می شود که همسایگی محلی 3×3 موقعیت آن ها در دو نقشه کاملاً یکسان باشد.

این روش در محیط مشابه ۵۵۳۳ حالت را منتقل کرده است:

$$5533/8986 = 61.57\%$$

اما در محیط متفاوت، تنها ۲۶۸۷ حالت شرایط انتقال را داشته اند:

$$2687/8986 = 29.90\%$$

کاهش شدید این نسبت با میزان بیشتر تغییر ساختاری نقشه متفاوت سازگار است.

۴-۱۰. معیارهای ارزیابی انتقال

برای جلوگیری از نتیجه گیری صرفاً بر اساس عملکرد نهایی، سه مرحله جداگانه ارزیابی شده اند.

Jumpstart عملکرد سیاست حریصانه پیش از انجام حتی یک اپیزود آموزش در محیط مقصد است. این معیار مشخص می کند دانش مبدأ در لحظه ورود به محیط جدید تا چه اندازه قابل استفاده است.

برای اندازه‌گیری سرعت یادگیری از دو معیار استفاده شده است. نخست، Success AUC که در پیاده‌سازی برابر میانگین موفقیت در کل ۶۰,۰۰۰ اپیزود آموزشی است:

$$Success_{AUC} = (1/N) \sum_{i=1}^N Success_i$$

مقدار بالاتر نشان می‌دهد عامل بخش بیشتری از کل دوره آموزش را در سطح عملکرد مناسب سپری کرده است.

معیار دوم، اولین اپیزودی است که میانگین متحرک ۱۰۰۰ اپیزودی نرخ موفقیت به ۳۰ درصد می‌رسد:

$$Success_{MA} \geq 0.30$$

در نهایت، عملکرد نهایی با اجرای ۳۰۰ اپیزود حریصانه پس از پایان آموزش اندازه‌گیری شده است.

این تفکیک اهمیت زیادی دارد؛ زیرا همان‌طور که نتایج نشان می‌دهند، یک روش انتقال می‌تواند یادگیری را سریع‌تر کند اما در نهایت سیاست ضعیف‌تری نسبت به آموزش از صفر ایجاد نماید.

۵-۱۰. نتایج انتقال در محیط مشابه

نتایج کامل محیط مشابه در جدول ۲-۱۰ ارائه شده‌اند.

روش	نسبت انتقال	Jumpstart Return	Success AUC	رسیدن به موفقیت ۳۰٪	موفقیت نهایی	بازده نهایی
Scratch	0%	-129.92	0.0038	نرسید	6.00%	-27.50
Full	100%	-49.19	0.1227	46,624	46.00%	23.09
$\beta=0.25$	100%	-49.19	0.0756	نرسید	32.00%	7.65
$\beta=0.50$	100%	-49.19	0.1101	46,475	42.67%	18.49
$\beta=0.75$	100%	-49.35	0.1138	48,775	33.67%	6.75
Selective	61.57%	-79.46	0.0855	57,757	41.33%	17.86

جدول ۲-۱۰: نتایج انتقال یادگیری در محیط مقصد مشابه

نخستین نکته این است که نرخ موفقیت Jumpstart در تمام سناریوها صفر بوده است. بنابراین، هیچ‌یک از سیاست‌های منتقل‌شده بدون سازگاری با نقشه جدید قادر به تکمیل مأموریت در ۳۰۰ اپیزود ارزیابی اولیه نبوده‌اند.

بالین حال، بازده اولیه اطلاعات بیشتری ارائه می‌کند. آموزش از صفر میانگین بازده 129.92- داشته است، درحالی که انتقال کامل و مقیاس شده این مقدار را به حدود 49.2- رسانده‌اند. بنابراین، دانش مبدأ حتی اگر برای موفقیت کامل کافی نبوده، عامل را به رفتار بسیار مناسب‌تری نسبت به جدول Q خالی هدایت کرده است.

۱-۵-۱۰. برتری آشکار انتقال نسبت به آموزش از صفر

پس از ۶۰,۰۰۰ اپیزود، آموزش از صفر فقط به نرخ موفقیت ۶ درصد رسیده است. در مقابل، انتقال کامل موفقیت ۴۶ درصد ایجاد کرده است:

$$46\% - 6\% = 40\%$$

یعنی ۴۰ واحد درصد بهبود مطلق.

میانگین بازده نیز از 27.50- به 23.09 افزایش یافته است؛ اختلافی برابر با:

$$23.09 - (-27.50) = 50.59$$

همچنین، مدل scratch در طول ۶۰,۰۰۰ اپیزود هرگز به میانگین متحرک موفقیت ۳۰ درصد نرسیده است، درحالی که انتقال کامل تقریباً در اپیزود ۴۶,۶۲۴ از این آستانه عبور کرده است.

این نتایج یک نمونه روشن از انتقال مثبت هستند. ساختار بخش بزرگی از محیط مشابه بدون تغییر باقی مانده و مقدارهای Q مبدأ، حتی پس از جابه‌جایی چند مانع، اطلاعات قابل استفاده‌ای درباره جهت کلی مأموریت حفظ کرده‌اند.

۲-۵-۱۰. اثر ضریب β

در میان انتقال‌های مقیاس شده، $\beta=0.5$ بهترین نتیجه را ارائه کرده است. این مدل در اپیزود ۴۶,۴۷۵ به آستانه موفقیت ۳۰ درصد رسیده که حتی اندکی زودتر از انتقال کامل است و نرخ موفقیت نهایی ۴۲,۶۷ درصد دارد.

در مقابل، کاهش بیش از حد مقادیر با $\beta=0.25$ موجب شده مدل در ۶۰,۰۰۰ اپیزود به آستانه ۳۰ درصد نرسد و موفقیت نهایی آن فقط ۳۲ درصد باشد.

از سوی دیگر، $\beta=0.75$ با وجود AUC نسبتاً مناسب، در ارزیابی نهایی تنها ۳۳,۶۷ درصد موفقیت داشته است.

بنابراین، در محیط مشابه کاهش متوسط اعتماد به دانش قبلی مفید بوده، اما انتقال کامل همچنان بهترین عملکرد نهایی و بالاترین AUC را به دست آورده است.

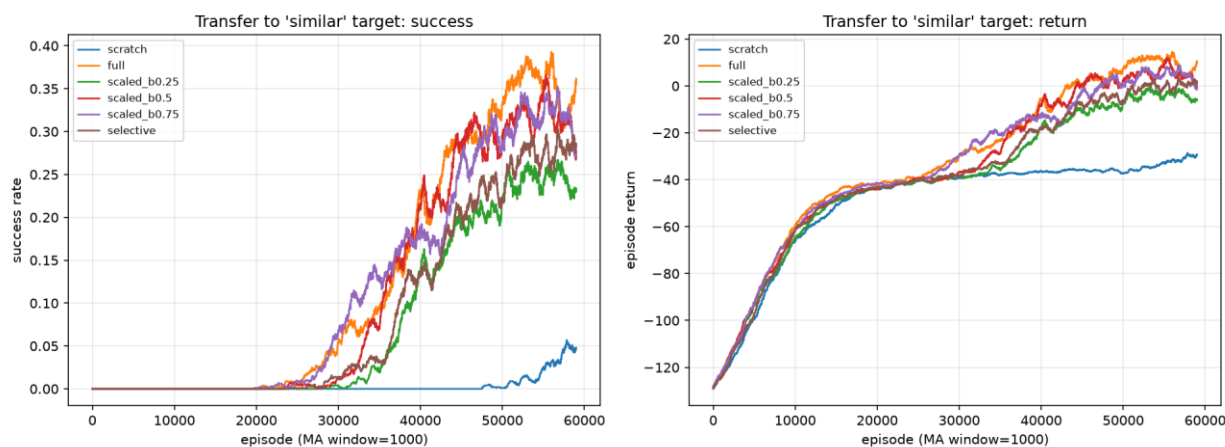
۳-۵-۱۰. انتقال انتخابی

انتقال انتخابی تنها ۶۱،۵۷ درصد حالت‌های جدول مبدأ را حفظ کرده است. موفقیت نهایی آن ۴۱،۳۳ درصد است که بسیار بهتر از آموزش از صفر، اما کمی پایین‌تر از انتقال کامل است.

این نتیجه منطقی است: در محیطی که تغییرات ساختاری محدود هستند، حذف نزدیک به ۳۸ درصد دانش مبدأ تا حدی محافظه‌کارانه بوده است. بخش زیادی از دانش حذف‌شده احتمالاً همچنان قابل استفاده بوده است.

در نتیجه، هنگامی که محیط مقصد واقعاً مشابه محیط مبدأ است، انتقال گسترده دانش سودمندتر از فیلتر بسیار محافظه‌کارانه آن بوده است.

۳-۶-۱۰. منحنی‌های یادگیری محیط مشابه



شکل ۱۰-۲: منحنی نرخ موفقیت و بازده در سناریوهای انتقال یادگیری: (الف) محیط مقصد مشابه

این تصویر شامل نرخ موفقیت و بازده با میانگین متحرک ۱۰۰۰ اپیزودی برای تمام سناریوهای انتقال است.

در بخش ابتدایی منحنی‌ها، تمام روش‌ها نرخ موفقیت نزدیک صفر دارند. با ادامه آموزش، انتقال کامل و $\beta=0.5$ زودتر از سایر روش‌ها شروع به افزایش می‌کنند. منحنی scratch در بیشتر دوره آموزش تقریباً در سطح صفر باقی می‌ماند.

بنابراین، اثر اصلی انتقال در محیط مشابه نه یک موفقیت فوری، بلکه کاهش زمان لازم برای کشف و تثبیت سیاست قابل استفاده بوده است.

۷-۱۰. نتایج انتقال در محیط متفاوت

رفتار انتقال در محیط متفاوت به‌طور محسوسی پیچیده‌تر است. نتایج در جدول ۳-۱۰ ارائه شده‌اند.

روش	نسبت انتقال	Jumpstart Return	Success AUC	رسیدن به موفقیت ۳۰٪	موفقیت نهایی	بازده نهایی
Scratch	0%	-154.59	0.2019	40,933	73.33%	60.41
Full	100%	-229.85	0.2547	33,699	66.33%	49.93
$\beta=0.25$	100%	-229.85	0.2696	32,149	70.67%	54.79
$\beta=0.50$	100%	-229.85	0.2559	33,118	67.00%	52.42
$\beta=0.75$	100%	-229.85	0.2521	33,685	63.67%	45.56
Selective	29.90%	-165.19	0.2125	39,196	74.00%	62.26

جدول ۳-۱۰: نتایج انتقال یادگیری در محیط مقصد متفاوت

برخلاف محیط مشابه، آموزش از صفر در محیط متفاوت عملکرد بسیار مناسبی داشته است و به نرخ موفقیت نهایی ۷۳٫۳۳ درصد رسیده است.

این نتیجه بار دیگر نشان می‌دهد که «متفاوت‌تر بودن از نقشه مبدأ» الزاماً به معنای «سخت‌تر بودن» نیست. کوتاه‌ترین مسیر این محیط تنها ۲۸ گام است، درحالی‌که در محیط مشابه ۳۲ گام است. چینش موانع نیز می‌تواند بر دشواری یادگیری اثر داشته باشد. بنابراین، فاصله ساختاری و دشواری خود مسئله دو مفهوم جداگانه‌اند.

۷-۱۰-۱. انتقال منفی در عملکرد آغازین

در محیط متفاوت، انتقال کامل به‌جای بهبود Jumpstart، بازده اولیه را به‌شدت کاهش داده است:

$$Scratch = -154.59$$

در برابر:

$$Full = -229.85$$

بنابراین، عامل دارای دانش مبدأ در ابتدای ورود به محیط جدید عملکردی بدتر از عامل بدون دانش اولیه داشته است.

این یک نمونه واضح از انتقال منفی است. موقعیت کلید تغییر کرده، بیش از ۴۱ درصد موانع قابل جابه‌جایی تغییر کرده‌اند و چهار خانه جریمه جدید نیز اضافه شده‌اند. در نتیجه، تعدادی از ترجیحات قوی جدول Q مبدأ دیگر با ساختار مقصد سازگار نیستند.

نکته مهم این است که مقیاس کردن جدول Q این مشکل Jumpstart را برطرف نکرده است. چون ضرب تمام مقادیر Q یک حالت در ضریب مثبت β ترتیب اعمال حریصانه را تغییر نمی‌دهد، رفتار اولیه مدل‌های $\beta=0.25$ ، 0.5 و 0.75 تقریباً همان رفتار انتقال کامل باقی می‌ماند.

۲-۷-۱۰. سرعت یادگیری: مزیت انتقال با وجود Jumpstart منفی

با وجود عملکرد اولیه نامناسب، انتقال کامل در ادامه آموزش سریع‌تر از scratch یاد گرفته است.

عامل scratch در اپیزود ۴۰,۹۳۳ به نرخ موفقیت ۳۰ درصد رسیده است، درحالی‌که انتقال کامل در اپیزود ۳۳,۶۹۹ به این سطح رسیده است؛ یعنی تقریباً:

$$40933 - 33699 = 7234$$

اپیزود زودتر.

مدل $\beta=0.25$ حتی بهتر عمل کرده و در اپیزود ۳۲,۱۴۹ به آستانه رسیده است:

$$40933 - 32149 = 8784$$

اپیزود زودتر از scratch.

همچنین، بالاترین Success AUC کل محیط متفاوت متعلق به $\beta=0.25$ و برابر ۰,۲۶۹۶ بوده است. بنابراین، دانش مبدأ با وجود ایجاد رفتار ضعیف در نقطه شروع، پس از آغاز سازگاری موجب افزایش سرعت یادگیری شده است.

این نتیجه اهمیت تفکیک معیارهای انتقال را نشان می‌دهد. در این آزمایش، یک روش می‌تواند هم‌زمان:

- Jumpstart منفی داشته باشد؛
- سرعت یادگیری را افزایش دهد؛
- ولی عملکرد نهایی را کاهش دهد.

بنابراین، استفاده از عبارتی مانند «انتقال موفق بود» یا «انتقال شکست خورد» بدون مشخص کردن معیار کافی نیست.

۳-۷-۱۰. عملکرد نهایی و باقی ماندن اثر منفی

در پایان ۶۰,۰۰۰ اپیزود، انتقال کامل موفقیت ۶۶,۳۳ درصد داشته است، در حالی که scratch به ۷۳,۳۳ درصد رسیده است:

$$66.33\% - 73.33\% = -7.00\%$$

بنابراین، با وجود یادگیری سریع تر، انتقال کامل در پایان هفت واحد درصد ضعیف تر از آموزش از صفر بوده است.

مقادیر مقیاس شده نیز نتوانسته اند کاملاً از scratch عبور کنند. بهترین آن ها، $\beta=0.25$ ، موفقیت ۷۰,۶۷ درصد دارد که ۲,۶۶ واحد درصد کمتر از scratch است.

این یافته نشان می دهد بخشی از اطلاعات اشتباه مبدأ می تواند حتی پس از ده ها هزار اپیزود اثر خود را در جدول Q حفظ کند، به ویژه در حالت هایی که در مقصد کمتر مورد بازدید قرار می گیرند.

۸-۱۰. موفقیت انتقال انتخابی در محیط متفاوت

مهم ترین نتیجه محیط متفاوت مربوط به انتقال انتخابی است. این روش فقط ۲۹,۰۹ درصد حالت های مبدأ را منتقل کرده است و بقیه جدول از صفر آغاز شده است.

Jumpstart Return آن برابر ۱۶۵.۱۹- بوده که بسیار نزدیک تر به scratch با ۱۵۴.۵۹- است و به مراتب بهتر از انتقال کامل با ۲۲۹.۸۵- محسوب می شود.

در ادامه، مدل انتقال انتخابی در اپیزود ۳۹,۱۹۶ به موفقیت ۳۰ درصد رسیده است؛ حدود ۱۷۳۷ اپیزود زودتر از scratch.

مهم تر از همه، عملکرد نهایی آن برابر است با:

$$Success_selective = 74.00\%$$

که حتی اندکی بالاتر از scratch است:

$$74.00\% - 73.33\% = 0.67\%$$

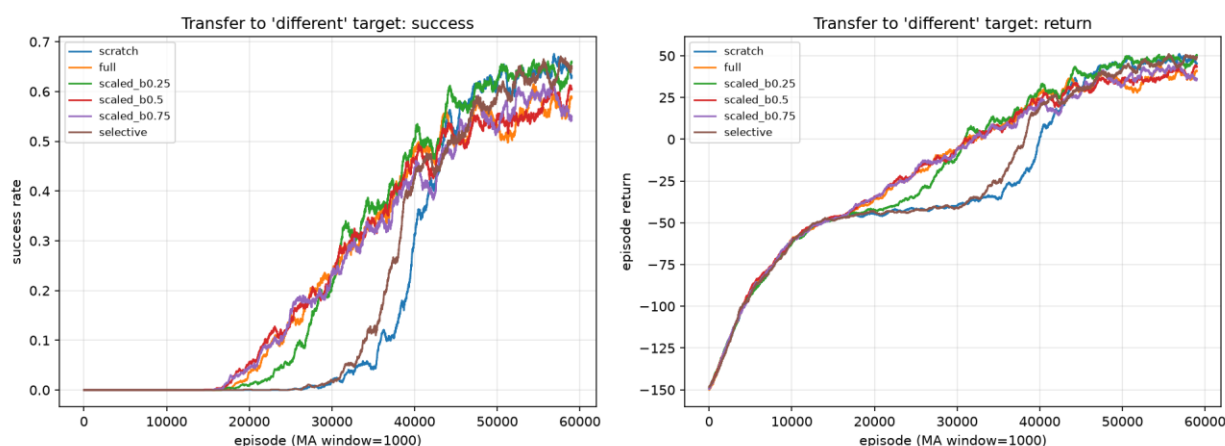
میانگین بازده نهایی نیز از 60.41 به 62.26 افزایش یافته است.

بنابراین، در محیطی با تغییرات گسترده، انتقال انتخابی بهترین تعادل را ایجاد کرده است. این روش بخش‌هایی از دانش مبدأ را که ساختار محلی آن‌ها همچنان معتبر بوده حفظ کرده و از انتقال بخش بزرگی از اطلاعات بالقوه گمراه‌کننده خودداری کرده است.

نتیجه محیط مشابه و متفاوت در کنار یکدیگر یک الگوی روشن ارائه می‌کنند:

- وقتی شباهت محیط زیاد است، انتقال گسترده سودمندتر است.
- وقتی تغییرات ساختاری زیاد است، انتقال محافظه‌کارانه و انتخابی قابل اعتمادتر است.

۹-۱۰. منحنی‌های یادگیری محیط متفاوت



شکل ۱۰-۲: منحنی نرخ موفقیت و بازده در سناریوهای انتقال یادگیری: (ب) محیط مقصد متفاوت. منحنی‌ها با میانگین متحرک ۱۰۰۰ / اپیزودی رسم شده‌اند.

مقایسه این دو نمودار تفاوت بنیادی دو محیط را نشان می‌دهد. در محیط مشابه، scratch تقریباً قادر به یادگیری مناسب در بودجه ۶۰,۰۰۰ اپیزودی نیست و دانش منتقل‌شده عامل اصلی پیشرفت است. در محیط متفاوت، scratch خود به‌خوبی یاد می‌گیرد و اثر انتقال بیشتر به شکل تغییر سرعت اولیه و جهت سازگاری مشاهده می‌شود.

۱۰-۱۰. نمونه واقعی انتقال منفی و اصلاح آن

برای نشان دادن انتقال منفی در سطح یک حالت مشخص، حالت زیر از محیط مشابه انتخاب شده است:

$$s = (2,6,0,37)$$

در جدول Q مبدأ، مقادیر اعمال عبارت بودند از:

عمل	Q در محیط مبدأ
UP	11.0429
DOWN	-1.8130
LEFT	21.3676
RIGHT	51.2454

بنابراین، سیاست مبدأ با اطمینان زیادی عمل RIGHT را انتخاب می‌کرد. اختلاف بهترین عمل با دومین عمل برابر بود با:

$$51.2454 - 21.3676 = 29.8778$$

در محیط مبدأ، حرکت از خانه (2,6) به (2,7) معتبر بود. اما در محیط مقصد مشابه، خانه (2,7) به دیوار تبدیل شده است.

در نتیجه، همان دانشی که در محیط مبدأ کاملاً مفید بود، در محیط جدید عامل را مستقیماً به یک مانع هدایت می‌کند.

پس از ادامه آموزش در محیط مقصد با سناریوی انتقال کامل، مقادیر Q این حالت به شکل زیر تغییر کردند:

عمل	Q مبدأ	Q پس از آموزش مقصد
UP	11.0429	34.8665
DOWN	-1.8130	19.4083
LEFT	21.3676	22.0055
RIGHT	51.2454	21.9472

جدول ۱۰-۴: اصلاح یک نمونه انتقال منفی پس از آموزش در مقصد

عمل حریصانه در نتیجه از RIGHT به UP تغییر کرده است.

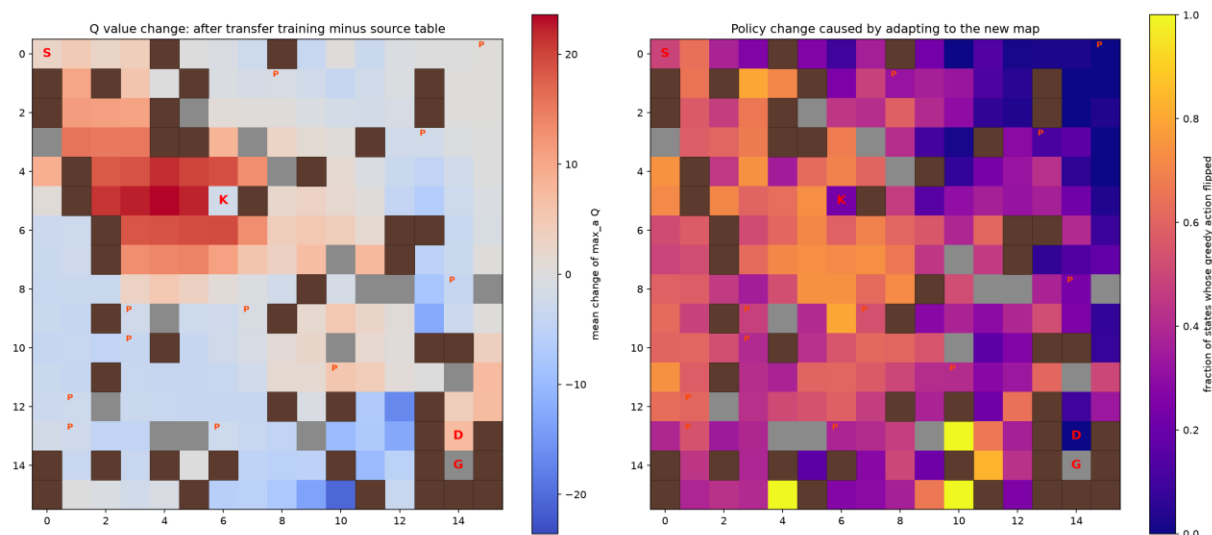
این مثال نشان می‌دهد Q-Learning می‌تواند دانش منتقل شده اشتباه را در صورت تجربه کافی محیط جدید اصلاح کند. مقدار عمل RIGHT که ابتدا با فاصله زیادی بیشترین Q را داشت، پس از برخوردهای نامطلوب ارزش خود را از دست داده و عمل UP جایگزین آن شده است.

بالین حال، در فایل تحلیل انتقال منفی نمونه‌هایی نیز وجود دارند که پس از ۶۰,۰۰۰ اپیزود همچنان اصلاح نشده‌اند. دلیل محتمل این است که عامل در محیط جدید برخی از ترکیب‌های دقیق موقعیت، وضعیت کلید و انرژی را دوباره تجربه نکرده یا تعداد بازدید آن‌ها برای تغییر مقدار Q کافی نبوده است. بنابراین، انتقال کامل می‌تواند تعدادی مقدار قدیمی و ناسازگار را برای مدت طولانی در جدول نگه دارد.

این موضوع یکی از دلایل برتری انتقال انتخابی در محیط متفاوت است: به جای انتظار برای اصلاح همه اطلاعات اشتباه، بخش‌هایی که پیرامون آن‌ها ساختار نقشه تغییر کرده است از ابتدا منتقل نمی‌شوند.

۱۱-۱۰. میزان تغییر جدول Q پس از سازگاری

برای مشاهده اینکه آموزش در محیط جدید تا چه اندازه جدول Q منتقل شده را تغییر داده است، مدل قبل و بعد از انتقال کامل به محیط متفاوت مقایسه شده است.



شکل ۱۰-۳: تغییر جدول Q پس از انتقال کامل و آموزش در محیط متفاوت. بخش چپ میانگین تغییر بیشینه Q در هر خانه و بخش راست نسبت حالت‌هایی را نشان می‌دهد که عمل حریصانه آن‌ها پس از سازگاری تغییر کرده است.

این شکل نشان می‌دهد تغییرات Q در سراسر نقشه یکنواخت نیستند. در برخی نواحی، مقدارهای انتقال یافته تقریباً بدون تغییر باقی مانده‌اند، در حالی که در نواحی تحت تأثیر جابه‌جایی موانع و مسیرهای جدید، تغییرهای بزرگ‌تری مشاهده می‌شود.

بخش سمت راست نیز نشان می‌دهد در تعدادی از خانه‌ها عمل حریصانه برای درصد قابل توجهی از سطوح انرژی تغییر کرده است. بنابراین، سازگاری با محیط مقصد صرفاً شامل اصلاح مقدار عددی Q نیست؛ ساختار خود سیاست نیز در بخش‌هایی از نقشه بازسازی شده است.

۱۰-۱۲. مقایسه انتقال مثبت و منفی در دو محیط

نتایج دو محیط نشان می‌دهند انتقال مثبت و منفی می‌توانند هم‌زمان و در مراحل مختلف یادگیری وجود داشته باشند.

در محیط مشابه، انتقال کامل هم سرعت یادگیری و هم عملکرد نهایی را به‌طور چشمگیر افزایش داد. نرخ موفقیت نهایی از ۶ درصد در scratch به ۴۶ درصد رسید. بنابراین، دانش مبدأ در مجموع کاملاً سودمند بوده است.

در محیط متفاوت، وضعیت پیچیده‌تر است. انتقال کامل در ابتدا بازده را از ۱۵۴.۵۹- به ۲۲۹.۸۵- کاهش داد که انتقال منفی آشکار است. سپس همین دانش باعث شد عامل حدود ۷۲۰۰ اپیزود زودتر به نرخ موفقیت ۳۰ درصد برسد، یعنی از نظر سرعت یادگیری اثر مثبت داشت. اما در ارزیابی نهایی، موفقیت آن هفت واحد درصد کمتر از scratch باقی ماند.

در نتیجه، انتقال کامل در محیط متفاوت دارای ترکیب زیر بوده است:

اثر آغازین: منفی → سرعت یادگیری: مثبت → عملکرد نهایی: منفی

این نتیجه یکی از مهم‌ترین یافته‌های این مرحله است؛ زیرا نشان می‌دهد انتقال دانش را نمی‌توان تنها با یک عدد توصیف کرد.

انتقال انتخابی این تعارض را تا حد زیادی کاهش داده است. با حذف حالت‌های دارای همسایگی تغییرکرده، Jumpstart بسیار کمتر آسیب دیده و در نهایت بهترین موفقیت و بازده محیط متفاوت حاصل شده است.

۱۰-۱۳. اثر شباهت محیط بر راهبرد مناسب انتقال

نتایج اجازه می‌دهند یک نتیجه عمومی‌تر درباره انتخاب روش انتقال در همین مسئله ارائه شود.

در محیط مشابه، ۶۱٫۵۷ درصد حالت‌های جدول Q شرایط انتقال انتخابی را داشتند، اما انتقال کامل عملکرد بهتری ایجاد کرد. این موضوع نشان می‌دهد فیلتر محلی بیش از حد محافظه‌کارانه بوده و بخشی از اطلاعاتی را حذف کرده است که هنوز می‌توانستند مفید باشند.

در محیط متفاوت، تنها ۲۹٫۹ درصد حالت‌ها همسایگی محلی بدون تغییر داشتند. در این حالت، انتقال کامل مقدار زیادی دانش ناسازگار وارد جدول کرد و انتقال انتخابی بهتر عمل نمود.

بنابراین، مناسب‌ترین شدت انتقال به میزان تغییر محیط وابسته است. اگر شباهت زیاد باشد، حفظ مقدار بیشتری از جدول Q می‌تواند سرعت سازگاری را افزایش دهد. با کاهش شباهت، اعتماد کامل به مقادیر قبلی خطر انتقال منفی را افزایش می‌دهد و استفاده از روش‌های انتخابی یا کاهش شدت انتقال مناسب‌تر می‌شود.

نتایج β نیز این مسئله را تأیید می‌کنند. در محیط متفاوت، کوچک‌ترین ضریب آزمایش شده یعنی:

$$\beta = 0.25$$

بهترین AUC و سریع‌ترین رسیدن به آستانه موفقیت را میان روش‌های انتقال کامل/مقیاس‌شده ایجاد کرده است. بنابراین، وقتی شباهت کمتر است، کاهش اعتماد عددی به دانش قبلی می‌تواند سازگاری را سریع‌تر کند.

۱۴-۱۰. جمع‌بندی نتایج

آزمایش انتقال یادگیری نشان داد ارزش جدول Q مبدأ به میزان شباهت ساختاری محیط مقصد وابستگی زیادی دارد.

در محیط مشابه، تنها ۱۷٫۹۵ درصد موانع جابه‌جا شده بودند و عناصر اصلی مأموریت ثابت باقی مانده بودند. در این شرایط، انتقال کامل بهترین نتیجه را ایجاد کرد و نرخ موفقیت نهایی را از ۶ درصد به ۴۶ درصد افزایش داد. مدل $\beta=0.5$ نیز تقریباً همان سرعت یادگیری را با موفقیت نهایی ۴۲٫۶۷ درصد ارائه کرد.

در محیط متفاوت، بیش از ۴۱ درصد موانع تغییر کردند، کلید جابه‌جا شد و چهار خانه جریمه جدید اضافه شدند. انتقال کامل در این شرایط Jumpstart را به‌شدت کاهش داد و عملکرد نهایی آن نیز از scratch ضعیف‌تر بود. بالاین‌حال، همین دانش باعث شد عامل زودتر به نرخ موفقیت ۳۰ درصد برسد. بنابراین، انتقال در این محیط هم‌زمان شامل آثار مثبت و منفی بوده است.

مدل $\beta=0.25$ سریع‌ترین یادگیری را در محیط متفاوت ایجاد کرد، اما بهترین نتیجه نهایی متعلق به انتقال انتخابی بود که تنها ۲۹،۹ درصد حالت‌های مبدأ را منتقل کرد و به نرخ موفقیت ۷۴ درصد رسید؛ اندکی بالاتر از آموزش از صفر با ۷۳،۳۳ درصد.

نمونه انتقال منفی نیز نشان داد یک عمل که در محیط مبدأ دارای Q بسیار بالا بود، پس از تبدیل خانه مقصد آن به دیوار کاملاً نامناسب شد. ادامه آموزش توانست این ترجیح را اصلاح کند، اما همه حالت‌های ناسازگار الزاماً در بودجه محدود آموزش دوباره مشاهده نمی‌شوند.

در مجموع، نتایج نشان می‌دهند انتقال دانش زمانی مؤثر است که علاوه بر وجود دانش قبلی، میزان اعتبار آن دانش در محیط جدید نیز در نظر گرفته شود. انتقال کامل برای محیط‌های نزدیک به مبدأ سودمند است، درحالی‌که با افزایش تغییر ساختاری، روش‌های انتخابی یا انتقال با اعتماد کاهش یافته می‌توانند خطر انتقال منفی را محدود کنند. این نتیجه مستقیماً هدف بخش انتقال پروژه، یعنی بررسی حدود استفاده از دانش قبلی در محیط‌های جدید، را نشان می‌دهد.

فصل یازدهم: رابط گرافیکی و نمایش تعاملی عامل

۱۱-۱. طراحی رابط گرافیکی

برای مشاهده مستقیم رفتار عامل و بررسی کیفی سیاست‌های آموخته‌شده، یک رابط گرافیکی مستقل با استفاده از **Tkinter** پیاده‌سازی شده است. هدف این رابط، جایگزین کردن آزمایش‌های عددی فصل‌های قبل نیست؛ بلکه فراهم کردن امکان مشاهده مرحله‌به‌مرحله تصمیم‌های عامل، تغییر وضعیت محیط و رخدادهای مهم در طول یک اپیزود است.

رابط از دو فایل اصلی تشکیل شده است:

```
gui/app.py  
gui/renderer.py
```

فایل `app.py` مسئول کنترل اجرای محیط، انتخاب الگوریتم، بارگذاری مدل‌ها، آموزش تعاملی و مدیریت اجزای رابط است. فایل `renderer.py` نیز صرفاً وظیفه نمایش بصری نقشه، عامل، رخدادها، مسیر و انرژی را بر عهده دارد و منطق یادگیری در آن قرار نگرفته است.

این تفکیک باعث شده منطق نمایش از پیاده‌سازی الگوریتم‌های یادگیری مستقل باشد و همان کلاس `MazeEnv` که در آزمایش‌های عددی استفاده شد، در رابط گرافیکی نیز به کار گرفته شود.

اجرای رابط از مسیر اصلی پروژه با دستور زیر انجام می‌شود:

```
python main.py
```

در صورت نبود فایل نقشه مبدأ، `main.py` ابتدا آن را تولید کرده و سپس رابط را اجرا می‌کند. کتابخانه `tkinter` نیز برای اجرای GUI موردنیاز است.

۱۱-۲. ساختار کلی رابط

رابط گرافیکی از دو بخش اصلی تشکیل شده است. در سمت چپ، نقشه هزارتو و اجزای پویا نمایش داده می‌شوند و در سمت راست، پنل تنظیمات، کنترل اجرای اپیزود و اطلاعات لحظه‌ای قرار گرفته است.



شکل ۱۱-۱: نمای کلی رابط گرافیکی پروژه شامل محیط هزارتو، وضعیت عامل، نوار انرژی، سیاست حریصانه، مسیر طی شده و پنل کنترل و اطلاعات لحظه‌ای.

۳-۱۱. نمایش عناصر محیط

هر نوع خانه در نقشه دارای رنگ و علامت مجزا است تا اجزای محیط در طول اجرای عامل به راحتی قابل تشخیص باشند. علاوه بر تفاوت رنگ، برای عناصر مهم از برجسب متنی نیز استفاده شده است تا وابستگی نمایش صرفاً به رنگ کاهش یابد.

کد بصری اصلی رابط به صورت زیر است:

نمایش در GUI	عنصر
زمینه روشن	خانه عادی
قهوه‌ای تیره	دیوار
نارنجی با علامت !	خانه جریمه

نقطه شروع	آبی روشن با S
کلید	زرد با K
در	بنفش با D
هدف	سبز با G
عامل بدون کلید	دایره آبی
عامل دارای کلید	دایره سبز با علامت K

این نمایش به صورت مستقیم از وضعیت فعلی محیط به روزرسانی می شود. بنابراین، GUI یک تصویر از پیش ساخته شده از مسیر عامل نیست، بلکه در هر گام حالت واقعی MazeEnv را نمایش می دهد.

۴-۱۱. نمایش پویای وضعیت کلید و در

یکی از مهم ترین تغییرات حالت در محیط، دریافت کلید است. رابط این رخداد را فقط در پنل اطلاعات گزارش نمی کند، بلکه ساختار بصری نقشه را نیز تغییر می دهد.

تا پیش از دریافت کلید، خانه کلید با K و در با D نمایش داده می شود. به محض اجرای رویداد KEY_PICKUP:

- علامت کلید از خانه مربوط حذف می شود؛
- رنگ عامل تغییر می کند؛
- حرف K روی عامل قرار می گیرد؛
- در از حالت بسته به حالت open تغییر می کند.

در نتیجه، تغییر متغیر حالت k به صورت مستقیم برای کاربر قابل مشاهده است و مشخص می شود که سیاست عامل پس از این لحظه وارد مرحله دوم مأموریت شده است.

۵-۱۱. نمایش محدودیت انرژی

از آنجا که انرژی قابلیت تکمیلی انتخاب شده برای محیط است، وضعیت آن نیز به صورت بصری در رابط نمایش داده شده است. در پایین نقشه یک Energy Bar قرار دارد که هم مقدار عددی انرژی و هم نسبت باقی مانده آن را نمایش می دهد.

در ابتدای اپیزود:

energy = 50 / 50

است و با هر عمل، طول نوار کاهش می‌یابد. رنگ نوار نیز با کاهش انرژی تغییر می‌کند:

- برای بیش از ۵۰ درصد انرژی: سبز؛
- بین ۲۰ تا ۵۰ درصد: نارنجی؛
- کمتر از ۲۰ درصد: قرمز.

به این ترتیب، کاربر علاوه بر عدد دقیق، فوریت محدودیت انرژی را نیز به صورت بصری مشاهده می‌کند. این قابلیت به‌ویژه هنگام مقایسه مسیرهای کوتاه و طولانی سیاست‌ها مفید است.

۶-۱۱. نمایش رخدادهای محیط

تمام رخدادهای اصلی محیط هنگام وقوع در رابط قابل مشاهده هستند. برای افزایش وضوح، خانه مرتبط برای مدت کوتاهی با رنگ مخصوص رخداد برجسته می‌شود و نام رخداد نیز در زیر نقشه نمایش داده می‌شود.

از جمله رخدادهای پشتیبانی‌شده می‌توان به موارد زیر اشاره کرد:

- WALL_HIT
- PENALTY_CELL
- KEY_PICKUP
- DOOR_LOCKED_ATTEMPT
- DOOR_PASS
- GOAL_REACHED
- ENERGY_DEPLETED
- STEP_LIMIT

برای مثال، برخورد با دیوار با Flash قرمز و ورود به خانه جریمه با رنگ نارنجی مشخص می‌شود. دریافت کلید با رنگ زرد و عبور از در با بنفش نمایش داده می‌شود.

در پایان اپیزود نیز یک پیام بزرگ روی نقشه ظاهر می‌شود. در صورت رسیدن به هدف، پیام سبز:

SUCCESS - goal reached

و در صورت شکست، یک پیام قرمز همراه با علت پایان اپیزود نمایش داده می‌شود.

این طراحی امکان مشاهده دلیل شکست عامل را بدون مراجعه هم‌زمان به فایل‌های CSV فراهم می‌کند.

۷-۱۱. انتخاب الگوریتم و محیط

در پنل Setup سه الگوریتم قابل انتخاب هستند:

- Value Iteration
- Q-Learning
- SARSA (lambda)

همچنین سه محیط در اختیار کاربر قرار دارد:

- source
- target (similar)
- target (different)

با تغییر محیط، فایل JSON متناظر بارگذاری و نقشه دوباره ترسیم می‌شود. بنابراین، تفاوت میان نقشه‌های انتقال یادگیری که در فصل دهم به صورت ثابت مشاهده شد، در GUI نیز به طور تعاملی قابل بررسی است.

برای Q-Learning، مدل‌های ذخیره‌شده محیط مبدأ و هر دو مقصد در حالت ارزیابی قابل بارگذاری هستند. مدل اصلی SARSA برای محیط مبدأ در نظر گرفته شده است. در Value Iteration نیز اگر مدل ذخیره‌شده‌ای برای محیط انتخابی وجود نداشته باشد، سیاست از روی مدل همان محیط محاسبه می‌شود.

۸-۱۱. حالت‌های Evaluation و Training

رابط دو حالت اجرای متفاوت دارد:

Evaluation

در حالت evaluation، یک مدل آموزش‌دیده از پوشه:

results/models/

بارگذاری می‌شود و عامل با سیاست حریصانه اجرا می‌گردد: $\epsilon=0$

این حالت برای مشاهده و مقایسه سیاست‌های نهایی مناسب است. تصادفی بودن خود محیط همچنان فعال باقی می‌ماند؛ بنابراین، حتی در حالت ارزیابی ممکن است عمل مؤثر بر اثر لغزش با عمل انتخاب‌شده متفاوت باشد.

Training

در حالت training، برای Q-Learning و SARSA یک جدول Q جدید ایجاد می‌شود و عامل در هنگام نمایش واقعاً یاد می‌گیرد. به عبارت دیگر، هر گام نمایش داده شده شامل همان به‌روزرسانی Q یا $SARSA(\lambda)$ است که در پیاده‌سازی اصلی استفاده شده است.

این امکان باعث می‌شود تغییر تدریجی رفتار عامل در حین یادگیری قابل مشاهده باشد.

بالین حال، نتایج عددی فصل‌های هفتم تا دهم از اسکرپت‌های آزمایشی اصلی تولید شده‌اند و نه از آموزش تعاملی GUI. حالت Training رابط با هدف نمایش فرایند یادگیری طراحی شده است و جایگزین آزمایش کنترل شده و بازتولیدپذیر نیست.

۹-۱۱. آموزش سریع یا Fast-forward

مشاهده مرحله به مرحله ۱۵۰۰۰۰ اپیزود از طریق انیمیشن عملی نیست. به همین دلیل، دکمه‌ای با عنوان زیر در GUI قرار داده شده است:

Fast-forward 10000 episodes

با انتخاب آن، عامل ۱۰۰۰۰۰ اپیزود بدون رندرکردن تک تک گام‌ها آموزش می‌بیند و سپس کاربر می‌تواند دوباره اجرای بصری را ادامه دهد.

برنامه کاهش ϵ در GUI به صورت تجمعی مدیریت می‌شود؛ یعنی با هر بار استفاده از Fast-forward از نقطه قبلی ادامه پیدا می‌کند و از $\epsilon=1$ آغاز نمی‌شود. در تنظیمات رابط، پس از مجموع ۱۰۰۰۰۰۰ اپیزود آموزش، مقدار ϵ از ۱ به حداقل ۰.۰۰۵ می‌رسد.

این افق برای استفاده تعاملی انتخاب شده است. در مقابل، آزمایش اصلی Q-Learning از افق کاهش ۱۲۰۰۰۰۰ اپیزودی استفاده می‌کرد. بنابراین، همان‌طور که اشاره شد، Training در GUI بیشتر یک ابزار نمایشی و آزمایشی است و نباید نتایج آن با آزمایش‌های رسمی گزارش مخلوط شود.

در مورد Value Iteration، گزینه Fast-forward کاربرد ندارد؛ زیرا این روش آموزش اپیزودی ندارد.

۱۰-۱۱. کنترل اجرای اپیزود

رابط امکان کنترل کامل پخش را از طریق دکمه‌های زیر فراهم می‌کند:

Start •

- Pause
- Resume
- Reset
- Re-run episode

Reset عامل و اپیزود را به وضعیت اولیه بازمی گرداند. گزینه Re-run episode اهمیت تحلیلی بیشتری دارد، زیرا همان seed اپیزود فعلی را دوباره استفاده می کند.

Seed هر اپیزود در GUI به صورت قطعی از شماره اپیزود ساخته می شود. بنابراین، هنگام Re-run، همان توالی تصادفی محیط مجدداً ایجاد می شود. این قابلیت به کاربر اجازه می دهد رفتار سیاست را در یک تحقق تصادفی مشخص چند بار مشاهده کند.

سرعت انیمیشن نیز با یک Slider از ۱ تا ۱۲۰ گام بر ثانیه قابل تنظیم است. در نتیجه، می توان رویدادهای حساس را با سرعت پایین بررسی کرد یا یک اپیزود را با سرعت زیاد مشاهده نمود.

۱۱-۱۱. نمایش مسیر طی شده عامل

گزینه show agent trail مسیر حرکت عامل را روی نقشه نگه می دارد. پس از هر جابه جایی موفق، یک خط آبی نیمه شفاف میان موقعیت قبلی و جدید رسم می شود.

اگر عامل به دلیل برخورد با دیوار یا در بسته در همان خانه باقی بماند، مسیر جدیدی رسم نمی شود. بنابراین، Trail فقط جابه جایی واقعی عامل را نشان می دهد.

این قابلیت برای بررسی کیفی موضوعاتی مانند موارد زیر مفید است:

- وجود دورزدن های غیرضروری؛
- طول تقریبی مسیر؛
- مسیر پیش از و پس از دریافت کلید؛
- میزان انحراف عامل بر اثر لغزش؛
- تفاوت مسیر سیاست های مختلف.

تحلیل کمی مسیرها در فصل نهم انجام شده است؛ بنابراین، در این فصل هدف فقط توضیح قابلیت نمایش آن در GUI است.

۱۱-۱۲. نمایش سیاست حریصانه

یکی از قابلیت‌های مهم رابط، گزینه:

show policy arrows

است. با فعال کردن آن، عمل حریمانه به صورت فلش روی خانه‌های قابل عبور نمایش داده می‌شود.

از آنجا که سیاست به وضعیت کلید وابسته است، رابط در کنار فلش‌ها مشخص می‌کند کدام لایه سیاست در حال نمایش است. برای مثال، پیش از کلید:

key=0 : no key -> find key

و پس از دریافت کلید:

key=1 : with key -> door/goal

به محض دریافت کلید، فلش‌ها نیز به صورت خودکار به لایه جدید تغییر می‌کنند.

این موضوع اهمیت دارد، زیرا نمایش فلش‌های $k=0$ پس از دریافت کلید می‌توانست برداشت نادرستی از تصمیم عامل ایجاد کند.

۱۱-۱۳. نحوه نمایش سیاست Value Iteration و روش‌های مدل آزاد

به دلیل وجود انرژی در حالت، نمایش یک سیاست چهاربعدی روی نقشه دوبعدی نیازمند انتخاب یک برش مناسب است.

برای **Value Iteration** این مسئله ساده‌تر است؛ زیرا سیاست برای تمام حالت‌های فضای تعریف شده موجود است. بنابراین، GUI از مقدار فعلی کلید و انرژی استفاده می‌کند و سیاست دقیق همان لایه را نمایش می‌دهد:

$$(r, c, k_{current}, e_{current})$$

اما Q-Learning و SARSA فقط حالت‌هایی را در جدول Q دارند که در طول آموزش واقعاً مشاهده کرده‌اند. انتخاب یک انرژی ثابت می‌تواند باعث شود بخش بزرگی از خانه‌ها هیچ مقدار Q متناظری نداشته باشند.

برای حل این مشکل، در هر خانه و برای وضعیت فعلی کلید، GUI انرژی‌ای را انتخاب می‌کند که آن خانه در آن بیشترین تعداد بازدید را داشته است. سپس عمل حریمانه همان حالت نمایش داده می‌شود.

بنابراین، فلش مدل‌های یادگیری‌شده نشان‌دهنده رفتار معمول آموخته‌شده در آن موقعیت است و الزاماً متعلق به یک مقدار انرژی یکسان در تمام خانه‌های نقشه نیست.

این تفاوت در نحوه نمایش از ایجاد یک نقشه تقریباً خالی برای الگوریتم‌های مدل‌آزاد جلوگیری می‌کند.

۱۴-۱۱. اطلاعات لحظه‌ای

در سمت راست رابط، بخش Live status اطلاعات جاری اپیزود را نمایش می‌دهد. متغیرهای موجود عبارت‌اند از:

متغیر	مفهوم
episode	شماره اپیزود جاری
episodes trained	تعداد اپیزودهای واقعی آموزش‌دیده
step	شماره گام در اپیزود
episode reward	پاداش تجمعی اپیزود
epsilon	مقدار فعلی ϵ
key	وضعیت دریافت کلید
energy	انرژی باقی‌مانده
recent success (100 ep)	نرخ موفقیت ۱۰۰ اپیزود اخیر
states learned	تعداد حالت‌های موجود در جدول Q

این اطلاعات باعث می‌شوند تغییر رفتار عامل را بتوان هم‌زمان با اجرای بصری آن دنبال کرد. برای مثال، در حالت Training می‌توان مشاهده کرد که با افزایش تعداد اپیزودهای آموزش‌دیده و کاهش ϵ ، تعداد حالت‌های یادگرفته‌شده و نرخ موفقیت اخیر چگونه تغییر می‌کنند.

برای Value Iteration، states learned نمایش داده نمی‌شود؛ زیرا مفهوم مشاهده تدریجی حالت‌ها در این الگوریتم وجود ندارد.

۱۵-۱۱. اجرای Value Iteration در رابط

Value Iteration برخلاف دو روش دیگر دارای اپیزود آموزشی نیست. بنابراین، انتخاب training برای این الگوریتم به معنای اجرای آموزش مرحله‌به‌مرحله مانند Q-Learning نیست.

اگر یک مدل از پیش ذخیره شده برای ترکیب انتخابی موجود باشد، سیاست مستقیماً بارگذاری می شود. در غیر این صورت، Value Iteration برای همان محیط محاسبه می گردد.

از آنجا که این محاسبه ممکن است چند ثانیه طول بکشد، اجرای Value Iteration در یک Thread جداگانه انجام می شود. رابط اصلی به صورت دوره ای پایان محاسبات را بررسی می کند و پس از آماده شدن سیاست، آن را نمایش می دهد.

این تصمیم اجرایی مانع از آن می شود که پنجره Tkinter در زمان محاسبه Value Iteration بدون پاسخ به نظر برسد.

۱۶-۱۱. هماهنگی GUI با منطق اصلی پروژه

یکی از نکات مهم طراحی این است که GUI یک شبیه سازی جداگانه از محیط ایجاد نکرده است. همان کلاس های اصلی پروژه مستقیماً در آن استفاده می شوند:

```
MazeEnv
QLearningAgent
SarsaLambdaAgent
ValueIterationAgent
```

بنابراین، احتمال لغزش، منطق کلید و در، کاهش انرژی، تابع پاداش و شرایط پایانی در GUI از همان پیاده سازی مورد استفاده در آزمایش های اصلی پیروی می کنند.

برای Q-Learning در حالت Training، همان تابع update عامل فراخوانی می شود. برای SARSA نیز رابطه به روزرسانی و replacing eligibility trace مستقیماً مطابق کلاس اصلی در حلقه انیمیشن اجرا شده است.

فایل renderer.py هیچ تصمیم یادگیری یا انتخاب عملی انجام نمی دهد و تنها نتیجه حالت و رخداد دریافت شده را روی Canvas نمایش می دهد. این جداسازی احتمال ایجاد تفاوت ناخواسته میان «محیط واقعی پروژه» و «محیطی که در GUI دیده می شود» را کاهش می دهد.

۱۷-۱۱. انطباق با الزامات رابط گرافیکی پروژه

در تعریف پروژه تأکید شده است که رابط نباید صرفاً یک تصویر ثابت باشد و باید امکان مشاهده مرحله به مرحله حرکت عامل و تغییرات کلید، در، موانع، خانه های جریمه و پایان اپیزود را فراهم کند. همچنین، انتخاب الگوریتم

و محیط، حالت آموزش و ارزیابی، کنترل اجرای اپیزود، سرعت انیمیشن، نمایش سیاست و اطلاعات لحظه‌ای از قابلیت‌های موردنیاز هستند.

پیاده‌سازی فعلی این الزامات را پوشش می‌دهد. خلاصه تطابق در جدول ۱-۱۱ ارائه شده است.

وضعیت پیاده‌سازی	قابلیت موردنیاز
پیاده‌سازی شده	نمایش مرحله به مرحله حرکت عامل
پیاده‌سازی شده	نمایش متمایز دیوار، جریمه، کلید، در و هدف
پیاده‌سازی شده	نمایش تغییر وضعیت کلید و در
پیاده‌سازی شده	نمایش انرژی
پیاده‌سازی شده	انتخاب الگوریتم
پیاده‌سازی شده	انتخاب محیط
پیاده‌سازی شده	Training / Evaluation
پیاده‌سازی شده	Start / Pause / Resume / Reset
پیاده‌سازی شده	اجرای مجدد اپیزود
پیاده‌سازی شده	کنترل سرعت
پیاده‌سازی شده	نمایش سیاست حریصانه
پیاده‌سازی شده	نمایش مسیر عامل
پیاده‌سازی شده	نمایش پاداش و ع
پیاده‌سازی شده	نمایش وضعیت کلید و انرژی
پیاده‌سازی شده	نرخ موفقیت اخیر
پیاده‌سازی شده	نمایش موفقیت و شکست

جدول ۱-۱۱: قابلیت‌های رابط گرافیکی پیاده‌سازی شده

فصل دوازدهم: پاسخ به پرسش‌های تحلیلی

۱۲-۱. تعریف مسئله به صورت MDP و حفظ خاصیت مارکوف

مسئله این پروژه را می‌توان به صورت یک فرایند تصمیم‌گیری مارکوف به شکل زیر نمایش داد:

$$M = (S, A, P, R, \gamma, S_T)$$

حالت عامل به صورت زیر تعریف شده است:

$$s_t = (r_t, c_t, k_t, e_t)$$

که در آن r_t و c_t موقعیت عامل، k_t وضعیت داشتن کلید و e_t میزان انرژی باقی‌مانده را نشان می‌دهند. متغیر کلید دارای دو مقدار صفر و یک است:

$$k_t \in \{0,1\}$$

و حالت اولیه برابر است با:

$$s_0 = (0,0,0,50)$$

فضای عمل نیز شامل چهار جهت حرکت است:

$$A = \{UP, DOWN, LEFT, RIGHT\}$$

انتقال‌ها تصادفی هستند. عمل انتخابی با احتمال ۰٫۸ در جهت موردنظر و با احتمال ۰٫۱ در هر یک از دو جهت عمود بر آن اجرا می‌شود:

$$P(a_{effective} = a_{intended}) = 0.8$$

$$P(a_{effective} = a_{perp1}) = 0.1$$

$$P(a_{effective} = a_{perp2}) = 0.1$$

تابع پاداش مرجع نیز براساس رویدادهای محیط تعریف شده است:

$$r = -1 - 4I_{wall} - 4I_{locked} - 9I_{penalty} + 50I_{key} + 100I_{goal} - 20I_{energy}$$

و در آزمایش‌های اصلی ضریب تنزیل برابر است با:

$$\gamma = 0.99$$

وجود هر چهار مؤلفه (r, c, k, e) برای حفظ خاصیت مارکوف ضروری است. برای مثال، دانستن موقعیت عامل به‌تنهایی برای پیش‌بینی نتیجه ورود به خانه در کافی نیست؛ زیرا باید مشخص باشد عامل کلید را قبلاً دریافت کرده است یا خیر. همچنین، پایان اپیزود به میزان انرژی باقی‌مانده وابسته است. بنابراین، دو عامل در یک مختصات یکسان ولی با انرژی یا وضعیت کلید متفاوت، الزاماً توزیع انتقال یکسانی ندارند.

با در نظر گرفتن این متغیرها، حالت فعلی تمام اطلاعات لازم برای تعیین توزیع حالت بعدی و پاداش را در خود دارد:

$$\begin{aligned} P(s_{(t+1)}, r_{(t+1)} | s_t, a_t, history) \\ = \\ P(s_{(t+1)}, r_{(t+1)} | s_t, a_t) \end{aligned}$$

نقشه نیز در طول هر اپیزود ثابت است و در نتیجه لازم نیست به‌عنوان بخشی از بردار حالت ذخیره شود.

سقف $\max_steps$ برابر ۶۳۰ است، اما انرژی اولیه تنها ۵۰ واحد است؛ در نتیجه، در تنظیم فعلی عامل همیشه پیش از فعال‌شدن محدودیت ۶۳۰ گام بر اثر پایان انرژی متوقف می‌شود. به همین دلیل، شماره گام برای حفظ خاصیت مارکوف در شرایط فعلی موردنیاز نیست. اگر محدودیت زمانی می‌توانست زودتر از انرژی اپیزود را پایان دهد، لازم بود زمان باقی‌مانده نیز در حالت قرار گیرد.

۲-۱۲. تفاوت **on-policy** و **off-policy** در رفتار واقعی **SARSA** و **Q-Learning**

Q-Learning در این پروژه یک الگوریتم **off-policy** است. هدف به‌روزرسانی آن از بهترین عمل ممکن در حالت بعدی استفاده می‌کند:

$$Target_Q L = r + \gamma \max_{(a')} Q(s', a')$$

بنابراین، حتی اگر عامل هنگام آموزش از سیاست ϵ -greedy استفاده کند، مقدار **Q** براساس فرض انتخاب بهترین عمل در آینده به‌روزرسانی می‌شود.

در مقابل، SARSA یک روش **on-policy** است و از عملی استفاده می‌کند که سیاست رفتاری عامل واقعاً برای گام بعد انتخاب کرده است:

$$Target_{SARSA} = r + \gamma Q(s', a')$$

در نتیجه، احتمال انتخاب یک عمل اکتشافی نامطلوب نیز در فرآیند یادگیری SARSA منعکس می‌شود. یک نمونه واقعی در نتایج پروژه، حالت زیر است:

$$s = (9, 14, 1, 6)$$

در این موقعیت، خانه (8,14) یک خانه جریمه است. سیاست‌های نهایی اعمال زیر را انتخاب می‌کنند:

عمل	روش
LEFT	Value Iteration
LEFT	Q-Learning
DOWN	SARSA($\lambda=0.3$)

اگر عامل LEFT را انتخاب کند، به دلیل احتمال لغزش، با احتمال ۰٫۱ ممکن است به سمت بالا منحرف شده و وارد خانه جریمه شود:

$$P(\text{penalty}|\text{LEFT}) = 0.1$$

اما برای عمل DOWN انتخاب‌شده توسط SARSA، در این حالت احتمال ورود مستقیم ناشی از لغزش به همان خانه جریمه صفر است:

$$P(\text{penalty}|\text{DOWN}) = 0$$

بنابراین، در این نمونه واقعی، SARSA رفتار محافظه‌کارانه‌تری در نزدیکی خانه خطرناک آموخته است. این رفتار با on-policy بودن آن سازگار است، زیرا خطر ناشی از اکتشاف نیز هنگام آموزش وارد ارزش‌گذاری اعمال می‌شود.

با این حال، داده‌های کل ارزیابی نشان می‌دهند که نمی‌توان نتیجه گرفت SARSA همیشه تعداد خانه‌های جریمه را حداقل می‌کند. میانگین ورود به خانه جریمه در هر اپیزود برای Q-Learning حدود ۰٫۱۵۷ و برای SARSA($\lambda=0.3$) حدود ۰٫۲۲۳ بوده است. در مقابل، نرخ موفقیت SARSA بسیار بالاتر بوده است:

$$Q - Learning = 64.67\%$$

$$SARSA(\lambda = 0.3) = 85.33\%$$

در نتیجه، اثر on-policy بودن در این پروژه بیشتر به شکل در نظر گرفتن ریسک واقعی سیاست رفتاری در تصمیم‌گیری ظاهر شده است، نه صرفاً کمینه کردن تعداد برخورد با خانه‌های خطرناک.

۱۲-۳. علت نیاز Value Iteration به مدل انتقال و یادگیری بدون مدل در SARSA و Q-Learning

Value Iteration در هر به‌روزرسانی باید مقدار مورد انتظار تمام حالت‌های بعدی ممکن را محاسبه کند:

$$V_{(k+1)}(s) = \max_a \sum (s') P(s'|s, a) [r + \gamma V_k(s')]$$

بنابراین، این روش باید از قبل بداند که برای هر حالت و عمل، چه حالت‌های بعدی‌ای ممکن هستند و احتمال وقوع هر کدام چقدر است.

در پیاده‌سازی پروژه، این اطلاعات از تابع زیر دریافت می‌شود:

```
env.transitions(s, a)
```

این تابع تمام شاخه‌های احتمالی مربوط به احتمال‌های ۰٫۸، ۰٫۱ و ۰٫۱ را در اختیار Value Iteration قرار می‌دهد.

Q-Learning و SARSA به این اطلاعات دسترسی ندارند و فقط با محیط تعامل می‌کنند:

```
env.step(a)
```

در هر بار فراخوانی، فقط یک نتیجه واقعی از انتقال تصادفی مشاهده می‌شود و الگوریتم باید از تکرار این تجربه‌ها مقادیر Q را تخمین بزند.

مزیت اصلی Value Iteration در این پروژه دقت بالاتر و مستقل بودن فرآیند یادگیری از نمونه‌برداری تصادفی است. مدل $\gamma=0.99$ پس از ۵۱ sweep به همگرایی رسید و در ارزیابی اصلی نرخ موفقیت ۹۳ درصد ایجاد کرد.

در مقابل، Q-Learning و SARSA برای یادگیری به تعداد زیادی تجربه نیاز داشتند. Q-Learning حدود ۷،۴ میلیون و SARSA حدود ۷،۳ میلیون انتقال محیط را طی ۱۵۰،۰۰۰ اپیزود تجربه کردند.

محدودیت Value Iteration آن است که تمام مدل محیط باید شناخته شده باشد و تمام ۲۰،۸۵۰ حالت تعریف شده پردازش شوند. با بزرگ یا پیوسته شدن فضای حالت، چنین روشی به سرعت پرهزینه می شود.

مزیت روش های مدل آزاد آن است که بدون دانستن احتمال انتقال قابل استفاده هستند و فقط از تجربه یاد می گیرند. این ویژگی در مرحله انتقال یادگیری نیز اهمیت داشت؛ جدول Q محیط مبدأ مستقیماً در اختیار عامل محیط مقصد قرار گرفت و عامل با تجربه جدید توانست برخی اطلاعات قدیمی را اصلاح کند.

در نتیجه، در این پروژه Value Iteration در صورت وجود مدل کامل بهترین سیاست مرجع را فراهم کرد، در حالی که Q-Learning و SARSA انعطاف بیشتری در شرایط بدون مدل دارند، اما این مزیت را با هزینه نمونه بیشتر و حساسیت بالاتر به فرآیند آموزش به دست می آورند.

۴-۱۲. بهترین مقدار λ از نظر سرعت یادگیری و پایداری

برای پاسخ به این سؤال باید نتایج چند اجرای مستقل در نظر گرفته شوند، زیرا نتیجه یک seed به تنهایی معیار مناسبی برای پایداری نیست.

در اجرای اصلی با seed برابر ۹:

$$\text{Success}_{(\lambda=0.3)} = 85.33\%$$

و:

$$\text{Success}_{(\lambda=0.7)} = 85.00\%$$

بود. بنابراین، در همان اجرای اولیه $\lambda=0.3$ اندکی بهتر عمل کرد.

اما در آزمایش پنج seed، نتایج زیر حاصل شدند:

λ	میانگین موفقیت	انحراف معیار	اپیزود رسیدن به ۳۰٪	زمان متوسط
0	80.47%	0.93%	77,813	22.8 s
0.3	84.87%	2.81%	69,687	31.6 s

47.2 s	67,489	2.00%	86.67%	0.7
55.3 s	نرسید	0%	0%	0.9

از نظر تعداد اپیزود موردنیاز برای یادگیری و پایداری آماری، $\lambda=0.7$ بهترین نتیجه را داشته است. این مقدار سریع‌تر از سایر مدل‌های موفق به نرخ ۳۰ درصد رسیده، بالاترین میانگین موفقیت را کسب کرده و انحراف معیار آن نیز کمتر از $\lambda=0.3$ بوده است.

باین‌حال، هزینه محاسباتی آن بیشتر است. در $\lambda=0.7$ تا Eligibility Trace ۲۶ می‌توانند هم‌زمان فعال باشند و زمان آموزش متوسط آن حدود ۴۷٫۲ ثانیه بوده است؛ درحالی‌که برای $\lambda=0.3$ این زمان حدود ۳۱٫۶ ثانیه است.

در نتیجه، اگر عبارت «تعادل میان سرعت یادگیری و پایداری» را دقیقاً برحسب اپیزود و واریانس بین اجراها تفسیر کنیم، پاسخ $\lambda=0.7$ است. اما اگر زمان محاسباتی نیز به تعادل اضافه شود، $\lambda=0.3$ گزینه اقتصادی‌تری است و به همین دلیل به‌عنوان مدل اصلی SARSA در تحلیل‌های پروژه انتخاب شد.

مقدار $\lambda=0.9$ نیز در همه اجراها نرخ موفقیت صفر داشت و نشان داد Eligibility Trace بسیار طولانی در این مسئله باعث ناپایداری شدید یادگیری می‌شود.

۵-۱۲. سه حالت دارای اختلاف میان سیاست مدل آزاد و Value Iteration

حالت اول: اختلاف Q-Learning در حالت شروع

حالت:

$$s_1 = (0,0,0,50)$$

اعمال انتخابی:

عمل	روش
RIGHT	Value Iteration
DOWN	Q-Learning
RIGHT	SARSA(0.3)

مقادیر Value Iteration برای دو عمل اصلی:

$$Q_{VI}(RIGHT) = 64.718$$

$$Q_{VI}(DOWN) = 59.892$$

اما Q-Learning مقادیر زیر را آموخته است:

$$Q_{QL}(RIGHT) = 30.611$$

$$Q_{QL}(DOWN) = 32.595$$

بنابراین، Q-Learning رتبه‌بندی این دو عمل را برعکس Value Iteration تخمین زده است.

این اختلاف ناشی از کمبود بازدید نیست، زیرا حالت شروع در تمام اپیزودها مشاهده می‌شود. علت محتمل‌تر باقی‌ماندن خطای تخمین ناشی از نمونه‌برداری تصادفی، نرخ یادگیری ثابت و ماهیت off-policy الگوریتم است.

از آنجا که این اختلاف در اولین تصمیم هر اپیزود رخ می‌دهد، می‌تواند مسیر کلی عامل را نیز طولانی‌تر کند.

حالت دوم: اختلاف SARSA میان دو تصمیم تقریباً هم‌ارزش

حالت:

$$s_2 = (1,0,0,49)$$

اعمال انتخابی:

عمل	روش
RIGHT	Value Iteration
RIGHT	Q-Learning
UP	SARSA(0.3)

مقادیر Value Iteration :

$$Q_{VI}(RIGHT) = 60.931$$

$$Q_{VI}(UP) = 60.206$$

$$Q_{VI}(DOWN) = 60.209$$

اختلاف ارزش میان بهترین عمل و سایر گزینه‌ها کمتر از یک واحد است. بنابراین، اگر SARSA عمل UP را به جای RIGHT انتخاب کند، معیار policy agreement آن را یک اختلاف کامل حساب می‌کند، درحالی‌که از نظر بازده مورد انتظار دو تصمیم بسیار نزدیک هستند.

این نمونه نشان می‌دهد که پایین بودن درصد توافق خام سیاست الزاماً به معنای اشتباه بودن شدید همان درصد از تصمیم‌ها نیست.

حالت سوم: اختلاف مهم SARSA در نزدیکی هدف

حالت:

$$s_3 = (13,14,1,3)$$

در این حالت عامل کلید را دارد، روی در قرار گرفته و فقط سه واحد انرژی باقی مانده است.

عمل	روش
DOWN	Value Iteration
DOWN	Q-Learning
RIGHT	SARSA(0.3)

مقادیر Value Iteration :

$$Q_{VI}(DOWN) = 96.593$$

$$Q_{VI}(RIGHT) = 84.673$$

بنابراین، در این حالت عمل DOWN به وضوح بهتر است و می‌تواند عامل را مستقیماً به هدف برساند.

اما SARSA مقادیر زیر را آموخته است:

$$Q_{SARSA}(DOWN) = 74.243$$

$$Q_{SARSA}(RIGHT) = 89.167$$

در نتیجه، عمل RIGHT را بیش‌برآورد کرده است.

یکی از دلایل محتمل این اختلاف، تصادفی بودن انتقال‌هاست. حتی هنگام انتخاب RIGHT ممکن است عامل در اثر لغزش به جهت دیگری منتقل شود و گاهی پاداش مناسبی دریافت کند. Value Iteration تمام این

حالات را با احتمال دقیق آن‌ها وزن می‌دهد، اما SARSA مقادیر را از نمونه‌های محدود تجربی تخمین می‌زند و ممکن است در بعضی حالت‌های کم‌تکرار رتبه‌بندی نادرستی ایجاد شود.

این سه حالت نشان می‌دهند که اختلاف سیاست‌ها ممکن است ناشی از عوامل متفاوتی باشد: خطای تخمین در یک حالت پرتکرار، اختلاف میان اعمال تقریباً هم‌ارزش یا بیش‌برآورد یک عمل در یک حالت حساس.

۶-۱۲. تفاوت انتقال یادگیری در محیط مشابه و محیط متفاوت

دو محیط مقصد از نظر میزان تغییر نسبت به مبدأ تفاوت قابل‌توجهی داشتند. در محیط مشابه حدود ۱۷,۹۵ درصد موانع جابه‌جا شدند و محل کلید تغییر نکرد. در محیط متفاوت حدود ۴۱,۰۳ درصد موانع تغییر کردند، کلید جابه‌جا شد و چهار خانه جریمه جدید نیز اضافه شدند.

مقایسه اصلی نتایج به‌صورت زیر است:

معیار	محیط مشابه	محیط متفاوت
تغییر موانع	17.95%	41.03%
جابه‌جایی کلید	خیر	بله
Scratch Jumpstart Return	-129.92	-154.59
Full Transfer Jumpstart Return	-49.19	-229.85
Scratch: رسیدن به ۳۰%	نرسید	40,933
Full Transfer: رسیدن به ۳۰%	46,624	33,699
Scratch: موفقیت نهایی	6.00%	73.33%
Full Transfer: موفقیت نهایی	46.00%	66.33%
Selective: موفقیت نهایی	41.33%	74.00%

عملکرد اولیه

در محیط مشابه، انتقال کامل باعث شد بازده اولیه از:

-129.92

به:

-49.19

بهبود یابد. بنابراین، دانش مبدأ از همان ابتدای ورود به محیط جدید مفید بوده است، هرچند هنوز نرخ موفقیت اولیه صفر بود.

اما در محیط متفاوت، انتقال کامل بازده را از:

-154.59

در آموزش از صفر به:

-229.85

کاهش داد. بنابراین، در این محیط انتقال کامل در ابتدا انتقال منفی ایجاد کرده است.

سرعت یادگیری

در محیط مشابه، عامل Scratch طی کل ۶۰,۰۰۰ اپیزود حتی به نرخ موفقیت متحرک ۳۰ درصد نرسید، ولی Full Transfer در اپیزود ۴۶,۶۲۴ به این سطح رسید. بنابراین، انتقال دانش سرعت یادگیری را به طور چشمگیری افزایش داد.

در محیط متفاوت، Scratch در اپیزود:

40933

به آستانه ۳۰ درصد رسید، درحالی که Full Transfer در:

33699

و انتقال با $\beta=0.25$ در:

32149

به همان سطح رسیدند.

در نتیجه، حتی با وجود Jumpstart منفی، دانش منتقل شده باعث افزایش سرعت سازگاری شد.

عملکرد نهایی

در محیط مشابه:

Scratch=6.00%

و:

Full Transfer=46.00%

بود. بنابراین، انتقال کامل هم در سرعت یادگیری و هم در عملکرد نهایی کاملاً مفید بود.

در محیط متفاوت:

Scratch=73.33%

اما:

Full Transfer=66.33%

شد. یعنی بخشی از اثر دانش نامناسب مبدأ حتی پس از ۶۰,۰۰۰ اپیزود باقی مانده است.

بهترین نتیجه در این محیط متعلق به انتقال انتخابی بود:

Selective Transfer=74.00%

در انتقال انتخابی فقط حالت‌هایی منتقل شدند که ساختار محلی اطراف آن‌ها میان دو محیط تغییر نکرده بود. در محیط متفاوت تنها ۲۹,۹ درصد جدول مبدأ این شرط را داشت.

نمونه انتقال منفی

یک نمونه واقعی از انتقال منفی مربوط به حالت زیر است:

$$s = (2,6,0,37)$$

در محیط مبدأ، عمل RIGHT بالاترین Q را داشت:

$$Q_{source}(RIGHT) = 51.2454$$

درحالی که بهترین گزینه بعدی فقط:

21.3676

بود.

اما در محیط مقصد مشابه، خانه (2,7) که عامل با RIGHT وارد آن می‌شد به دیوار تبدیل شده بود. بنابراین، سیاست انتقال یافته عامل را به یک عمل نامناسب هدایت می‌کرد.

پس از ادامه آموزش در مقصد:

$$Q_target(RIGHT) = 21.9472$$

شد، درحالی‌که:

$$Q_target(UP) = 34.8665$$

و عمل حریمانه از RIGHT به UP تغییر کرد.

این نمونه نشان می‌دهد که Q-Learning می‌تواند دانش منتقل شده اشتباه را با تجربه جدید اصلاح کند، اما این اصلاح فقط زمانی رخ می‌دهد که حالت موردنظر در محیط مقصد به اندازه کافی دوباره مشاهده شود.

در نتیجه، محیط مشابه از انتقال گسترده دانش سود بیشتری برد، درحالی‌که در محیط متفاوت انتقال انتخابی مؤثرتر بود. همچنین، نتایج نشان دادند انتقال می‌تواند در مراحل مختلف اثرهای متفاوتی داشته باشد؛ برای نمونه Full Transfer در محیط متفاوت عملکرد اولیه را بدتر، سرعت یادگیری را بهتر و عملکرد نهایی را دوباره بدتر از Scratch کرد. بنابراین، برای ارزیابی انتقال یادگیری باید این سه معیار به صورت جداگانه بررسی شوند.

فصل سیزدهم: جمع‌بندی و نتیجه‌گیری

۱۳-۱. جمع‌بندی کلی پروژه

در این پروژه یک مسئله یادگیری تقویتی در قالب هزارتوی تصادفی طراحی و پیاده‌سازی شد که در آن عامل باید با انرژی محدود ابتدا کلید را پیدا کرده، سپس از در عبور کند و در نهایت به هدف برسد. وجود لغزش تصادفی در حرکت، دیوارها، خانه‌های جریمه، در قفل‌شده و محدودیت انرژی باعث شد مسئله از یک مسیریابی قطعی ساده فراتر رود و تصمیم‌گیری عامل علاوه بر فاصله، به ریسک و وضعیت جاری محیط نیز وابسته باشد.

مسئله به صورت یک MDP با حالت (r, c, k, e) مدل شد و سه الگوریتم **Q-Learning**، **Value Iteration** و **SARSA(λ)** از ابتدا پیاده‌سازی و تحت شرایط مشترک ارزیابی شدند. علاوه بر مقایسه این الگوریتم‌ها، اثر طراحی تابع پاداش، برنامه کاهش اکتشاف، **Eligibility Trace** و انتقال یادگیری نیز به صورت تجربی بررسی شد. در نهایت، رابط گرافیکی برای مشاهده تعاملی رفتار عامل و سیاست‌های حاصل توسعه داده شد.

۱۳-۲. نتایج اصلی الگوریتم‌ها

Value Iteration به عنوان روش مدل‌محور و سیاست مرجع پروژه مورد استفاده قرار گرفت. این الگوریتم برای $\gamma=0.99$ پس از ۵۱ sweep همگرا شد و در ارزیابی اصلی نرخ موفقیت ۹۳ درصد را به دست آورد. نتایج همچنین نشان دادند که تغییر γ اثر قابل توجهی بر مقدار تابع ارزش دارد، اما ساختار سیاست نسبتاً پایدار باقی می‌ماند. برای مثال، سیاست حاصل از $\gamma=0.90$ همچنان در بیش از ۹۳ درصد حالت‌ها با سیاست $\gamma=0.99$ یکسان بود.

در **Q-Learning** مشخص شد که برنامه کاهش اکتشاف تأثیر زیادی بر عملکرد دارد. در شرایط یکسان، استفاده از کاهش نمایی ϵ نسبت به کاهش خطی باعث افزایش نرخ موفقیت از ۵۴ درصد به ۶۷،۶۴ درصد در حالت پاداش تنک شد. همچنین، کاهش نمایی باعث شد عامل در مراحل زودتری از آموزش به نرخ‌های موفقیت قابل توجه برسد. بر این اساس، مدل **Sparse + Exponential** به عنوان مدل اصلی **Q-Learning** انتخاب شد.

نتایج بررسی تابع پاداش نیز نشان دادند که **Reward Shaping** باعث شروع زودتر یادگیری، به‌ویژه در ترکیب با کاهش نمایی ϵ ، می‌شود؛ اما این مزیت الزاماً به عملکرد نهایی بهتر منجر نمی‌شود. در ارزیابی نهایی، مدل دارای پاداش تنک موفقیت بیشتری نسبت به نسخه شکل‌دهی شده داشت. بنابراین، پاداش تنک برای آزمایش‌های اصلی و مقایسه الگوریتم‌ها حفظ شد.

در $SARSA(\lambda)$ ، استفاده از Eligibility Trace متوسط باعث بهبود قابل توجه عملکرد شد. در اجرای اصلی، با $\lambda=0.3$ نرخ موفقیت ۸۵٫۳۳ درصد بهترین نتیجه را ایجاد کرد. در آزمایش‌های پنج‌گانه $\lambda=0.7$ با میانگین موفقیت ۸۶٫۶۷ درصد و رسیدن سریع‌تر به نرخ موفقیت ۳۰ درصد، بهترین نتیجه آماری را داشت؛ هرچند هزینه محاسباتی آن بیشتر بود. در مقابل، $\lambda=0.9$ در تمام اجراها شکست خورد و نرخ موفقیت آن به صفر رسید. این نتیجه نشان داد که افزایش طول Eligibility Trace الزاماً مفید نیست و انتقال اعتبار بیش از حد طولانی می‌تواند یادگیری را ناپایدار کند.

۳-۱۳. مقایسه نهایی روش‌های اصلی

مقایسه سه روش نشان داد که Value Iteration در صورت دسترسی به مدل کامل محیط بهترین عملکرد را ارائه می‌کند. با این حال، این روش نیازمند پردازش تمام فضای حالت و دانستن احتمال انتقال‌هاست.

در میان روش‌های مدل‌آزاد، $SARSA(\lambda)$ عملکرد بهتری از Q-Learning به دست آورد. در اجرای اصلی:

Value Iteration = 93.00%
 $SARSA(\lambda=0.3)$ = 85.33%
 Q-Learning = 64.67%

بود.

Q-Learning از نظر ساختار و هزینه محاسباتی ساده‌تر بود، اما برای رسیدن به یک سیاست مناسب به تجربه بیشتری نیاز داشت و سیاست نهایی آن نیز در محیط تصادفی شکننده‌تر بود. $SARSA$ با در نظر گرفتن سیاست رفتاری واقعی و استفاده از Eligibility Trace توانست انتقال اعتبار مؤثرتری ایجاد کند و در اکثر اجراها نرخ موفقیت بالاتری داشته باشد.

در عین حال، بررسی سیاست‌ها نشان داد درصد توافق حالت‌به‌حالت با Value Iteration به‌تنهایی معیار مناسبی برای قضاوت نیست. Q-Learning حدود ۵۰٫۱۸ درصد و $SARSA$ حدود ۴۸٫۳۹ درصد توافق با سیاست مرجع داشتند، اما $SARSA$ نرخ موفقیت بسیار بیشتری کسب کرد. تحلیل حالت‌های نمونه نشان داد برخی اختلاف‌های سیاست میان اعمال تقریباً هم‌ارزش اتفاق می‌افتند، درحالی‌که تعداد کمتری از اختلاف‌ها در موقعیت‌های حساس می‌توانند تأثیر زیادی بر موفقیت اپیزود داشته باشند.

۴-۱۳. نتایج انتقال یادگیری

آزمایش انتقال یادگیری نشان داد که مفید بودن دانش قبلی به میزان شباهت ساختاری میان محیط مبدأ و مقصد وابسته است.

در محیط مشابه که حدود ۱۷,۹۵ درصد موانع جابه‌جا شده بودند و محل کلید ثابت مانده بود، انتقال کامل جدول Q بسیار مفید بود. نرخ موفقیت نهایی از ۶ درصد در آموزش از صفر به ۴۶ درصد در انتقال کامل افزایش یافت. بنابراین، بخش بزرگی از دانش آموخته‌شده در محیط مبدأ همچنان در محیط جدید قابل استفاده بود.

در محیط متفاوت، شرایط کاملاً متفاوت بود. بیش از ۴۱ درصد موانع تغییر کردند، کلید جابه‌جا شد و خانه‌های جریمه جدید به نقشه اضافه شدند. در این حالت، انتقال کامل در ابتدا بازده عامل را کاهش داد و نمونه مشخصی از انتقال منفی ایجاد شد. با وجود آنکه دانش قبلی در ادامه باعث افزایش سرعت یادگیری شد، عملکرد نهایی Full Transfer با ۶۶,۳۳ درصد همچنان کمتر از آموزش از صفر با ۷۳,۳۳ درصد باقی ماند.

بهترین روش در محیط متفاوت انتقال انتخابی بود. در این روش فقط حدود ۲۹,۰۹ درصد حالت‌هایی که ساختار محلی آن‌ها تغییر نکرده بود منتقل شدند و نرخ موفقیت نهایی به ۷۴ درصد رسید.

بنابراین، نتایج نشان دادند که انتقال بیشتر لزوماً بهتر نیست. در محیط‌های نزدیک به مبدأ، انتقال گسترده مناسب‌تر است؛ اما با افزایش تغییرات ساختاری، انتخاب دانش معتبر و جلوگیری از انتقال اطلاعات ناسازگار اهمیت بیشتری پیدا می‌کند.

۵-۱۳. نکات حاصل از تحلیل تجربی

یکی از نتایج مهم پروژه این بود که معیارهای مختلف یادگیری همیشه به یک نتیجه واحد منجر نمی‌شوند. یک روش ممکن است یادگیری را زودتر آغاز کند، اما در پایان بهترین سیاست را نداشته باشد. Reward Shaping نمونه‌ای از این وضعیت بود؛ همان‌طور که در انتقال یادگیری نیز Full Transfer در محیط متفاوت عملکرد اولیه را کاهش داد، سرعت یادگیری را افزایش داد و در نهایت دوباره عملکردی پایین‌تر از Scratch ایجاد کرد.

همچنین مشخص شد پارامترهای الگوریتم‌ها نقش تعیین‌کننده‌ای در نتیجه دارند. انتخاب برنامه کاهش ϵ بیش از ۱۰ واحد درصد اختلاف در موفقیت Q-Learning ایجاد کرد و تغییر λ در SARSA عملکرد را از بیش از ۸۵ درصد به صفر رساند. در نتیجه، ارزیابی یک الگوریتم بدون بررسی حساسیت پارامتری می‌تواند تصویری ناقص از عملکرد واقعی آن ارائه دهد.

محدودیت انرژی نیز نقش مهمی در مسئله داشت. عامل فقط ۵۰ گام فرصت دارد، درحالی‌که کوتاه‌ترین مسیر قطعی مبدأ ۳۰ گام است. بنابراین، سیاستی که مسیر کمی طولانی‌تر یا شکننده‌تر انتخاب می‌کند، در حضور

لغزش‌های تصادفی ممکن است به آسانی انرژی خود را پیش از رسیدن به هدف از دست بدهد. این موضوع به‌ویژه در تفاوت عملکرد Q-Learning و SARSA قابل مشاهده بود.

۶-۱۳. محدودیت‌های پروژه

اگرچه آزمایش‌های انجام‌شده جنبه‌های مختلف سه الگوریتم را پوشش دادند، پروژه دارای محدودیت‌هایی نیز هست. نخست، هر سه الگوریتم به‌صورت **جدولی** پیاده‌سازی شده‌اند. این روش برای فضای حالت ۲۰,۸۵۰ حالتۀ فعلی مناسب است، اما در محیط‌های بزرگ‌تر یا دارای حالت‌های پیوسته، تعداد مقادیر موردنیاز به‌سرعت افزایش پیدا می‌کند.

دوم، آزمایش‌های اصلی روی یک ساختار هزارتوی مشخص انجام شده‌اند. اگرچه دو محیط مقصد و چند seed مختلف برای بررسی تعمیم و پایداری استفاده شدند، نتایج عددی دقیق الزاماً به تمام انواع محیط‌های مسیریابی قابل تعمیم نیستند.

سوم، نرخ یادگیری در Q-Learning و SARSA ثابت در نظر گرفته شده است. استفاده از برنامه کاهش نرخ یادگیری یا روش‌های سازگارتر ممکن است خطاهای باقی‌مانده Q در برخی حالت‌ها را کاهش دهد.

همچنین انتقال انتخابی بر اساس یک معیار نسبتاً ساده، یعنی یکسان‌بودن همسایگی محلی 3×3 ، انجام شد. این معیار در محیط متفاوت نتیجه مناسبی ایجاد کرد، اما شباهت واقعی دو حالت می‌تواند علاوه بر ساختار هندسی به پاداش، فاصله از زیرهدف، احتمال انتقال و اهمیت آن حالت در سیاست نیز وابسته باشد.

۷-۱۳. پیشنهادهایی برای توسعه آینده

برای ادامه این پروژه می‌توان فضای حالت و اندازه نقشه را افزایش داد و روش‌های مبتنی بر تقریب تابع را جایگزین جدول Q کرد. استفاده از **Deep Q-Network (DQN)** می‌تواند بررسی کند که آیا سیاست قابل تعمیم‌تری برای نقشه‌های بزرگ‌تر قابل یادگیری است یا خیر.

در بخش SARSA نیز می‌توان اثر نرخ یادگیری متغیر، روش‌های مختلف Eligibility Trace و الگوریتم‌های چندمرحله‌ای مانند n-step SARSA را بررسی کرد. مقایسه این روش‌ها می‌تواند مشخص کند چه مقدار انتقال اعتبار برای مسئله‌های مختلف مناسب‌تر است.

در انتقال یادگیری، می‌توان به‌جای معیار محلی ثابت، یک معیار شباهت برای حالت‌ها تعریف کرد و شدت انتقال را به‌طور خودکار براساس میزان شباهت تعیین نمود. در این صورت، به‌جای انتخاب یک β ثابت برای تمام جدول، هر حالت می‌تواند ضریب انتقال متفاوتی داشته باشد.

از سوی دیگر، اجرای آزمایش‌ها روی مجموعه بزرگ‌تری از نقشه‌های تصادفی می‌تواند ارزیابی دقیق‌تری از قابلیت تعمیم الگوریتم‌ها ارائه دهد. در چنین حالتی می‌توان علاوه بر میانگین عملکرد، فاصله اطمینان و آزمون‌های آماری مناسب را نیز گزارش کرد.

۸-۱۳. نتیجه‌گیری نهایی

این پروژه نشان داد که یک مسئله ظاهراً ساده مسیریابی، با اضافه‌شدن تصادفی‌بودن حرکت، زیرهدف کلید، در قفل‌شده، خانه‌های جریمه و انرژی محدود، می‌تواند زمینه مناسبی برای مقایسه عملی مفاهیم اصلی یادگیری تقویتی ایجاد کند. Value Iteration در شرایط دسترسی کامل به مدل، دقیق‌ترین سیاست را تولید کرد. در نبود مدل، SARSA با Eligibility Trace متوسط در این مسئله عملکرد بهتری نسبت به Q-Learning داشت. همچنین مشخص شد طراحی تابع پاداش، برنامه اکتشاف و مقدار λ می‌توانند به‌اندازه انتخاب خود الگوریتم در نتیجه نهایی مؤثر باشند.

بخش انتقال یادگیری نیز نشان داد دانش قبلی تنها در صورتی مفید است که با ساختار محیط جدید سازگاری کافی داشته باشد. انتقال کامل در محیط مشابه عملکرد را به‌شدت بهبود داد، اما در محیط متفاوت باعث انتقال منفی شد و انتقال انتخابی نتیجه مطمئن‌تری ایجاد کرد.

در مجموع، نتایج پروژه اهمیت سه موضوع را به‌طور تجربی نشان دادند: انتخاب نمایش صحیح حالت، ارزیابی الگوریتم براساس چند معیار به‌جای یک معیار منفرد، و توجه به اعتبار دانشی که میان محیط‌ها منتقل می‌شود. ترکیب تحلیل عددی، بررسی سیاست‌ها، آزمایش پایداری، انتقال یادگیری و رابط گرافیکی نیز امکان ارزیابی مسئله را هم از دید کمی و هم از دید رفتاری فراهم کرد.

پیوست‌ها

در این بخش، تنظیمات نهایی آزمایش‌ها، ساختار مخزن، نحوه بازتولید نتایج، فایل‌های خروجی و آزمون‌های صحت پیاده‌سازی ارائه می‌شوند. همچنین مطابق الزام صورت مسئله، نحوه استفاده از ابزارهای هوش مصنوعی و نمونه‌هایی از پیشنهادهای ناقص یا نیازمند اصلاح نیز ثبت شده است. صورت مسئله تأکید می‌کند که داده‌های خام، تنظیمات، مدل‌ها، نمودارها و روش اجرای پروژه باید در مخزن قابل ردیابی باشند و استفاده از ابزارهای هوش مصنوعی نیز به‌صورت شفاف گزارش شود.

پیوست الف: تنظیمات نهایی محیط و آزمایش‌ها

الف-۱. پارامترهای محیط

جدول الف-۱ مهم‌ترین پارامترهای محیط مبدأ را خلاصه می‌کند.

پارامتر	مقدار
شناسه دانشجویی	40301694
Seed پایه	9
اندازه هزار تو	16×16
تعداد کل خانه‌ها	256
تعداد دیوارها	46
نسبت دیوارها	17.97%
تعداد خانه‌های جریمه	7
نقطه شروع	(0,0)
موقعیت کلید	(0,10)
موقعیت در	(13,14)
موقعیت هدف	(14,14)
انرژی اولیه	50
احتمال اجرای عمل موردنظر	0.8
احتمال لغزش به هر جهت عمود	0.1
کوتاه‌ترین مسیر شروع تا کلید	12 گام
کوتاه‌ترین مسیر کلید تا هدف	18 گام

30 گام	کوتاه‌ترین مسیر کامل
630	حداکثر تعداد گام نظری
Up, Down, Left, Right	فضای عمل
(row, col, key, energy)	نمایش حالت

جدول الف-۱. مشخصات محیط اصلی

کدگذاری خانه‌های نقشه نیز در پیاده‌سازی به صورت جدول الف-۲ است.

نوع خانه	مقدار
خانه عادی	0
دیوار	1
خانه جریمه	2
شروع	3
کلید	4
در	5
هدف	6

جدول الف-۲. کد عددی خانه‌های نقشه

الف-۲. پارامترهای Value Iteration

پارامتر	مقدار
نقشه	source_map.json
تابع پاداش	Sparse
مقادیر γ	0.90, 0.95, 0.99
آستانه همگرایی θ	10-6
حداکثر Sweep	500
مقدار اولیه V	0
مدل مرجع نهایی	$\gamma=0.99$

جدول الف-۳. تنظیمات آزمایش Value Iteration

فایل تنظیمات:

experiments/configs/value_iteration.json

الف-۳. پارامترهای Q-Learning

پارامتر	مقدار
تعداد اپیزود	150,000
نرخ یادگیری α	0.2
ضریب تنزیل γ	0.99
ϵ اولیه	1.0
ϵ نهایی	0.05
بخش کاهش ϵ	80% آموزش
پایان کاهش ϵ	اپیزود ۱۲۰,۰۰۰
برنامه‌های کاهش	Linear, Exponential
توابع پاداش	Sparse, Shaped
تعداد اپیزود ارزیابی	300
Seed اصلی	9
مدل منتخب	Sparse + Exponential

جدول الف-۴. تنظیمات اصلی Q-Learning

در نتیجه چهار آزمایش اصلی اجرا شدند:

Sparse + Linear
 Sparse + Exponential
 Shaped + Linear
 Shaped + Exponential

فایل تنظیمات:

experiments/configs/q_learning.json

الف-۴. پارامترهای SARSA(λ)

پارامتر	مقدار
تعداد اپیزود	150,000
α	0.2
γ	0.99
ϵ اولیه	1.0

0.05	ϵ نهایی
Exponential	برنامه کاهش ϵ
Sparse	تابع پاداش
0, 0.3, 0.7, 0.9	λ های اصلی
0.75, 0.8, 0.85	λ های تکمیلی
Replacing	نوع Trace اصلی
$\lambda=0.9$	آزمایش Accumulating
10-4	آستانه حذف Trace
300	تعداد اپیزود ارزیابی
$\lambda=0.3$	مدل منتخب اصلی

جدول الف-۵. تنظیمات $SARSA(\lambda)$

فایل تنظیمات:

experiments/configs/sarsa_lambda.json

الف-۵. تنظیمات آزمایش پایداری

برای بررسی حساسیت نتایج به تصادفی بودن آموزش، پنج Seed مستقل استفاده شدند:

{9,17,23,42,101}

پیکربندی‌های بررسی شده عبارت بودند از Q-Learning و SARSA با مقادیر:

$\lambda = \{0,0.3,0.7,0.9\}$

هر اجرا شامل ۱۵۰۰۰۰ اپیزود آموزش و ۳۰۰ اپیزود ارزیابی بود. آستانه مورد استفاده برای سنجش سرعت یادگیری نیز نرخ موفقیت متحرک ۳۰ درصد تعیین شد.

فایل تنظیمات:

experiments/configs/stability.json

الف-۶. تنظیمات انتقال یادگیری

پارامتر	مقدار
مدل مبدأ	ql_sparse_exponential_q.pkl
تابع پاداش	Sparse
تعداد اپیزود مقصد	60,000
α	0.2
γ	0.99
ϵ اولیه	1.0
ϵ نهایی	0.05
β های آزمایش شده	0.25, 0.50, 0.75
شعاع انتقال انتخابی	1
اپیزودهای ارزیابی	300
Seed اصلی	9
Seed محیط مشابه	91
Seed محیط متفاوت	92

جدول الف-۶. تنظیمات Transfer Learnin

چهار نوع راهبرد اصلی عبارتند از:

Scratch
Full Transfer
Scaled Transfer
Selective Transfer

مشخصات دو محیط مقصد نیز در جدول الف-۷ آمده است.

ویژگی	Similar	Different
موانع جابه جاشده	7	16
نسبت تغییر موانع	17.95%	41.03%
کل خانه های تغییر کرده	14	36
کلید جابه جا شده	خیر	بله
خانه جریمه جدید	0	4
موقعیت کلید	(0,10)	(5,6)
مسیر شروع تا کلید	10	11
مسیر کلید تا هدف	22	17

فایل تنظیمات:

experiments/configs/transfer.json

پیوست ب: ساختار مخزن پروژه

ساختار نهایی مخزن به صورت زیر است:

```
RL_FinalProject_40301694/
├── environments/
│   ├── generator.py
│   ├── maze.py
│   └── maps/
│       ├── source_map.json
│       ├── target_similar.json
│       └── target_different.json
├── agents/
│   ├── value_iteration.py
│   ├── q_learning.py
│   └── sarsa_lambda.py
├── transfer/
│   └── transfer_learning.py
├── gui/
│   ├── app.py
│   └── renderer.py
├── experiments/
│   ├── run_value_iteration.py
│   ├── run_q_learning.py
│   ├── run_sarsa_lambda.py
│   ├── run_stability.py
│   ├── run_transfer.py
│   ├── analysis.py
│   └── configs/
│       ├── value_iteration.json
│       ├── q_learning.json
│       ├── sarsa_lambda.json
│       ├── stability.json
│       └── transfer.json
├── results/
│   └── raw_data/
```

```

├── models/
│   └── figures/
├── tests/
│   └── test_environment.py
├── main.py
├── requirements.txt
└── README.md

```

این ساختار منطق جداسازی محیط، عامل‌ها، انتقال یادگیری، رابط گرافیکی، آزمایش‌ها و خروجی‌ها را حفظ می‌کند. چنین جداسازی‌ای در صورت مسئله نیز به‌عنوان ساختار پیشنهادی تحویل در نظر گرفته شده است.

پیوست ج: نصب و بازتولید نتایج

ج-۱. نیازمندی‌های نرم‌افزاری

پروژه با Python نسخه ۳.۱۰ یا بالاتر قابل اجرا است و در توسعه اصلی با Python 3.12 نیز آزمایش شده است.

کتابخانه‌های موردنیاز عبارت‌اند از:

```

numpy>=1.24
matplotlib>=3.7
pandas>=2.0
pytest>=7.4

```

نصب وابستگی‌ها با دستور زیر انجام می‌شود:

```
pip install -r requirements.txt
```

کتابخانه tkinter برای GUI استفاده شده است ولی از طریق pip نصب نمی‌شود و باید همراه Python یا از طریق package manager سیستم‌عامل نصب شده باشد.

ج-۲. ترتیب بازتولید نتایج

برای بازتولید کامل پروژه، دستورات زیر از مسیر اصلی repository به‌ترتیب اجرا می‌شوند:

```

pytest tests/ -v
python environments/generator.py
python experiments/run_value_iteration.py

```

```
python experiments/run_q_learning.py
python experiments/run_sarsa_lambda.py
python experiments/run_transfer.py
python experiments/analysis.py
```

و برای اجرای رابط گرافیکی:

```
python main.py
```

خروجی هر مرحله در پوشه مربوط به خود ذخیره می‌شود. تنظیمات در:

```
experiments/configs/
```

داده‌های خام در:

```
results/raw_data/
```

مدل‌ها در:

```
results/models/
```

و نمودارها در:

```
results/figures/
```

قرار می‌گیرند.

بنابراین، اعداد استفاده‌شده در گزارش از خروجی آزمایش‌ها قابل ردیابی هستند و نیازی به واردکردن دستی مقادیر در نمودارها وجود ندارد. این موضوع با الزام صورت مسئله مبنی بر تولید نمودارها از داده‌های خام و قابل ردیابی بودن تنظیمات و زمان اجرا مطابقت دارد.

پیوست د: فایل‌های اصلی خروجی

د-۱. خروجی‌های Value Iteration

داده‌های عددی:

```
results/raw_data/vi_summary.csv
results/raw_data/vi_convergence.csv
results/raw_data/vi_policy_evaluation.csv
```

مدل‌ها:

```
results/models/vi_gamma_0.9.pkl  
results/models/vi_gamma_0.95.pkl  
results/models/vi_gamma_0.99.pkl
```

مهم‌ترین نمودارها:

```
results/figures/vi_convergence.png  
results/figures/vi_heatmap_gamma0.9_k0.png  
results/figures/vi_heatmap_gamma0.9_k1.png  
results/figures/vi_heatmap_gamma0.95_k0.png  
results/figures/vi_heatmap_gamma0.95_k1.png  
results/figures/vi_heatmap_gamma0.99_k0.png  
results/figures/vi_heatmap_gamma0.99_k1.png
```

د-۲. خروجی‌های Q-Learning

خلاصه نتایج:

```
results/raw_data/ql_summary.csv
```

برای هر پیکربندی، سه نوع فایل اصلی ذخیره شده‌اند:

```
*_episodes.csv  
*_steps_sampled.csv  
*_qupdates.csv
```

برای مثال:

```
ql_sparse_exponential_episodes.csv  
ql_sparse_exponential_steps_sampled.csv  
ql_sparse_exponential_qupdates.csv
```

مدل منتخب:

```
results/models/ql_sparse_exponential_q.pkl
```

نمودارهای اصلی:

```
ql_epsilon_schedules.png  
ql_learning_curves_success.png  
ql_learning_curves_reward.png  
ql_learning_curves_steps.png
```

د-۳. خروجی‌های SARSA(λ)

خلاصه:

results/raw_data/sarsa_summary.csv

برای هر مقدار λ :

*_episodes.csv
*_steps_sampled.csv
*_traces.csv

مدل اصلی:

results/models/sarsa_lam0.3_replacing_q.pkl

نمودارهای مهم:

sarsa_lambda_final_comparison.png
sarsa_trace_dynamics.png
sarsa_learning_curves_success.png
sarsa_learning_curves_reward.png
sarsa_learning_curves_penalty.png

د-۴. خروجی‌های مقایسه و پایداری

results/raw_data/stability_runs.csv
results/raw_data/stability_aggregate.csv
results/raw_data/memory_footprint.csv

و نمودارهای:

stability_success.png
stability_curves_q_learning.png
stability_curves_sarsa_lam0.0.png
stability_curves_sarsa_lam0.3.png
stability_curves_sarsa_lam0.7.png
stability_curves_sarsa_lam0.9.png
analysis_final_paths.png
analysis_ql_policy_diff.png
analysis_sarsa_policy_diff.png

د-۵. خروجی‌های انتقال یادگیری

خلاصه اصلی:

results/raw_data/transfer_summary.csv

نمونه انتقال منفی:

results/raw_data/transfer_negative_example.csv

برای هر سناریو نیز فایل‌های episode ، step و Q-update جداگانه ذخیره شده‌اند.

نمودارهای اصلی:

transfer_maps_comparison.png
transfer_curves_similar.png
transfer_curves_different.png
transfer_jumpstart_vs_final.png
analysis_transfer_q_diff.png

مدل‌های آموزش‌دیده نیز با نام‌هایی مانند:

transfer_similar_full_q.pkl
transfer_similar_selective_q.pkl
transfer_different_full_q.pkl
transfer_different_selective_q.pkl

در پوشه results/models ذخیره شده‌اند.

پیوست ه: آزمون‌های صحت پیاده‌سازی

برای بررسی مستقل رفتار محیط، مجموعه‌ای از Unit Test ها در فایل زیر پیاده‌سازی شده است:

tests/test_environment.py

در نسخه نهایی پروژه ۱۷ آزمون وجود دارد. اجرای:

pytest tests/ -q

نتیجه زیر را تولید می‌کند:

17 passed

آزمون‌ها موارد اصلی زیر را پوشش می‌دهند:

بخش مورد آزمایش	هدف
Seed و اندازه نقشه	صحت استفاده از شناسه دانشجویی
نسبت دیوار و جریمه	رعایت محدودیت‌های ساخت نقشه
BFS validity	وجود مسیر معتبر کلید و هدف
Reproducibility	تولید یکسان نقشه برای Seed یکسان
Reset	صحت حالت اولیه
Transition probabilities	مجموع احتمال انتقال برابر یک
Energy decrement	کاهش انرژی در تمام اعمال
Wall hit	باقی ماندن در مکان و اعمال جریمه
Key pickup	تغییر متغیر کلید و پاداش صحیح
Locked door	مسدود بودن در بدون کلید
Door pass	امکان عبور پس از دریافت کلید
Goal	پایان موفق و پاداش هدف
Energy depletion	پایان اپیزود با انرژی صفر
Markov property	وابستگی مدل فقط به حالت و عمل فعلی
Reward shaping	سازگاری Sparse و Potential term
Step limit	عملکرد صحیح Truncation
Episode reproducibility	تکرار یکسان اپیزود با Seed یکسان

جدول ه-۱. پوشش آزمون‌های محیط

این آزمون‌ها به‌طور خاص برای بررسی منطق محیط طراحی شده‌اند و از کتابخانه آماده Reinforcement Learning برای پیاده‌سازی الگوریتم‌ها استفاده نشده است. استفاده از الگوریتم‌های RL آماده نیز در صورت مسئله ممنوع اعلام شده و NumPy، Pandas، Matplotlib و کتابخانه رابط گرافیکی مجاز دانسته شده‌اند.

پیوست و: استفاده از ابزارهای هوش مصنوعی

ابزارهای هوش مصنوعی در این پروژه به‌عنوان دستیار برای بررسی کد، یافتن اشکال، پیشنهاد ساختار تحلیل، تفسیر نتایج و ویرایش نگارشی گزارش استفاده شدند. نتایج عددی نهایی از فایل‌های خام پروژه استخراج و

پیشنهادها با کد، CSVها و اجرای آزمایش‌ها بررسی شدند. هیچ الگوریتم آماده RL به‌عنوان جایگزین پیاده‌سازی پروژه استفاده نشده است.

مورد استفاده	پیشنهاد یا خروجی اولیه ابزار	بررسی و تغییر انجام‌شده	دلیل تغییر
بررسی و اشکال‌زدایی کد	پیشنهادهایی برای بررسی تعریف حالت، Transition و Reward	پیشنهادها با <code>Unit maze.py</code> Testها مقایسه و فقط موارد سازگار اعمال شدند	جلوگیری از تغییر منطق واقعی محیط و حفظ خاصیت مارکوف
تحلیل نتایج-Q Learning و SARSA	پیشنهاد استفاده از معیارهایی مانند Success Rate، Return و Learning Curve	مقادیر نهایی مستقیماً از CSVها استخراج و تحلیل‌ها براساس داده واقعی نوشته شدند	جلوگیری از گزارش عدد بدون فایل اجرای متناظر
تهیه متن گزارش	پیشنهاد ساختار فصل‌ها و توضیح الگوریتم‌ها	بخش‌های تکراری حذف و تمرکز فصل‌های نتایج روی داده‌های واقعی قرار گرفت	جلوگیری از افزایش بیش از حد حجم گزارش و تکرار مطالب نظری
پیشنهاد ناقص : بررسی مخزن	در یک بررسی اولیه براساس نسخه ناقص/قدیمی مخزن نتیجه‌گیری شد که README یا بخشی از ساختار پروژه کامل نیست	نسخه ZIP نهایی پروژه مستقیماً بررسی شد و ارزیابی براساس فایل‌های واقعی اصلاح گردید	نتیجه اولیه بر مبنای منبع ناقص بود و با نسخه نهایی repository مطابقت نداشت
تحلیل Reward Shaping	پیشنهاد بررسی Return در کنار Success Rate	در مقایسه Sparse و Shaped ، Return خام به‌عنوان معیار مستقیم مقایسه کنار گذاشته شد و Success Rate مبنای اصلی قرار گرفت	مقیاس پاداش در دو تابع متفاوت است و Return خام بین آن‌ها مستقیماً قابل مقایسه نیست

انتخاب گزارش	تصاویر	پیشنهاد استفاده از مجموعه‌ای از Heatmapها و Learning Curveها	فقط تصاویر ضروری در متن اصلی قرار گرفتند و نمودارهای مشابه حذف شدند	جلوگیری از شلوغ شدن گزارش و تکرار بصری
بررسی یادگیری	انتقال	پیشنهاد تحلیل Full ، Scaled و Selective Transfer	عملکرد اولیه، سرعت یادگیری و عملکرد نهایی به صورت جداگانه بررسی شدند	یک روش ممکن است در مراحل مختلف هم اثر مثبت و هم منفی داشته باشد

جدول و-۱. موارد استفاده از ابزار هوش مصنوعی و اصلاحات انجام شده

این جدول نشان می‌دهد خروجی ابزار هوش مصنوعی مستقیماً به عنوان نتیجه نهایی پذیرفته نشده است و پیشنهادها در موارد لازم با کد و داده‌های واقعی کنترل و اصلاح شده‌اند.

پیوست ز: منابع اصلی مورد استفاده

برای مفاهیم اصلی یادگیری تقویتی و الگوریتم‌های مورد استفاده در پروژه، منابع زیر قابل درج در بخش منابع نهایی گزارش هستند:

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.

[2] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," *Proceedings of the 16th International Conference on Machine Learning (ICML)*, 1999.

منبع اول برای مفاهیم MDP، Value Iteration، Temporal-Difference Learning، Q-Learning، SARSA و Eligibility Traces مورد استفاده قرار گرفته است. منبع دوم نیز مبنای نظری-Potential Based Reward Shaping استفاده شده در پروژه است.